

الحمد لله
الرحمن الرحيم



معاونت پژوهش و فن آوری

به نام خدا

مشور اخلاق پژوهش

بایاری از خداوند سبحان و اعتقاد به اینکه عالم محضر خداست و به‌مواره ناظر بر اعمال انسان و به‌منظور پاسداشت مقام بلند دانش و پژوهش و نظریه اهمیت جایگاه دانشگاه در اعتلای فرهنگ و تمدن بشری و دانشجویمان و اعطاء هیئت علمی واحدهای دانشگاه آزاد اسلامی متعهد می‌گردیم اصول زیر را در انجام فعالیت‌های پژوهشی مد نظر قرار داده و از آن تخطی نکنیم:

- ۱- اصل حقیقت‌جویی: تلاش در راستای پی‌جویی حقیقت و وفاداری به آن و دوری از هرگونه پنهان‌سازی حقیقت.
- ۲- اصل رعایت حقوق: التزام به رعایت کامل حقوق پژوهشگران و پژوهیدگان (انسان، حیوان و نبات) و سایر صاحبان حق.
- ۳- اصل مالکیت مادی و معنوی: تعهد به رعایت کامل حقوق مادی و معنوی دانشگاه و کلیه همکاران پژوهش.
- ۴- اصل منافع ملی: تعهد به رعایت مصالح ملی و در نظر داشتن پیشبرد و توسعه کشور در کلیه مراحل پژوهش.
- ۵- اصل رعایت انصاف و امانت: تعهد به اجتناب از هرگونه جانب‌داری غیر علمی و حفاظت از اموال، تجهیزات و منابع در اختیار.
- ۶- اصل رازداری: تعهد به صیانت از اسرار و اطلاعات محرمانه افراد، سازمان‌ها و کشور و کلیه افراد و ستادهای مرتبط با تحقیق.
- ۷- اصل احترام: تعهد به رعایت حریم‌ها و حرمت‌ها در انجام تحقیقات و رعایت جانب‌نقد و خودداری از هرگونه حرمت‌شکنی.
- ۸- اصل ترویج: تعهد به رواج دانش و اشاعه نتایج تحقیقات و انتقال آن به همکاران علمی و دانشجویمان به غیر از مواردی که منع قانونی دارد.
- ۹- اصل برانست: التزام به برانست‌جویی از هرگونه رفتار غیر حرفه‌ای و اعلام موضع نسبت به کسانی که حوزه علم و پژوهش را به‌شائبه‌های غیر علمی می‌آلایند.



دانشگاه آزاد اسلامی

واحد مشهد

تعهد اصالت رساله یا پایان نامه تحصیلی

اینجانب امیرحسین بانوی دانش آموخته مقطع کارشناسی ارشد در رشته مهندسی کامپیوتر گرایش نرم افزار که در تاریخ از پایان نامه/رساله خود تحت عنوان «مدیریت سرمایه گذاری املاک و مستغلات بر بستر بلاکچین و بر اساس قرارداد هوشمند و NFT» با کسب نمره و درجه دفاع نموده ام بدینوسیله متعهد می شوم:

- 1) این پایان نامه/رساله حاصل تحقیق و پژوهش انجام شده توسط اینجانب بوده و در مواردی که از دستاوردهای علمی و پژوهشی دیگران (اعم از پایان نامه، کتاب، مقاله و...) استفاده نموده ام، مطابق ضوابط و رویه موجود، نام منبع مورداستفاده و سایر مشخصات آن را در فهرست مربوطه ذکر و درج کرده ام.
- 2) این پایان نامه/رساله قبلاً برای هیچ مدرک تحصیلی (هم سطح، پایین تر یا بالاتر) در سایر دانشگاه ها و مؤسسات آموزش عالی ارائه نشده است.
- 3) چنانچه بعد از فراغت از تحصیل، قصد استفاده و هرگونه بهره برداری اعم از چاپ کتاب، ثبت اختراع و... از این رساله داشته باشم، از حوزه معاونت پژوهشی واحد مجوزهای مربوطه را اخذ نمایم.
- 4) چنانچه در هر مقطع زمانی خلاف موارد فوق ثابت شود، عواقب ناشی از آن را می پذیرم و واحد دانشگاهی مجاز است با اینجانب مطابق ضوابط و مقررات رفتار نموده و در صورت ابطال مدرک تحصیلی ام هیچ گونه ادعایی نخواهم داشت.

نام و نام خانوادگی:

تاریخ و امضا



دانشگاه آزاد اسلامی
واحد مشهد
دانشکده مهندسی، گروه کامپیوتر

پایان نامه برای دریافت درجه کارشناسی ارشد در رشته مهندسی کامپیوتر (M.Sc.)
گرایش نرم افزار

عنوان:

مدیریت سرمایه گذاری املاک و مستغلات بر بستر بلاکچین و بر اساس قرارداد
هوشمند و NFT

استاد راهنما:

دکتر داود بهره پور

استاد مشاور:

دکتر سیدرضا کامل

نگارش:

امیرحسین بانوی

تابستان ۱۴۰۳



صورت جلسه دفاع از پایان نامه کارشناسی ارشد (M.Sc.)

نام و نام خانوادگی دانشجو: امیرحسین بانوی در تاریخ رشته: مهندسی نرم افزار
از پایان نامه خود با عنوان: مدیریت سرمایه گذاری املاک و مستغلات بر بستر بلاکچین و بر اساس
قرارداد هوشمند و NFT
با درجه و نمره دفاع نموده است.

نام و نام خانوادگی اعضای هیات داور	سمت	امضاء اعضای هیات داور
------------------------------------	-----	-----------------------

۱- دکتر داود بهره پور	استاد (استادان) راهنما	
-----------------------	------------------------	--

۲- دکتر سیدرضا کامل	استاد (استادان) مشاور	
---------------------	-----------------------	--

۳- دکتر حسن شاکری	استاد داور	
-------------------	------------	--

۴-	استاد داور	
----	------------	--

مراتب فوق مورد تأیید است. مدیرکل امور آموزشی و تحصیلات تکمیلی

مهر و امضاء

سپاسگزاری

بدین وسیله از زحمات و تلاش بی دریغ اساتید محترم و پدر و مادر عزیزم صمیمانه سپاسگزاری می نمایم و همچنین از سایر همکاران و دوستانی که هر کدام به نحوی در تهیه این پایان نامه همکاری داشته اند تشکر نموده و موفقیت همه آنها را آرزومندم.

تقدیم به

تقدیم به پدران و مادران عزیزمان که الفبای انسانیت و چگونه زیستن را به ما آموختند.

فهرست مطالب

شماره صفحه	عنوان
1	چکیده.....
	فصل اول: مقدمه
3	1-1 مقدمه.....
3	2-1 بیان مسئله.....
4	3-1 اهداف تحقیق.....
4	4-1 اهمیت و ضرورت تحقیق.....
5	5-1 نوآوری.....
6	6-1 پرسش و فرضیه‌های تحقیق.....
7	7-1 دامنه مساله.....
7	8-1 مروری بر روش تحقیق.....
8	9-1 ساختار پایان‌نامه.....
	فصل دوم: مرور ادبیات
11	1-2 مقدمه.....
11	2-2 بلاکچین.....
11	1-2-2 معرفی فناوری بلاکچین.....
13	2-2-2 معماری بلاکچین.....
15	3-2-2 اجماع.....
18	4-2-2 مزایا و محدودیت‌های استفاده از بلاکچین.....
20	3-2 اتریوم.....
21	1-3-2 ماشین مجازی اتریوم (EVM).....
21	2-3-2 وب ۳ (Web3) و برنامه‌های کاربردی غیرمتمرکز (DApps).....
22	3-3-2 حساب‌های شبکه‌ی اتریوم.....
23	4-3-2 تراکنش.....
24	5-3-2 گاز و کارمزد.....
25	6-3-2 رویداد.....
25	4-2 قرارداد هوشمند.....
26	1-4-2 مقایسه قراردادهای هوشمند و قراردادهای قانونی مرسوم.....

26	5-2 توکن.....
27	1-5-2 استانداردهای توکن.....
29	2-5-2 توکنیزه کردن و مزایای آن.....
30	6-2 سازمان غیرمتمرکز خودگردان (DAO).....
31	7-2 مرور ادبیات بلاکچین در املاک و مستغلات.....
31	1-7-2 توکنیزه کردن املاک و مستغلات.....
33	2-7-2 سایر کاربردهای بلاکچین در املاک و مستغلات.....
35	3-7-2 یافته‌های تحقیقات موجود.....
36	4-7-2 شکاف‌ها و محدودیت‌های تحقیقات موجود.....
37	8-2 جمع‌بندی.....

فصل سوم: روش پیشنهادی

39	1-3 مقدمه.....
39	2-3 معرفی سامانه پیشنهادی.....
39	1-2-3 عرضه اولیه املاک.....
41	2-2-3 مدیریت و اجرای واحدهای تقسیم شده.....
41	3-2-3 مدیریت و جمع‌آوری هزینه سرانه‌ی خدمات ساختمان.....
42	3-3 بلاکچین و فناوری‌های مورد استفاده.....
42	1-3-3 شبکه‌های آزمایش و توسعه اتریوم.....
43	2-3-3 قراردادهای هوشمند Solidity.....
43	3-3-3 محیط توسعه Hardhat.....
44	4-3-3 سامانه پرونده IPFS.....
44	4-3-4 نمای کلی سامانه.....
45	1-4-3 بازیگران سامانه.....
48	2-4-3 معرفی قراردادهای هوشمند سامانه.....
50	5-3 پیاده‌سازی قراردادهای هوشمند.....
51	1-5-3 قرارداد هوشمند FractoRealNFT.sol یا FRN.....
56	2-5-3 قرارداد هوشمند FractoRealFractions.sol یا FRF.....
63	3-5-3 قرارداد هوشمند ChargeManagement.sol یا CM.....
68	6-3 گردش کار و تعاملات سامانه.....

68	1-6-3 توکنیزه کردن.....
72	2-6-3 عملیات DAO.....
75	3-6-3 مدیریت ساختمان.....
78	7-3 جمع بندی.....

فصل چهارم: تحلیل و ارزیابی روش پیشنهادی

80	1-4 مقدمه.....
80	2-4 آزمون.....
82	3-4 تجزیه و تحلیل آسیب پذیری.....
84	1-3-4 یافته های آگاه کننده.....
85	2-3-4 یافته های سطح پایین.....
87	4-4 استقرار.....
90	5-4 تجزیه و تحلیل هزینه های شبکه.....
93	6-4 تأخیر و توان عملیاتی.....
94	7-4 سایر ملاحظات امنیتی.....
94	1-7-4 یکپارچگی داده ها.....
95	2-7-4 دسترس پذیری.....
95	3-7-4 مسئولیت پذیری و عدم انکار.....
96	4-7-4 محرمانگی داده ها.....
96	8-4 مقایسه با کارهای مشابه.....
99	9-4 جمع بندی.....

فصل پنجم: بحث و نتیجه گیری

102	1-5 نتیجه گیری.....
105	2-5 محدودیت ها.....
106	3-5 پیشنهاداتی برای مطالعات آتی.....
106	4-5 جمع بندی.....
108	منابع و مأخذ.....
112	پیوست ها.....
131	چکیده انگلیسی.....

فهرست جداول

<u>شماره صفحه</u>	<u>عنوان</u>
26	جدول 1-2 مقایسه قراردادهای هوشمند با قراردادهای مرسوم.....
51	جدول 1-3 توابع قرارداد هوشمند FRN.....
52	جدول 2-3 توابع اصلی قرارداد هوشمند استاندارد ERC-721 ارث‌بری شده توسط FRN.....
52	جدول 3-3 توابع قرارداد هوشمند RentManagement.sol ارث‌بری شده توسط FRN.....
53	جدول 4-3 رویدادهای قرارداد هوشمند FractoRealNFT.sol.....
53	جدول 5-3 رویدادهای اصلی قرارداد هوشمند استاندارد ERC-721 ارث‌بری شده توسط FRN.....
53	جدول 6-3 رویدادهای RentManagement.sol ارث‌بری شده توسط FRN.....
56	جدول 7-3 توابع قرارداد هوشمند FRF.....
57	جدول 8-3 توابع اصلی قرارداد هوشمند استاندارد ERC-1155 ارث‌بری شده توسط FRF.....
57	جدول 9-3 توابع قرارداد هوشمند FractionsDAO.sol ارث‌بری شده توسط FRF.....
58	جدول 10-3 رویدادهای قرارداد هوشمند FractoRealFractions.sol.....
58	جدول 11-3 رویدادهای اصلی قرارداد هوشمند استاندارد ERC-1155 ارث‌بری شده توسط FRF.....
58	جدول 12-3 رویدادهای FractionsDAO.sol ارث‌بری شده توسط FRF.....
61	جدول 13-3 پارامترهای هر پیشنهاد در FD.....
64	جدول 14-3 توابع قرارداد هوشمند CM.....
64	جدول 15-3 توابع قرارداد هوشمند BuildingManagerElection.sol ارث‌بری شده توسط CM.....
65	جدول 16-3 رویدادهای قرارداد هوشمند ChargeManagement.sol.....
65	جدول 17-3 رویدادهای قرارداد هوشمند BuildingManagerElection.sol ارث‌بری شده توسط CM.....
67	جدول 18-3 وضعیت‌های مختلف در فرایند انتخابات مدیر ساختمان.....
83	جدول 1-4 مسائل پیدا شده توسط Slither در کد بر اساس نوع.....
89	جدول 2-4 آدرس قراردادهای هوشمند مستقر شده در شبکه‌ی Sepolia.....
90	جدول 3-4 برآورد هزینه‌ی استقرار هر قرارداد هوشمند در شبکه اتریوم.....
97	جدول 4-4 مقایسه راه‌حل پیشنهادی با مقالات مرتبط.....

فهرست شکل‌ها

<u>شماره صفحه</u>	<u>عنوان</u>
12	شکل 2-1 زنجیره‌ای از بلوک‌ها در بلاکچین.....
40	شکل 3-1 شمای کلی روش پیشنهادی جهت عرضه اولیه املاک.....
45	شکل 3-2 شمای کلی از معماری روش سامانه پیشنهادی.....
69	شکل 3-3 نمودار توالی آماده‌سازی قرارداد هوشمند FRN.....
70	شکل 3-4 نمودار توالی فروش موفق یک توکن در گام اول.....
71	شکل 3-5 نمودار توالی آماده‌سازی قراردادهای هوشمند جهت گام دوم فروش.....
72	شکل 3-6 نمودار توالی خرید موفق یک توکن جزئی FRF.....
73	شکل 3-7 نمونه فرایند ارسال و اجرای یک پیشنهاد در DAO.....
75	شکل 3-8 نمودار توالی تخصیص و برداشت اجاره یک واحد اشتراکی.....
76	شکل 3-9 نمودار توالی انتخابات مدیر ساختمان.....
77	شکل 3-10 نمودار توالی جمع‌آوری و صرف هزینه‌های ساختمان.....
81	شکل 4-1 خروجی دستور اجرای آزمون.....
82	شکل 4-2 خروجی دستور اندازه‌گیری سطح پوشش کد.....
84	شکل 4-3 گزارش خلاصه‌شده Slither.....
85	شکل 4-4 گزارش یافته‌های آگاه‌کننده Slither.....
86	شکل 4-5 گزارش آسیب‌پذیری‌های سطح پایین Slither.....
89	شکل 4-6 پیکربندی‌های موردنیاز Hardhat جهت فرایند استقرار.....
89	شکل 4-7 اجرای اسکریپت استقرار قراردادهای هوشمند در شبکه‌ی Sepolia.....
90	شکل 4-8 تراکنش‌های مرتبط با ایجاد قراردادهای هوشمند در شبکه‌ی Sepolia.....
91	شکل 4-9 گزارش گاز مصرفی هنگام استقرار قراردادهای هوشمند و فراهوانی توابع آن‌ها.....
94	شکل 4-10 مدت زمان استقرار و اجرای تراکنش‌ها.....

چکیده

سرمایه‌گذاری در املاک و مستغلات همواره به دلیل پتانسیل ایجاد جریان درآمدی ثابت، دارایی ارزشمندی تلقی شده است. ظهور فناوری بلاکچین و قراردادهای هوشمند منجر به توکنیزه کردن دارایی‌ها شده است که می‌تواند نقدشوندگی را افزایش داده، دسترسی سرمایه‌گذاران خرد را تسهیل و شفافیت را بهبود بخشد. با این حال، تحقیقات قبلی علی‌رغم محدود بودن تعدادشان، عمدتاً بر یک جنبه از کاربردهای بلاکچین در املاک تمرکز داشته‌اند و اغلب به چالش‌هایی مانند مدیریت اشتراکی دارایی‌ها و جمع‌آوری و مدیریت هزینه‌های اجاره و خدمات واحدهای چند مالکیتی نپرداخته‌اند. این پژوهش با ارائه رویکردی جامع برای توسعه و پیاده‌سازی یک سامانه مدیریت سرمایه‌گذاری در بازار املاک و مستغلات، این محدودیت‌ها را برطرف می‌کند. نوآوری اصلی این تحقیق، استفاده ترکیبی از توکن‌های مختلف برای نمایندگی و مدیریت دارایی‌ها است. این سامانه شامل امکاناتی نظیر توکنیزه کردن کامل و جزئی دارایی‌ها و ارائه مالکیت تکی و اشتراکی، مدیریت واحدهای اشتراکی از طریق سازمان غیرمتمرکز خودگردان، مدیریت جامع اجاره و خدمات، و انتخابات شفاف مدیر ساختمان در بستر بلاکچین است. یافته‌ها نشان می‌دهد که این سامانه می‌تواند به طور مؤثری نقدشوندگی و دسترسی به بازار املاک را از طریق توکنیزه کردن افزایش دهد و با کاهش نیاز به واسطه‌ها، هزینه‌های معاملات را کاهش دهد. آزمون‌های جامع و ارزیابی‌های مرتبط با هزینه، صحت عملکرد و مقرون‌به‌صرفه بودن سامانه‌ی پیشنهادی را تضمین می‌کنند. همچنین، استفاده از بلاکچین شفافیت و اعتماد در بازار را بهبود می‌بخشد و راه‌حل‌های کارآمدی برای مشکلات فعلی ارائه می‌دهد. این تحقیق نشان می‌دهد که توکنیزه کردن دارایی‌ها از طریق بلاکچین می‌تواند بازار سنتی املاک را متحول کند.

کلیدواژه‌ها: بلاکچین، توکنیزه کردن، قرارداد هوشمند، NFT، اتریوم، املاک و مستغلات

فصل اول

مقدمه

1-1 مقدمه

صنعت املاک و مستغلات از دیرباز با موانعی از جمله نقدینگی محدود برای ورود به بازار مواجه است. رویکردهای مرسوم برای سرمایه‌گذاری در املاک و مستغلات اغلب به منابع مالی قابل‌توجهی نیاز دارند و همچنین ممکن است از نظر جغرافیایی محدودکننده باشند. علاوه‌برآن، مدیریت و نگهداری املاک و مستغلات می‌تواند یک فرایند پیچیده و زمان‌بر برای ذی‌نفعان مختلف این صنعت، از جمله مالکان و مستأجران باشد. این فصل با ارائه یک نمای کلی از مسئله، تشریح ضرورت تحقیق، بیان سؤال و اهداف آن و همچنین ارائه‌ی ساختار پایان‌نامه، زمینه را برای مطالعه‌ی ارائه شده فراهم کرده و مفهوم توکنیزه کردن املاک و مستغلات را به عنوان یک راه‌حل بالقوه برای چالش‌های ذکر شده معرفی می‌کند.

2-1 بیان مسئله

املاک و مستغلات به طور مرسوم یک طبقه دارایی غیر قابل نقد و غیرشفاف بوده و چالش‌هایی را برای خریداران و فروشندگان ایجاد می‌کنند [1]، [2]. فرایند خرید و فروش املاک و مستغلات اغلب آهسته و دست‌وپاگیر است و به حجم قابل‌توجهی کاغذبازی نیاز دارد. علاوه‌برآن، یکی دیگر از مشکلات بازار مرسوم، عدم دسترسی سرمایه‌گذاران خرد به این بازار است. اصطلاح «دسترسی» که در این پایان‌نامه به کار رفته است، به توانایی سرمایه‌گذاران، صرف‌نظر از وضعیت مالی یا موقعیت مکانی آن‌ها، برای دسترسی راحت به سرمایه‌گذاری در املاک و مستغلات اشاره دارد. در ابتدا، نشان‌دهنده‌ی کاهش هزینه‌ی موردنیاز برای ورود به صنعت سرمایه‌گذاری املاک و مستغلات است. فرصت‌های مرسوم سرمایه‌گذاری در املاک و مستغلات به دلیل سرمایه اولیه موردنیاز بالا عمدتاً به افراد دارای ارزش خالص بالا و سرمایه‌گذاران نهادی محدود شده است و دسترسی به بازار را برای عموم مردم و سرمایه‌گذاران خرد محدود می‌کند [3]. از جنبه‌ی دیگر، این افزایش دسترسی، عموم مردم و سرمایه‌گذاران را قادر می‌سازد تا به راحتی با استفاده از دستگاه‌های هوشمند مجهز به اینترنت، سرمایه‌گذاری در توکن‌های املاک و مستغلات را به طور امن و با سرعت بالا انجام دهند و با این امر، نقدشوندگی را نیز در بازار ارتقا دهند.

ظهور فناوری بلاکچین این پتانسیل را دارد که صنعت املاک و مستغلات را با ایجاد توکن‌های دیجیتالی که نشان‌دهنده مالکیت جزئی در دارایی‌ها هستند، متحول کند [4]. این توکن‌ها را می‌توان در صرافی‌های مبتنی بر بلاکچین معامله و دسترسی سرمایه‌گذاران در هر اندازه‌ای را به بازار املاک و مستغلات آسان‌تر کرد [5]. این پایان‌نامه بر طراحی و پیاده‌سازی یک سامانه مبتنی بر بلاکچین برای توکنیزه کردن املاک و مستغلات با استفاده از قراردادهای هوشمند متمرکز است. به طور خاص، این پایان‌نامه چالش‌های فنی مربوط به توسعه و استقرار یک سامانه ایمن و کارآمد برای توکنیزه کردن املاک و مستغلات را بررسی خواهد کرد و با ذکر جزئیات پیاده‌سازی قراردادهای هوشمند، شرح خواهد داد که چگونه می‌توان از آن‌ها برای خودکارسازی فرایندهای معاملات و مدیریت املاک و مستغلات استفاده کرد. هدف این پایان‌نامه،

کمک به ارتقای شفافیت، امنیت و نقدشوندگی در صنعت املاک و مستغلات از طریق ارائه و پیاده‌سازی راه‌حلی کاربردی مبتنی بر فناوری بلاکچین است. این راه‌حل، سازندگان و سرمایه‌گذاران فعال در این صنعت را به استفاده از فناوری بلاکچین در املاک و مستغلات ترغیب و تشویق می‌کند و کارایی و اثربخشی این فناوری را در این زمینه نشان می‌دهد.

3-1 اهداف تحقیق

اهداف اصلی این پایان‌نامه به شرح زیر می‌باشد:

- طراحی و پیاده‌سازی یک سامانه مبتنی بر بلاکچین ایمن و کارآمد برای توکنیزه کردن و مدیریت املاک و مستغلات با استفاده از قراردادهای هوشمند
- امکان‌پذیر ساختن مالکیت تکی و اشتراکی دارایی‌ها به طور همزمان
- تسهیل جمع‌آوری و توزیع هزینه‌های اجاره و سرانه‌ی خدمات ساختمان
- استفاده از سازمان‌های غیرمتمرکز خودگردان (DAO) برای عملکردهای مدیریت واحدهای چند مالکی

4-1 اهمیت و ضرورت تحقیق

همان‌طور که در بخش قبلی ذکر شد، صنعت سنتی املاک و مستغلات به دلیل آستانه سرمایه‌گذاری بالا و مشکلات مالی، برای سرمایه‌گذاران خرد غیرقابل دسترس بوده است. توکنیزه کردن املاک و مستغلات با استفاده از فناوری بلاکچین و قراردادهای هوشمند این پتانسیل را دارد که این صنعت را با افزایش نقدشوندگی و دسترسی برای سرمایه‌گذاران خرد متحول کند.

بر اساس مطالعه‌ی A. Saari و همکاران [6]، تحقیقات حوزه‌ی بلاکچین و املاک و مستغلات به چهار دسته‌بندی زیر تقسیم می‌شوند:

1. مدیریت اراضی¹: سامانه ثبت و تصدیق مالکیت، ارزش و کاربری زمین و اموال.
2. معاملات املاک و مستغلات²: فرایند انتقال مالکیت و حقوق دارایی‌های املاک و مستغلات از یک طرف به طرف دیگر.
3. توکنیزه کردن املاک و مستغلات³: ایجاد و تجارت سهام دیجیتال املاک و مستغلات از طریق توکن‌های دیجیتال.

¹ Land Administration

² Real Estate Transactions

³ Real Estate Tokenization

4. مدیریت املاک و مستغلات¹: بهره‌برداری و نگهداری دارایی‌های املاک و مستغلات برای اطمینان از عملکرد، سودآوری و پایداری آن‌ها.

با این حال، علی‌رغم مزایای بالقوه، تحقیقات زیادی در مورد اجرای توکنیزه کردن املاک مبتنی بر بلاکچین با استفاده از قراردادهای هوشمند وجود ندارد و اغلب تحقیقات اخیر از چهار دسته‌بندی ذکر شده، صرفاً بر روی مدیریت زمین و توکنیزه کردن تمرکز کرده‌اند [6]. همچنین اکثر تحقیقات پیشین این حوزه به صورت علمی بوده و روش عملیاتی برای پیاده‌سازی یک سامانه مبتنی بر بلاکچین پیشنهاد نکرده‌اند که باعث شده است وعده‌ی انجام معاملات املاک و مستغلات بر بستر بلاکچین به یک روش غیرقابل اجرا تبدیل شود [6]. همچنین به گفته‌ی A. Saari و همکاران، کاربردهای کنونی بلاکچین در املاک و مستغلات عمدتاً در مراحل اولیه و آزمایشی هستند [6]. در این راستا، سامانه پیشنهادی این پایان‌نامه که بر لایه‌ی کاربردی فناوری بلاکچین متمرکز است و از قراردادهای هوشمند استفاده می‌کند، این پتانسیل را دارد که در پر کردن این خلأ تحقیقاتی موجود مشارکت کند.

1-5 نوآوری

جنبه‌های نوآوری در سامانه پیشنهادی این پایان‌نامه را می‌توان به دو قسمت زیر تقسیم کرد:

نوآوری در رویکرد توکنیزه کردن:

این پایان‌نامه رویکرد جدیدی را برای توکنیزه کردن املاک و مستغلات معرفی کرده است که از ترکیب استانداردهای توکن غیرقابل معاوضه NFT و نیمه قابل معاوضه به طور هم‌زمان استفاده می‌کند. در بخش مرور ادبیات فصل دوم بیان خواهد شد که مطالعات قبلی توکنیزه کردن خود را با استفاده از توکن‌های دیگری پیاده‌سازی کرده‌اند و هیچ‌کدام از رویکرد ترکیبی بهره نبرده‌اند. رویکرد ترکیبی پیشنهادی انعطاف‌پذیری بیشتری را در مدیریت، سرمایه‌گذاری و فروش دارایی‌های املاک و مستغلات ارائه می‌دهد. نوآوری این رویکرد ترکیبی به شرح زیر می‌باشد:

- **مالکیت جزئی و افزایش دسترسی:** این سامانه از استاندارد ERC-721 (NFT) برای فروش کلی واحدها و از ERC-1155 برای مالکیت جزئی استفاده کرده است؛ بنابراین این سامانه هم برای سرمایه‌گذاران فردی که به دنبال یک ملک کامل هستند و هم برای سرمایه‌گذاران خرد که علاقه‌مند به مالکیت اشتراکی هستند، پاسخگو می‌باشد. این امر دسترسی به بازار املاک و مستغلات را برای طیف وسیع‌تری از ذی‌نفعان این صنعت گسترش می‌دهد.
- **انعطاف‌پذیری در توکنیزه کردن دارایی:** با روش پیاده‌سازی شده در این سامانه، واحدهای خرد

¹ Real Estate Management

شده را می‌توان بر اساس نیاز پس از تجزیه، به یک واحد کامل تبدیل کرد (و بلعکس) که به انعطاف‌پذیری بیشتری در مدیریت و فروش دارایی‌های املاک و مستغلات پس از عرضه‌ی اولیه منجر شده و دسترسی را برای عموم و بر اساس نیاز آن‌ها افزایش می‌دهد.

نوآوری در کاربردهای قراردادهای هوشمند:

سامانه پیشنهادی با پیاده‌سازی قراردادهای هوشمند برای خودکارسازی جنبه‌های مختلف مدیریت املاک و مستغلات، کاربردهای بلاکچین در این صنعت را بیشتر گسترش می‌دهد و در نتیجه باعث افزایش شفافیت شده و اتکا به واسطه‌ها را کاهش می‌دهد. این جنبه‌ها عبارت‌اند از:

- **مدیریت غیرمتمرکز واحدهای اشتراکی (DAO):** این سامانه دارای یک چارچوب تصمیم‌گیری جمعی تحت عنوان سازمان غیرمتمرکز خودگردان یا DAO می‌باشد که صاحبان توکن‌های اشتراکی را قادر می‌سازد تا از طریق آن در فرایندهای رأی‌گیری و ارائه‌ی پیشنهادها برای مدیریت واحدهایشان شرکت کنند.
- **مدیریت هزینه‌های سرانه‌ی خدمات و اجاره در بستر بلاکچین:** پیاده‌سازی و اجرای قراردادهای هوشمند مربوطه، جمع‌آوری و توزیع اجاره را بین مالکان جزئی و جمع‌آوری و مدیریت هزینه‌های سرانه‌ی خدمات ساختمان را در بستر بلاکچین و به‌صورت شفاف ممکن می‌سازد.

در مجموع، این تحقیق با معرفی یک رویکرد جدید توکنیزه‌کردن ترکیبی با استفاده از استانداردهای مرتبط و پیاده‌سازی قراردادهای هوشمند دیگر برای عملکردهای مدیریتی، نقش بلاکچین در صنعت املاک و مستغلات را پررنگ‌تر کرده و پتانسیل آن را در جنبه‌های مختلف املاک و مستغلات نشان می‌دهد.

6-1 پرسش و فرضیه‌های تحقیق

پرسش اصلی این تحقیق عبارت است از: «چگونه می‌توانیم به طور مؤثر یک سامانه مبتنی بر بلاکچین را برای توکنیزه کردن و مدیریت املاک و مستغلات با تضمین امنیت و کارایی و درعین حال دستیابی به اهدافی مانند افزایش نقدشوندگی و دسترسی پیاده‌سازی کنیم؟»

فرضیه‌های این تحقیق نیز به شرح زیر است:

1. استفاده از قراردادهای هوشمند می‌تواند زمان و هزینه مربوط به معاملات املاک و مستغلات را به میزان قابل‌توجهی کاهش دهد.
2. توکنیزه کردن املاک و مستغلات مبتنی بر بلاکچین با استفاده از قراردادهای هوشمند، نقدشوندگی و دسترسی را برای سرمایه‌گذاران خرد افزایش می‌دهد.
3. سامانه مبتنی بر بلاکچین برای توکنیزه کردن املاک و مستغلات، شفافیت و امنیت بیشتری را در مقایسه با روش‌های سنتی ارائه می‌دهد.

4. استفاده از فناوری بلاکچین و قراردادهای هوشمند در سامانه پیشنهادی، یک محیط بدون نیاز به اعتماد (Trustless) ایجاد کرده و از دست‌کاری داده‌ها توسط عوامل مخرب جلوگیری می‌کند.
5. سامانه توسعه‌یافته راه‌حلی عملی و مؤثر برای سازندگان و سرمایه‌گذاران املاک و مستغلات جهت توکنیزه کردن و تجارت دارایی‌های املاک و مستغلات ارائه می‌کند.

7-1 دامنه مساله

در این پایان‌نامه، بر روی پیاده‌سازی یک سامانه جهت توکنیزه کردن و مدیریت املاک و مستغلات با استفاده از فناوری بلاکچین و قرارداد هوشمند تمرکز می‌شود. دامنه‌ی این پژوهش شامل موارد زیر است:

- طراحی و پیاده‌سازی یک سامانه مبتنی بر بلاکچین برای توکنیزه کردن و مدیریت دارایی‌های املاک در قالب قراردادهای هوشمند
 - آزمایش جامع بخش‌های مختلف سامانه پیاده‌سازی شده
 - استقرار سامانه در شبکه‌ی بلاکچین
 - ارزیابی عملکرد، کارایی و امنیت سامانه
- دامنه‌ی این پژوهش شامل موارد زیر نمی‌شود:

- ارائه یک تحلیل جامع از بازار املاک و مستغلات
- بررسی مسائل حقوقی و نظارتی مرتبط با توکنیزه کردن املاک و مستغلات
- توسعه یک رابط کاربری برای تعامل کاربران با سامانه
- توسعه یک مدل اقتصادی کامل برای سامانه

8-1 مروری بر روش تحقیق

این پژوهش از یک رویکرد چندمرحله‌ای برای طراحی و پیاده‌سازی سامانه توکنیزه کردن املاک و مستغلات بر بستر بلاکچین اتریوم استفاده می‌کند. مراحل اصلی این رویکرد به شرح زیر است:

1. **مرور ادبیات:** در این مرحله، یک بررسی جامع از مفاهیم مرتبط با فناوری بلاکچین، قراردادهای هوشمند، توکنیزه کردن و DAO انجام خواهد شد. این بررسی، چارچوب نظری لازم برای طراحی سامانه را فراهم می‌کند.
2. **توسعه سامانه:** در این مرحله، سامانه توکنیزه کردن املاک و مستغلات با استفاده از قراردادهای هوشمند و با زبان برنامه‌نویسی Solidity توسعه داده خواهد شد. سامانه‌ی موردنظر شامل شش قرارداد هوشمند می‌شود که هر یک بخشی از سامانه‌ی پیشنهادی را در راستای رسیدن به اهداف تحقیق شامل می‌گردد. این بخش‌ها شامل توکنیزه کردن به صورت تکی، توکنیزه کردن به صورت اشتراکی، مدیریت توکن‌های اشتراکی از طریق DAO، مدیریت و جمع‌آوری اجاره، مدیریت و جمع‌آوری

هزینه‌های سرانه‌ی خدمات ساختمان به همراه فرایند انتخابات مدیر ساختمان می‌شود. توکنیزه کردن این سامانه دارای مکانیزم عرضه‌ی اولیه سامانه در دو مرحله‌ی فروش کامل واحدها و سپس فروش جزئی (اشتراکی) واحدها می‌باشد.

3. **آزمون و ارزیابی:** پس از پیاده‌سازی سامانه، آزمایش‌های دقیقی برای بررسی صحت عملکرد، امنیت و کارایی آن انجام می‌گردد. این آزمایش‌ها شامل آزمون‌های جامع، تجزیه و تحلیل آسیب‌پذیری، هزینه و ارزیابی عملکرد در شبکه آزمایشی اتریوم خواهد شد.

4. **استقرار:** پس از تأیید صحت و کارایی، سامانه در شبکه آزمایشی اتریوم مستقر می‌گردد.

5. **ارزیابی جامع:** در نهایت، سامانه مستقر شده در معیارهای مختلف عملکردی مانند یکپارچگی داده‌ها، دسترس‌پذیری، مقرون به صرفه بودن، تأخیر و توان عملیاتی مورد ارزیابی قرار می‌گیرد و همچنین با سایر کارهای مرتبط گذشته مقایسه می‌گردد.

9-1 ساختار پایان‌نامه

این پایان‌نامه در پنج فصل ساختار یافته است که شامل بررسی جامعی از نقش بلاکچین در صنعت املاک و مستغلات می‌شود. در زیر نمای کلی این ساختار ارائه شده است:

1. **فصل اول، مقدمه:** این فصل با تشریح چالش‌های سرمایه‌گذاری سنتی در املاک و مستغلات، فناوری بلاکچین و توکنیزه کردن را به عنوان راه‌حلی نوآورانه برای غلبه بر این چالش‌ها معرفی می‌کند.

2. **فصل دوم، مرور ادبیات:** این فصل به بررسی جامع ادبیات موجود در حوزه‌های فناوری بلاکچین، اتریوم، قراردادهای هوشمند، توکنیزه کردن و سازمان‌های غیرمتمرکز خودگردان می‌پردازد. همچنین، مطالعات پیشین مرتبط با کاربرد بلاکچین در صنعت املاک و مستغلات را معرفی کرده و شکاف‌های و محدودیت‌های موجود در آن‌ها را شناسایی می‌کند.

3. **فصل سوم، روش پیشنهادی:** این فصل با ارائه شرح مفصلی از سامانه پیشنهادی توکنیزه کردن املاک و مستغلات، به طور عمیق به بررسی معماری روش پیشنهادی برای پیاده‌سازی سامانه، عملکردها و فناوری‌های بکار رفته در پیاده‌سازی آن می‌پردازد. علاوه بر آن، جزئیات تمامی قراردادهای هوشمند توسعه یافته، به همراه مستندات پیاده‌سازی توابع آن‌ها، تشریح شده و در نهایت، تعاملات بین بازیگران مختلف سامانه با این قراردادهای هوشمند به طور کامل توضیح داده می‌شود.

4. **فصل چهارم: تحلیل و ارزیابی روش پیشنهادی:** این فصل به تشریح فرایندهای دقیق آزمون و صحت‌سنجی قراردادهای هوشمند توسعه یافته در سامانه پیشنهادی توکنیزه کردن املاک و

مستغلات می‌پردازد. سپس، جزئیات مربوط به استقرار این قراردادها در شبکه بلاکچین را ارائه می‌کند و در نهایت، به بررسی و تحلیل کارایی و هزینه‌های عملیاتی مرتبط با آن‌ها و همچنین مقایسه با کارهای قبلی می‌پردازد.

5. فصل پنجم، نتیجه‌گیری و کارهای آینده: این فصل به‌عنوان جمع‌بندی پایانی پژوهش، خلاصه‌ای جامع از یافته‌های کلیدی را ارائه می‌کند و سهم مشارکت سامانه پیشنهادی در ارتقای توکنیزه کردن املاک و مستغلات را به طور برجسته به نمایش می‌گذارد. همچنین، ضمن اذعان به محدودیت‌های موجود در این پژوهش، حوزه‌های کلیدی برای مطالعات آینده در زمینه کاربرد بلاکچین در صنعت املاک و مستغلات را شناسایی و معرفی می‌کند.

فصل دوم

مرور ادبیات

1-2 مقدمه

این فصل به بررسی جامع ادبیات موجود در زمینه‌های مرتبط با موضوع تحقیق، شامل فناوری بلاکچین، اتریوم، قراردادهای هوشمند، توکنیزه کردن و سازمان‌های غیرمتمرکز خودگردان می‌پردازد. علاوه بر آن، مطالعات پیشین مرتبط با کاربردهای بلاکچین در صنعت املاک و مستغلات نیز مورد تحلیل و بررسی قرار می‌گیرد. هدف از این مرور، ایجاد بستری قوی برای درک عمیق مفاهیم کلیدی، فناوری‌ها و یافته‌های پیشین در این زمینه و همچنین شناسایی شکاف‌ها و محدودیت‌های موجود در مطالعات قبلی است. با اتکا به این دانش، در فصول بعدی می‌توان به ارائه راه‌حل‌های نوآورانه و کاربردی در حوزه املاک و مستغلات بر پایه ویژگی‌های ذاتی بلاکچین که در این فصل تشریح می‌شود، پرداخت.

در این راستا، این فصل به بررسی عمیق فناوری بلاکچین، معماری آن، مکانیزم‌های اجماع و مزایا و محدودیت‌های آن می‌پردازد. سپس، به تشریح پلتفرم بلاکچین محبوب اتریوم، اجزای اصلی آن (ماشین مجازی اتریوم، وب ۳، برنامه‌های کاربردی غیرمتمرکز، حساب‌ها، تراکنش و کارمزد) و قراردادهای هوشمند ادامه می‌دهد. مفاهیم توکن، توکنیزه کردن، استانداردها و مزایای آن نیز بررسی می‌شود. در ادامه، به سازمان‌های غیرمتمرکز خودگردان (DAO) و نقش آن‌ها در اکوسیستم بلاکچین پرداخته می‌شود. در نهایت، این فصل مطالعات پیشین در کاربرد بلاکچین در املاک و مستغلات را ارزیابی، شکاف‌های تحقیقاتی را مشخص و یافته‌های مهم آن‌ها را برجسته می‌کند. این مرور جامع، پایه و اساس درک عمیق از مفاهیم کلیدی بلاکچین و یافته‌های پیشین در حوزه کاربردهای آن در املاک و مستغلات را فراهم کرده و زمینه را برای ارائه راه‌حل‌های نوآورانه در فصول بعدی آماده می‌کند.

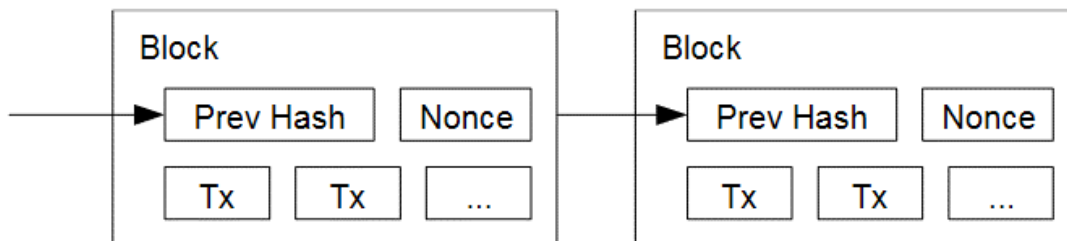
2-2 بلاکچین

فناوری بلاکچین برای اولین بار توسط Satoshi Nakamoto در سال ۲۰۰۸ به عنوان یک سامانه دفترکل توزیع شده و غیرمتمرکز معرفی شد که معاملات ایمن و شفاف را بدون نیاز به یک واسطه قابل اعتماد امکان‌پذیر کرد [7]. از آن زمان، استفاده از فناوری بلاکچین به بسیاری از صنایع مختلف از جمله املاک و مستغلات گسترش یافته است [8]. این بخش به مفاهیم اساسی و اهمیت فناوری بلاکچین می‌پردازد و پیامدهای مهم آن را برای بازار املاک و مستغلات بررسی می‌کند.

1-2-2 معرفی فناوری بلاکچین

در ساده‌ترین شکل، بلاکچین را می‌توان به عنوان یک پایگاه داده توزیع شده تعریف کرد. فناوری بلاکچین، یک سامانه دفترکل غیرمتمرکز و توزیع شده است که برای ثبت ایمن و شفاف تراکنش‌ها، بدون اتکا به واسطه و نهاد متمرکز، مهندسی شده است. مجموعه‌ای از این سوابق تراکنش‌ها تشکیل دهنده یک بلوک هستند که این بلوک‌ها از طریق هش به بلوک‌های قبلی به صورت زنجیره‌ای وصل شده‌اند (شکل 1-2).

این روند زنجیره‌ای از جدیدترین بلوک بلاکچین تا بلوک اول ادامه پیدا می‌کند تا یک زنجیره‌ای از بلوک‌ها را تشکیل دهد. تمامی تراکنش‌های بلاکچین و اقداماتی که بر روی آن‌ها صورت می‌گیرد، به دلیل ویژگی تغییرناپذیری و ساختار ذاتی بلاکچین، به صورت کاملاً امن نگهداری می‌شوند.



شکل 1-2 زنجیره‌ای از بلوک‌ها در بلاکچین [7]

استفاده از رمزگذاری‌های نامتقارن در بلاکچین، امکان احراز هویت و تأیید امن این تراکنش‌ها را فراهم می‌کند. فرایند اعتبارسنجی از رمزگذاری نامتقارن استفاده می‌کند که شامل دو کلید مجزا است: یک کلید عمومی و یک کلید خصوصی. این فرایند این امکان را به وجود می‌آورد که تراکنش‌ها در شبکه‌ی بلاکچین، بدون امکان دست‌کاری توسط عوامل مخرب، به گردش درآیند. بدین صورت که کلید خصوصی برای امضا و ایجاد یک تراکنش استفاده می‌شود و به عنوان یک امضای دیجیتال برای شناسایی فرستنده تراکنش عمل می‌کند. از سوی دیگر، کلید عمومی برای تأیید تراکنش و تعیین هویت فرستنده استفاده می‌شود. نکته قابل توجه این است که کلید عمومی نه تنها برای تأیید هویت فرستنده استفاده می‌شود، بلکه به عنوان آدرس گیرنده برای تراکنش‌های دریافتی نیز عمل می‌کند و به طور بدیهی، فاش شدن کلید خصوصی منجر به دسترسی غیرمجاز، جعل هویت و سو استفاده از کیف پول یک فرد می‌شود.

برای روشن شدن بیشتر این مفهوم، به مثال زیر توجه کنید:

1. فرستنده «الف» قصد دارد مقدار مشخصی پول برای گیرنده «ب» ارسال کند.
2. «الف» تراکنش را با کلید خصوصی خود امضا می‌کند و آن را به آدرس «ب» که همان کلید عمومی «ب» محسوب می‌شود ارسال می‌کند.
3. «ب» از کلید عمومی «الف» برای تأیید مشروعیت تراکنش استفاده می‌کند و اطمینان می‌دهد که مبدأ تراکنش واقعاً «الف» است.

مهم است که بدانیم این مکانیزم در محیط بلاکچین پیچیده تر است. در بخش‌های بعدی، مفاهیم اساسی بلوک‌ها و هش‌ها را که برای درک فرایندهای رمزگذاری ذکر شده در بالا بسیار مهم هستند، بررسی می‌گردد.

2-2-2 معماری بلاکچین

طراحی فناوری بلاکچین بر عملکرد چهارعنصر اصلی تکیه دارد که هر یک نقش مهمی در تضمین امنیت و یکپارچگی سامانه دارند. این عناصر عبارت‌اند از بلوک‌ها که اساساً محفظه‌هایی برای ذخیره داده‌ها هستند، هش‌ها که اثر انگشت دیجیتال منحصربه‌فرد برای هر بلوک هستند، نانس‌ها که اعداد تصادفی تولید شده در طی فرایند استخراج هستند، و معیار دشواری که مشخص‌کننده‌ی سختی فرایند استخراج یک بلوک جدید است. این مؤلفه‌ها با همکاری یکدیگر، یک معماری مقاوم و قابل‌اعتماد برای فناوری بلاکچین ارائه می‌کنند. در این بخش این اجزای کلیدی را یک‌به‌یک بررسی می‌کنیم.

1-2-2-2 بلوک

در فناوری بلاکچین، بلوک واحدی از داده است که حاوی اطلاعات مربوط به تراکنش‌ها مانند فرستنده، گیرنده، مبلغ و مهر زمانی می‌باشد. یک بلوک همچنین دارای یک مرجع به بلوک قبلی در بلاکچین است که پیوندی بین آن‌ها ایجاد می‌کند. اولین بلوک در بلاکچین، بلوک پیدایش¹ نام دارد و هیچ مرجع قبلی‌ای ندارد. هر بلوک به دو بخش سرآیند و بدنه تقسیم می‌گردد که سرآیند یک بلوک شامل مواردی مانند مهر زمانی، نانس، هش بلوک کنونی و هش بلوک قبلی است. بدنه‌ی یک بلوک، تشکیل شده از تراکنش‌هایی است که توسط کلید خصوصی کاربران امضا شده و از طریق کلید عمومی قابل تأیید می‌باشد. تعداد تراکنش‌های قابل‌ذخیره در هر بلوک از طریق پارامتر اندازه بلوک مشخص می‌شود که در هر بلاکچین متفاوت است. در نهایت، هنگامی که یک بلوک کامل می‌شود، برای تأیید و استخراج بین ماینرهای شبکه‌ی بلاکچین توزیع می‌گردد. ساختار یک بلوک تضمین می‌کند که داده‌های موجود در آن نمی‌توانند مورد تغییر یا دست‌کاری واقع شوند، و آن را به یک رکورد امن و تغییرناپذیر از همه تراکنش‌ها تبدیل می‌کند.

2-2-2-2 هش

توابع هش نقش مهمی در معماری بلاکچین دارند. هش به رشته‌ای با طول ثابت از حروف و اعداد اشاره دارد که توسط یک تابع هش تولید می‌گردد. توابع هش بدون توجه به طول رشته‌ی ورودی، همواره خروجی‌ای با طول ثابت مختص به الگوریتم مخصوص آن تابع تولید می‌کنند و به‌گونه‌ای عمل می‌کنند که همواره با ورودی یکسان، خروجی یکسانی حاصل می‌گردد. این ویژگی توابع هش باعث شده که مقدار هش خروجی این توابع را بتوان به‌عنوان اثر انگشت دیجیتال منحصربه‌فرد داده‌های ورودی تابع هش در نظر گرفت. این امر که می‌توان با تکرار توابع هش بر روی داده‌ها به خروجی منحصربه‌فرد یکسان مختص همان داده‌ها رسید نقش هش را در تأیید تمامیت داده‌ها مفید می‌سازد. شایان‌ذکر است که در دنیای دیجیتالی امروزه توابع هش مختلفی در دسترس هستند که هر کدام ویژگی‌های منحصربه‌فرد خود را دارند.

¹ Genesis Block

پرباربردترین توابع هش در پلتفرم‌های پیشرو بلاکچین مانند بیت‌کوین و اتریوم به ترتیب توابع هش SHA-256 و keccak-256 هستند [9].

3-2-2-2 نانس

همان‌طور که در شکل 1-2 مشخص است، هر بلوک در بلاکچین شامل یک نانس می‌باشد. نانس در مفهوم رمزنگاری، مخفف عبارت انگلیسی «number used once» و یک عدد تصادفی ۳۲ بیتی است که در فرایند استخراج خصوصاً در الگوریتم اجماع PoW¹ استفاده می‌شود. استخراج مکانیزمی است که توسط آن بلوک‌های جدیدی به بلاکچین اضافه می‌گردد. در بلاکچین‌های مبتنی بر PoW، ماینرها در تلاش هستند تا با پیدا کردن نانس صحیح و مطابق محدودیت‌های درجه‌ی سختی شبکه، بلوک را استخراج کنند. اطلاعات بیشتر مربوط به درجه سختی در بخش بعدی و PoW در بخش 1-2-3-2-2 به طور کامل توضیح داده می‌شود. قابل توجه است که نانس یک بلوک با نانس تراکنش که در بلاکچین اتریوم به هر حساب اختصاص داده می‌شود متفاوت است. در بلاکچین اتریوم هر تراکنش شامل یک نانس است که این نانس به تعداد تراکنش‌های قبلی ارسال شده توسط کاربر امضاکننده‌ی تراکنش اشاره می‌کند و کاربردی متفاوت (جلوگیری از حمله‌ی اجرای مجدد²) نسبت به نانس بلوک دارد [10].

4-2-2-2 معیار درجه سختی³

استخراج یک بلوک جدید در بلاکچین یک فرایند پیچیده است و معیار درجه سختی نقش مهمی در تعیین سطح پیچیدگی دارد. معیار درجه سختی به پیچیدگی استخراج موفقیت‌آمیز یک بلوک جدید اشاره دارد یا به عبارتی دیگر، میزان توان پردازشی موردنیاز برای استخراج بلوک‌ها به سطح درجه سختی بستگی دارد؛ اگرچه که هرچقدر درجه سختی بالاتر باشد منابع بیشتری برای استخراج موردنیاز است، اما شبکه را در برابر حملات احتمالی ایمن‌تر می‌کند. به صورت تخصصی‌تر، درجه سختی یا دشواری یک پارامتر قابل تنظیم در شبکه‌های مبتنی بر PoW است که به طور میانگین میزان محاسبه موردنیاز برای پیدا کردن یک نانس معتبر را کنترل می‌کند. به طور معمول، درجه سختی بیانگر تعداد صفرهای آغازین در عبارت هش حاصل شده از بلوک می‌باشد که برای معتبر تلقی شدن آن بلوک لازم است [10]. درجه سختی در بلاکچین بیت‌کوین هر ۲۰۱۶ بلوک (به طور تخمینی هر دو هفته) مجدداً طوری تنظیم می‌گردد که به طور میانگین زمان استخراج هر بلوک ۱۰ دقیقه باقی بماند.

با درک اجزای مختلف بلاکچین، می‌توانیم درک عمیق‌تری از مقاوم‌بودن و اقدامات امنیتی ادغام شده در

¹ Proof of Work

² Replay attack

³ Difficulty

بلاکچین به دست آوریم. در بخش‌های بعدی، نقش کلیدی این اجزا در شکل‌گیری ویژگی‌های بلاکچین مانند تغییرناپذیری و شفافیت را بررسی خواهیم کرد.

2-2-3 اجماع¹

در حوزه بلاکچین، دستیابی به اجماع بین گره‌های توزیع شده برای حفظ یکپارچگی و امنیت شبکه ضروری است. به بیان ساده، اجماع مکانیزمی است که توسط آن گره‌های مختلف شبکه توافق می‌کنند که آیا تراکنش‌ها و بلوک‌های توزیع شده در دفترکل معتبر هستند یا خیر. مکانیزم اجماع، اجازه می‌دهد تا اعتبار اطلاعات جدید وارد شده به شبکه‌ی بلاکچین صحت سنجی شود و شبکه را در برابر حملات و اقدامات مخرب ایمن می‌سازد. در دنیای بلاکچین، رویکرد اتخاذ شده برای دستیابی به اجماع بر عوامل عملکردی متعددی تکیه دارد که این عوامل نقش مهمی در تعیین اثربخشی شبکه دارند. این ویژگی‌های کلیدی شامل توانایی مقیاس‌پذیری کارآمد، سرعت پردازش تراکنش‌ها، زمان لازم برای تأیید یک بلوک به طور دائمی در شبکه، سطح امنیت ارائه شده و میزان مصرف منابع به‌ویژه انرژی در طول فرایند پردازش بلوک می‌شوند. برای دستیابی به اجماع در مورد یک دفترکل توزیع شده، مسئله فرماندهان بیزانسی (BGP²) باید حل شود. مکانیزم‌های اجماع مختلفی برای پرداختن به این مسئله تکامل یافته‌اند که در این بخش، پس از بررسی مسئله بیزانسی، چند روش برجسته اجماع که امروزه در شبکه‌های بلاکچین مورد استفاده قرار می‌گیرند را بررسی می‌کنیم.

2-2-3-1 خطای بیزانس

تصمیم‌گیری در سامانه‌های توزیع شده پیچیده است و برای حفظ قابلیت اطمینان و امنیت به مکانیزم‌های اجماع قابل اعتمادی نیاز دارند. مسئله فرماندهان بیزانسی و تحمل خطای بیزانس (BFT³) مفاهیم مهمی هستند که اطمینان و ثبات این مکانیزم‌ها را تضمین می‌کنند.

2-2-3-1-1 مسئله فرماندهان بیزانسی

مسئله فرماندهان بیزانسی (BGP) یک قیاس قدیمی است که به چالش هماهنگ کردن گروهی از فرماندهان که ممکن است برخی از آن‌ها خائن باشند می‌پردازد. هدف، دستیابی به اجماع در مورد یک برنامه اقدام مشترک (حمله یا عقب‌نشینی)، علی‌رغم اطلاعات نادرست احتمالی فرماندهان خائن است. در زمینه بلاکچین، این مشکل به طور استعاری به چالش دستیابی به اجماع بین گره‌ها در حضور بازیگران مخرب بالقوه تعریف می‌شود.

¹ Consensus

² Byzantine Generals Problem

³ Byzantine Fault Tolerance

2-1-3-2-2 تحمل خطای بیزانس

تحمل خطای بیزانس (BFT) مشخصه‌ای است که یک سامانه توزیع شده را قادر می‌سازد حتی در حضور عوامل معیوب یا مخرب به طور ایمن و قابل اطمینان عمل کند. در زمینه بلاکچین، دستیابی به تحمل خطای بیزانس به این معنی است که شبکه می‌تواند توافق و یکپارچگی عملیاتی را حفظ کند، حتی اگر برخی بازیگران (گره‌ها یا ماینرها) رفتار نامنظم و مخرب از خود نشان دهند یا تلاش کنند سامانه را مختل کنند. الگوریتم‌های اجماع مختلفی مانند الگوریتم تحمل خطای بیزانس ($^{1}PBFT$) برای رسیدگی به این چالش‌ها و اطمینان از انعطاف‌پذیری و امنیت شبکه‌های غیرمتمرکز در برابر خطاهای بیزانسی طراحی شده‌اند. در بخش بعدی به دو الگوریتم پرکاربرد در شبکه‌های بلاکچین خواهیم پرداخت.

2-3-2-2 الگوریتم‌های اجماع

اصلی‌ترین ویژگی که الگوریتم‌های اجماع باید داشته باشند این است که در مقابل خطای بیزانس مقاومت داشته باشند و حتی با وجود گره‌های معیوب و یا بازیگران مخرب، بتوانند به یک اجماع همگانی برسند. در این بخش، دو الگوریتم اجماع پرکاربرد شبکه‌های بلاکچین امروزی، یعنی PoW و ^{2}PoS که هر دو الگوریتم در مقابل خطای بیزانس مقاوم هستند را بررسی می‌کنیم.

1-2-3-2-2 الگوریتم اثبات کار (PoW)

الگوریتم اثبات کار یا (Proof of Work) اولین بار توسط ناکوموتو به عنوان راه‌حلی برای حل مسئله‌ی بیزانس معرفی شد و تا امروزه در شبکه‌ی بیت‌کوین مورد استفاده قرار می‌گیرد [7]. در PoW، گره‌های شبکه که به عنوان ماینر شناخته می‌شوند، برای حل یک مسئله‌ی پیچیده ریاضی با هم رقابت می‌کنند. اولین ماینری که معما را حل می‌کند راه‌حل را بین اعضای دیگر شبکه پخش می‌کند و در صورت تأیید اعتبار، مجاز است که یک بلوک جدید به بلاکچین اضافه کند. اولین ماینر که به جواب معما برسد، مقدار مشخص شده‌ای بیت‌کوین (سکه‌ی مخصوص شبکه) را به عنوان پاداش دریافت می‌کند.

نحوه‌ی عملکرد الگوریتم PoW را می‌توان در سه گام زیر خلاصه کرد:

1. ماینرها با مصرف زیاد انرژی و منابع سعی می‌کنند تا با به دست آوردن نانس مناسب، هش بلوک را با توجه به درجه سختی شبکه محاسبه کنند.
2. مقدار هش و نانس مناسب توسط یک ماینر به عنوان راه‌حل به گره‌های دیگر شبکه معرفی می‌شود.
3. پس از تأیید اعتبار، ماینر برنده مجاز است تا بلوک را به بلاکچین اضافه کند و جایزه مشخص شده را دریافت کند.

¹ Practical Byzantine Fault Tolerance

² Proof of Stake

توجه کنید که این فرایند به گونه‌ای پیاده‌سازی شده است که ارائه راه حل مناسب (پیدا کردن نانس صحیح) بسیار پیچیده باشد و درعین حال، گره‌های دیگر شبکه به سادگی و بدون نیاز به مصرف بالای منابع و انرژی بتوانند درستی این راه حل را صحت‌سنجی کنند. این الگوریتم به دلیل اینکه محاسبه‌ی نانس صحیح فرایندی پیچیده و با مصرف انرژی و منابع زیادی همراه است، سطح بالایی از امنیت شبکه را تضمین می‌کند چرا که هر عامل مخربی که بخواهد یک بلوک را مورد دست‌کاری قرار دهد، باید اثبات کار را نه تنها برای آن بلوک، بلکه برای تمامی بلوک‌هایی که به آن بلوک متصل شده است تکرار کند [7]. همان‌طور که توضیح داده شد، الگوریتم اثبات کار، برای صحت‌سنجی درستی کار گره‌های شبکه به مصرف بسیار زیادی از انرژی و منابع متکی است که این امر هرچند که باعث شده این الگوریتم به مقاومت و امنیت بالا مشهور شود؛ اما به همین دلیل ماهیت مصرف زیاد انرژی همواره مورد انتقاد فعالان محیط زیستی قرار گرفته است.

2-2-3-2-2 الگوریتم اثبات سهم (PoS)

الگوریتم اثبات سهم یا (Proof of Stake) به عنوان راه‌حلی برای مشکل مصرف انرژی الگوریتم اثبات کار معرفی شد. در این الگوریتم، ماینرها که در اینجا به عنوان تأییدکننده¹ شناخته می‌شوند، برخلاف الگوریتم PoW که با مصرف انرژی زیاد درستی کار خود را اثبات می‌کردند، مقداری سکه از شبکه را که ارزش مالی زیادی دارد به گرو می‌گذارند. در این مکانیزم در بازه‌های زمانی مشخص، یک تأییدکننده به عنوان پیشنهاددهنده‌ی بلوک انتخاب می‌شود که این گره وظیفه‌ی ایجاد و ارسال بلوک جدید به گره‌های دیگر شبکه را بر عهده دارد. در این بازه‌ی زمانی، در کنار پیشنهاددهنده‌ی بلوک، تعدادی از تأییدکنندگان به صورت تصادفی انتخاب شده و یک کارگروه را تشکیل می‌دهند. این کارگروه وظیفه‌ی صحت‌سنجی بلوک پیشنهاد شده توسط گره‌ی پیشنهاددهنده‌ی بلوک را بر عهده دارد [11]. در این روش، بر خلاف روش قبلی، تأییدکنندگان بخشی از کارمزد تراکنش را به عنوان پاداش دریافت می‌کنند و در صورت رفتار مخرب، مقداری از مبلغی که در گرو شبکه گذاشته‌اند را در قالب جریمه از دست می‌دهند.

در مقایسه با الگوریتم اجماع اثبات کار، این الگوریتم مصرف انرژی بسیار کمتری دارد و به همین دلیل منابع سخت‌افزاری زیادی مورد نیاز گره‌های شبکه نیست و از نظر مقیاس‌پذیری عملکرد بهتری را ارائه می‌دهد و مراتب بالاتری از تمرکززدایی² را تضمین می‌کند. هرچند که هزینه‌ی مورد نیاز برای سخت‌افزار و مصرف انرژی در این روش پایین‌تر است، حداقل سکه‌ی مورد نیاز برای گرو گذاشتن در شبکه‌های مختلف ممکن است قابل توجه باشد و از این نظر دسترسی کمتری را برای تأییدکنندگان ارائه کند. به طور مثال، شبکه‌ی بلاکچین اتریوم که از سال ۲۰۲۲ از روش اثبات کار به این روش مهاجرت کرده است، از

¹ Validator

² Decentralization

هر تأییدکننده می‌خواهد که حداقل ۳۲ اتریوم که در تاریخ‌نگارش این پایان‌نامه ارزشی معادل ۶۵ هزار دلار دارد را به‌عنوان گرو در شبکه ثبت کنند [11].

2-2-4 مزایا و محدودیت‌های استفاده از بلاکچین

مانند هر فناوری نوظهور، بلاکچین نیز دارای مزایا و معایب خاص خود است. درک این عوامل برای تصمیم‌گیری آگاهانه در مورد پیاده‌سازی بلاکچین در صنایع مختلف بسیار مهم است. در زیر به طور مختصر به برخی از این مزایا و معایب خواهیم پرداخت.

مزایا:

به‌عنوان یک فناوری نسبتاً جدید، بلاکچین به دلیل مزایایی که ارائه می‌دهد، به سرعت موردتوجه صنایع مختلف قرار گرفته است. این مزایا بر اساس ویژگی‌های منحصر به فرد بلاکچین که از ماهیت آن نشأت می‌گیرند، به صورت زیر تعریف می‌گردند:

- **یکپارچگی بالا:** یکی از نقاط قوت بلاکچین توانایی آن در تضمین یکپارچگی بالای داده‌هاست. هنگامی که یک تراکنش تأیید می‌گردد، به صورت رمزنگاری به تراکنش قبلی پیوند می‌خورد و یک زنجیره‌ی غیرقابل تغییر از داده‌ها را ایجاد می‌کند. این یکپارچگی ذاتی، دست‌کاری سوابق گذشته را برای اشخاص غیرمجاز بسیار چالش‌برانگیز کرده و نگرانی‌های مرتبط با هرگونه مداخله‌ای که ممکن است عوامل ثالث انجام دهند را از بین می‌برد.
- **بدون نیاز به اعتماد^۱:** این بدان معناست که در محیط بلاکچین، برای انجام معاملات نیازی به اعتماد به شخص ثالث نیست چرا که به لطف استفاده از قراردادهای هوشمند که در بخش 2-4 به آن خواهیم پرداخت، تمامی توافقات در بلاکچین به صورت خودکار اجرا شده و نیاز به واسطه و شخص ثالث از بین می‌رود.
- **تغییرناپذیر^۲:** ماهیت تغییرناپذیر بلاکچین به این واقعیت اشاره دارد که هنگامی که داده‌ها به شبکه اضافه می‌شوند، نمی‌توان آن‌ها را تغییر داد و یا حذف کرد. این امر از طریق استفاده از توابع هش رمزنگاری شده به دست می‌آید که تضمین می‌کند هرگونه تغییر در داده‌ها منجر به هش کاملاً متفاوتی می‌شود و دست‌کاری داده‌ها را از نظر محاسباتی غیرممکن می‌کند. مکانیزم‌های اجماع و فرایند اعتبارسنجی شبکه باعث می‌شود که هرگونه تحریف در بلوک‌های شبکه به راحتی شناسایی و از وقوع آن‌ها جلوگیری شود. این ویژگی نه تنها امنیت و قابلیت اطمینان داده‌ها را افزایش می‌دهد، بلکه ماهیت بدون نیاز به اعتماد فناوری بلاکچین را نیز تقویت می‌کند.

¹ Trustless

² Immutable

- **شفافیت¹:** فناوری بلاکچین یک دفترکل دیجیتال غیرمتمرکز را امکان‌پذیر می‌کند که به‌صورت عمومی به اشتراک گذاشته شده و در دسترس همه عوامل و بازیگران شبکه قرار می‌گیرد. این امر با ایجاد یک رکورد ضد دست‌کاری² و تغییرناپذیر از تمام تراکنش‌هایی که تاکنون در شبکه انجام شده است، شفافیت را ارتقا می‌دهد. در نتیجه، همه شرکت‌کنندگان در شبکه به اطلاعات یکسانی دسترسی دارند و یک محیط شفاف ایجاد می‌شود که در آن تاریخچه کل تراکنش‌های پردازش شده در شبکه قابل مشاهده، حسابرسی³، ردیابی⁴ و تأیید است.
- **تمرکززدایی⁵:** عدم تمرکز یا تمرکززدایی یک اصل کلیدی فناوری بلاکچین است. در معماری بلاکچین، هیچ سازمان یا گروهی متمرکزی وجود ندارد و این امر باعث شده حتی اگر گره‌ای از کار بیفتد یا با خطا مواجه شود (مسئله نقطه‌ی تکی شکست⁶)، کل سامانه همچنان بدون هیچ مشکلی به کار خود ادامه دهد. این معماری غیرمتمرکز همچنین به انعطاف‌پذیری، پایداری، مقاوم‌بودن⁷ و قابلیت اطمینان سامانه کمک می‌کند.
- **امنیت:** در کنار ماهیت تغییرناپذیر شبکه، استفاده از روش‌های رمزگذاری پیشرفته، از جمله رمزگذاری نامتقارن با استفاده از جفت کلید عمومی و خصوصی، امنیت تراکنش‌ها را در بلاکچین تضمین می‌کند. ماهیت غیرمتمرکز و توزیع‌شده شبکه با کاهش آسیب‌پذیری نقطه‌ی تکی شکست و حملات مخرب، نیز به‌عنوان یک‌لایه امنیتی اضافی عمل می‌کند.

محدودیت‌های بلاکچین:

درحالی‌که فناوری بلاکچین مزایای مختلفی را ارائه می‌دهد، مانند هر فناوری جدیدی بدون محدودیت نیست. یکی از چالش‌های مهم مسئله مقیاس‌پذیری است. ماهیت غیرمتمرکز بلاکچین، در عین این‌که امنیت و مقیاس‌پذیری را تضمین می‌کند، همراه با رشد شبکه می‌تواند منجر به به‌خطر افتادن امنیت، کندتر شدن زمان پردازش تراکنش‌ها و افزایش نیاز به منابع و در نتیجه مانعی برای مقیاس‌پذیری شود. این مفهوم در قالب سه عامل ناسازگار مقیاس‌پذیری⁸ برای اولین بار توسط Vitalik Buterin، هم بنیان‌گذار شبکه‌ی اتریوم، به این صورت مطرح شد که سه ویژگی کلیدی وجود دارد که هر سامانه توزیع شده باید برای دستیابی به آن‌ها تلاش کند و آن سه ویژگی، مقیاس‌پذیری، امنیت و عدم تمرکز هستند. این سه ویژگی

¹ Transparency

² Temper-proof

³ Audit

⁴ Traceable

⁵ Decentralization

⁶ Single point of failure

⁷ Robustness

⁸ Scalability Trilemma

به گونه ای هستند که اگر یکی از این سه ویژگی افزایش پیدا کند، دو ویژگی دیگر شروع به کاهش می کنند [12]. اگرچه در گذشته رسیدن به هر سه این ویژگی ها دور از ذهن بود، روش های جدیدی که قادر به بهبود عملکرد در هر سه این زمینه ها هستند توسعه یافته اند. از جمله این ها می توان به به روز رسانی های شبکه ای اتریوم اشاره کرد که به طور هم زمان شبکه ای اتریوم را مقیاس پذیر، امن و غیر متمرکز می کند [13].

محدودیت دیگر مربوط به مصرف انرژی مرتبط با مکانیزم های اجماع خاص به ویژه اثبات کار است. برخی از فرایندهای اجماع که برای ایمن سازی شبکه ضروری هستند، نیاز به قدرت محاسباتی قابل توجهی دارند که منجر به مصرف بالای انرژی و نگرانی های زیست محیطی می شوند. همان طور که قبل تر گفتیم، شبکه ای اتریوم با مهاجرت از مکانیزم اثبات کار به مکانیزم اثبات سهم این نگرانی را مرتفع کرده است.

نگرانی های امنیتی در شبکه های بلاکچین کوچک تر یا جدید تر نیز رایج است. یک حمله ۵۱ درصد¹ که در آن یک نهاد کنترل اکثریت قدرت استخراج شبکه را به دست می آورد، می تواند منجر به مشکل دو بار خرج کردن² شود و یکپارچگی و سازگاری داده ها و همچنین امنیت بلاکچین را به خطر بیندازد [14].

علاوه بر آن، قانون گذاری در مورد بلاکچین و ارزش های دیجیتال همواره در حال تغییر است و عدم قطعیت ها در این زمینه همچنان ادامه دارد و پیاده سازی و ادغام گسترده ی فناوری بلاکچین در سامانه های موجود را ممکن است با موانع قانونی و نظارتی روبرو گرداند.

2-3 اتریوم³

اتریوم که برای اولین بار توسط Vitalik Buterin و همکاران در اواخر سال ۲۰۱۳ معرفی و در سال ۲۰۱۵ راه اندازی شد، پیشرفت قابل توجهی در فناوری بلاکچین را ارائه می دهد [15]. بر خلاف بیت کوین، دستاوردهای اتریوم تنها به ارز دیجیتال محدود نمی شود، بلکه ایده ی انقلابی قراردادهای هوشمند را نیز در بر می گیرد. تمایز اصلی بین بیت کوین و اتریوم این است که تمرکز اتریوم، فراتر از عملکرد به عنوان یک ارز دیجیتال است، در حالی که بیت کوین در درجه اول به عنوان یک وسیله مبادله و ذخیره ارزش عمل می کند، اتریوم توافقات بدون نیاز به اعتماد و خودکار را از طریق اجرای قراردادهای هوشمند امکان پذیر می کند. این پیشرفت نوآورانه قابلیت های بلاکچین را از تراکنش های ساده به عملکردهای پیچیده تر و قابل برنامه ریزی گسترش می دهد. تفاوت مهم دیگر بین اتریوم و بیت کوین، مهاجرت اتریوم به سمت مکانیزم اجماع پایدارتر، مقیاس پذیرتر و کم مصرف تر اثبات سهم (PoS) است. از سال ۲۰۲۲، اتریوم از مدل اثبات کار (PoW) پر مصرف که توسط بیت کوین استفاده می شد فاصله گرفت و به جای آن الگوریتم

¹ 51% Attack

² Double-spending

³ Ethereum

PoS که با محیط زیست سازگارتر است را جایگزین کرد [11].

موفه‌های کلیدی کاربردی اتریوم، شامل ماشین مجازی اتریوم (EVM)، وب ۳، برنامه‌های کاربردی غیرمتمرکز (DApps)، انواع حساب‌ها، پردازش تراکنش‌ها و مفهوم گاز، در این بخش بررسی می‌شوند. این عناصر بنیادی، اتریوم را به پلتفرمی قدرتمند و انعطاف‌پذیر تبدیل می‌کنند و درک جامعی از عملکرد داخلی آن ارائه می‌دهند. شایان‌ذکر است که اطلاعات مرتبط با قراردادهای هوشمند و همچنین انواع توکن‌های شبکه‌ی اتریوم در بخش‌های 2-4 و 2-5 توضیح داده خواهد شد.

2-3-1 ماشین مجازی اتریوم (EVM)

از الزامات اصلی اجرای قرارداد هوشمند این است که این برنامه‌ها باید در محیطی امن اجرا شوند. ماشین مجازی اتریوم یا EVM¹ یک جز اساسی از بلاکچین اتریوم است که بر روی هر گره در شبکه نصب شده و به‌عنوان یک محیط اجرایی امن برای قراردادهای هوشمند عمل می‌کند [9]. ماشین‌های مجازی اتریوم عملکردی کاملاً ایزوله دارند به‌گونه‌ای که کد اجرا شده بر روی آن‌ها به شبکه، سامانه فایل و یا سایر فرایندها دسترسی ندارد که این امر باعث تضمین امنیت در هنگام اجرای قراردادهای هوشمند می‌شود [16]. یکی از جنبه‌های قابل توجه EVM، توانایی آن در اجرای کد قراردادهای هوشمند به شیوه‌ای قطعی² است. این امر قابلیت اطمینان را در اجرای قرارداد هوشمند تضمین می‌کند، چراکه EVM در هنگام مواجهه با ورودی یکسان خروجی یکسانی را ارائه می‌دهد. همچنین به‌منظور اطمینان از انسجام داده‌ها، EVM‌ها بر اساس اصل اجماع عمل می‌کنند. این مکانیزم تضمین می‌کند که تمام گره‌ها در شبکه یک نسخه دقیق و هماهنگ از یک داده را حفظ می‌کنند. قابل توجه است که کد ذخیره شده در ماشین‌های مجازی اتریوم به‌صورت بایت‌کد³ EVM است که یک‌زبان سطح پایین و قابل فهم برای ماشین مجازی اتریوم به‌حساب می‌آید. از آنجایی که کد نویسی به‌صورت بایت‌کد برای برنامه‌نویسان سخت است، زبان‌های سطح بالای بسیاری توسعه و طراحی شده‌اند که مهم‌ترین آن‌ها زبان برنامه‌نویسی Solidity می‌باشد. این زبان ابتدا توسط کامپایلر Solidity به بایت‌کد تبدیل شده و سپس برای ذخیره‌شدن به EVM‌ها فرستاده می‌شود.

2-3-2 وب ۳ (Web3) و برنامه‌های کاربردی غیرمتمرکز (DApps)

وب ۳ و برنامه‌های کاربردی غیرمتمرکز، اجزای جدایی‌ناپذیر در اکوسیستم اتریوم هستند که چشم‌انداز فناوری‌های غیرمتمرکز را شکل می‌دهند. جنبه‌های اساسی وب ۳ را می‌توان به‌صورت هویت دیجیتال، مالکیت دارایی در قالب توکن‌های دیجیتال و برنامه‌های کاربردی غیرمتمرکز خلاصه کرد. وب ۳ به عصر

¹ Ethereum Virtual Machine

² Deterministic

³ EVM Bytecode

جدیدی از اینترنت اشاره دارد که جهان حال حاضر در حال وارد شدن به آن است. نسخه‌ی اولیه اینترنت که از آن با عنوان وب ۱ یاد می‌شود، از وبگاه‌های ثابتی تشکیل شده بود که کمترین تعامل را با کاربران داشت. در مقابل، وب ۲ محتوای پویا، شبکه‌های اجتماعی و محتوای تولید شده توسط کاربر را در خود جای داده است. اکنون، تمرکز وب ۳ ایجاد یک وب غیرمتمرکز است که تا با استفاده از فناوری بلاکچین به کاربران خود اجازه دهد که کنترل بر هویت و داده‌های دیجیتال خود را حفظ کنند. این امر، افراد را قادر می‌سازد تا بدون واسطه با یکدیگر معامله کرده و ارتباط برقرار کنند. هدف اصلی وب ۳ افزایش باز بودن^۱، شفافیت و امنیت اینترنت است.

برنامه‌های کاربردی غیرمتمرکز، معروف به DApps^۲، نرم‌افزارهایی هستند از رابط کاربری فرانت‌اند^۳ مرسوم وب ۲ و بک‌اند^۴ مبتنی بر قراردادهای هوشمند و بلاکچین تشکیل شده‌اند و اکوسیستم‌های غیرمتمرکزی را ایجاد می‌کنند که برای عملکرد در زمان اجرا به نیروی انسانی نیاز ندارند. بر خلاف سرورهای متمرکزی که توسط برنامه‌های کاربردی سنتی استفاده می‌شود، بک‌اند یک Dapp متکی به یک یا چند قرارداد هوشمند است که در یک محیط غیرمتمرکز و هم‌تا به هم‌تا^۵ اجرا می‌گردد. سرویس‌های ارائه شده در قالب DApps خدمات مختلفی مانند امور مالی غیرمتمرکز (DeFi^۶)، بازارهای معاملات ارزهای دیجیتال و پلتفرم‌های بازی‌های ویدیویی را شامل می‌شوند.

3-3-2 حساب‌های شبکه‌ی اتریوم

در اکوسیستم اتریوم، دو نوع حساب وجود دارد که هر کدام اهداف جداگانه‌ای دارند [11]:

- **حساب‌های تحت مالکیت خارجی (EOA^۷):** این حساب‌ها، حساب‌های مرسوم کاربران شبکه بوده و از طریق کیف پول و کلید خصوصی کنترل می‌شوند و شامل هیچ کدی نیستند.
- **حساب‌های قراردادی^۸:** حساب‌هایی هستند که از طریق کد کنترل شده و کلید خصوصی ندارند. این حساب‌ها در واقع همان قراردادهای هوشمند به حساب می‌آیند.

هر حساب شبکه‌ی اتریوم، توسط یک آدرس عمومی منحصر به فرد ۱۶۰ بیتی شناسایی می‌شود. آدرس عمومی حساب‌های EOA در زمان افتتاح حساب از روی کلید عمومی کاربر مشخص می‌شود؛ اما آدرس

^۱ Openness

^۲ Decentralized Applications

^۳ Front-end

^۴ Back-end

^۵ Peer-to-peer

^۶ Decentralized Finance

^۷ Externally Owned Accounts

^۸ Contract Account

حساب‌های قراردادی در زمان استقرار آن‌ها روی شبکه و از روی آدرس و نانس حساب سازنده‌ی قرارداد تعیین می‌گردد [16].

در بلاکچین اتریوم، حساب‌های تحت مالکیت خارجی (EOA) و حساب‌های قراردادی (CA) ویژگی‌های متمایزی دارند که نقش‌های مربوطه را شکل می‌دهند. به‌عنوان نهادهای تحت مالکیت خارجی، EOAها هیچ هزینه‌ی ایجاد را متحمل نمی‌شوند و معمولاً با کاربران فردی مرتبط هستند. آن‌ها مجهز به یک جفت کلید رمزگذاری هستند که شامل یک کلید عمومی برای دریافت تراکنش‌ها و یک کلید خصوصی برای امضا و ارسال تراکنش‌ها و شروع فعالیت‌ها می‌شود. EOAها می‌توانند به‌طور مستقل تراکنش‌ها را آغاز کنند، البته معمولاً در صورت دخالت سایر EOAها به‌عنوان مقصد تراکنش، به انتقال رمزارز اتر (ETH) محدود می‌شوند. در مقابل، حساب‌های قراردادی متحمل هزینه‌های استفاده از فضای ذخیره‌سازی شبکه می‌شوند و کلید خصوصی ندارند، در عوض توسط منطق قرارداد هوشمند اداره می‌شوند. برخلاف EOAها، حساب‌های قراردادی فقط می‌توانند تراکنش‌ها را در پاسخ به دریافت تراکنشی دیگر انجام دهند. تراکنش‌هایی که از EOAها به حساب‌های قراردادی ارسال می‌شوند، این امکان را دارند که عملیات پیچیده تعبیه شده در کد قراردادهای هوشمند را راه‌اندازی و اجرا کنند [11]. شناخت این تفاوت‌های اساسی، زمینه را برای درک جامع از پویایی پیچیده بین کاربران و قراردادهای هوشمند در بلاکچین اتریوم فراهم می‌کند.

2-3-4 تراکنش

اتریوم بر اساس اصل یک ماشین حالت¹ مبتنی بر تراکنش عمل می‌کند که در آن تراکنش‌های بین حساب‌ها باعث تغییراتی در وضعیت سراسری شبکه‌ی اتریوم می‌شوند. این تراکنش‌ها شامل مجموعه‌ای از دستورالعمل‌ها هستند که به‌صورت رمزگذاری شده توسط یک حساب خارجی امضا شده و سپس به بلاکچین ارسال می‌شوند.

در شبکه‌ی اتریوم سه نوع تراکنش وجود دارد که عبارت‌اند از [11]:

- تراکنش‌های عادی² که از یک EOA به یک EOA دیگر ارسال می‌شود.
- تراکنش‌های استقرار قرارداد³ که مقصد ندارند و به همراه کد قرارداد توسط EOA ارسال می‌شود.
- تراکنش‌های اجرای قرارداد⁴ که به فراخوانی توابع موجود در قراردادهای هوشمند توسط EOA اشاره دارد.

¹ State Machine

² Regular Transactions

³ Contract Deployment Transactions

⁴ Execution of a Contract

تمامی تراکنش‌ها، فارغ از نوع آن‌ها، ابتدا توسط حساب‌های EOA که به‌عنوان پلی بین دنیای خارجی و وضعیت داخلی شبکه‌ی اتریوم عمل می‌کنند، آغاز و به بلاکچین منتقل می‌شوند. با این حال، قراردادهای هوشمند همچنان می‌توانند با یکدیگر ارتباط برقرار کنند. تعاملات بین قراردادهای از طریق «تراکنش‌های داخلی»¹ صورت می‌گیرد که می‌توان آن‌ها را معادل تراکنش‌های عادی در نظر گرفت، با این تفاوت که تراکنش‌های داخلی توسط حساب‌های EOA ایجاد نمی‌شوند، بلکه از طریق قراردادهای دیگر ایجاد می‌گردند. تراکنش‌های داخلی که به‌عنوان «پیام» نیز شناخته می‌شوند بر خلاف تراکنش‌های دیگر، صرفاً در محیط اجرایی اتریوم یعنی EVM وجود دارند و سریال‌سازی نمی‌گردند [10].

5-3-2 گاز و کارمزد

مکانیزم گاز² و کارمزد در اتریوم یک ویژگی حیاتی است که اجرای کارآمد و ایمن تراکنش‌ها در سراسر شبکه را تضمین می‌کند [13]. در اتریوم، همه‌ی عملیات‌ها، از تراکنش‌های ساده گرفته تا فراخوانی قراردادهای هوشمند پیچیده، برای اجرا به منابع محاسباتی نیاز دارند. گاز واحدی است که هزینه‌ی میزان مصرف این منابع محاسباتی را اندازه‌گیری کرده و به‌عنوان مکانیزمی مهم برای تخصیص منصفانه منابع و جلوگیری از سو استفاده شبکه عمل می‌کند. واحد پول بومی شبکه‌ی اتریوم، اتر (ETH)، تنها رمزارز قابل قبول برای پرداخت این هزینه‌ها در این شبکه می‌باشد. قیمت‌گذاری گاز معمولاً بر حسب gwei بیان می‌شود که واحدی کوچک شده از اتر است. کوچک‌ترین واحد اتر به‌صورت wei شناخته می‌شود که هر اتر معادل ۱۰ به توان ۸ wei تلقی می‌شود. مقدار gwei یا gigawei معادل یک میلیارد wei می‌باشد.

$$1 \text{ ETH} = 10^8 \text{ wei} = 1,000,000,000 \text{ gwei} \quad (1-2)$$

هنگام شروع یک تراکنش ساده یا تعامل با یک قرارداد هوشمند، کاربران باید حد گاز³ و قیمت گاز را مشخص کنند. حد گاز حداکثر مقدار کار محاسباتی که شبکه برای تراکنش انجام می‌دهد را مشخص می‌کند، درحالی‌که قیمت گاز مقدار اتر (ETH) پرداخت شده به‌ازای هر واحد گاز را تعیین می‌کند. برای محاسبه‌ی کل کارمزد یک معامله، حد گاز در قیمت گاز ضرب می‌شود. اگر تراکنش فاقد گاز موردنیاز برای تکمیل باشد، باطل تلقی شده و فسخ می‌گردد. در چنین مواردی، وضعیت سراسری اتریوم به حالت قبل از تراکنش باز می‌گردد و منطقاً هیچ گازی به کاربر بازپرداخت نمی‌گردد.

از دید کاربران مصرف‌کننده‌ی شبکه‌ی اتریوم، درک گاز و هزینه‌ها ضروری است. این درک کاربران را قادر می‌سازد تا تراکنش‌های خود را با تنظیم مناسب محدودیت‌ها و قیمت‌های گاز بر اساس اولویت‌های

¹ Internal Transactions

² Gas

³ Gas Limit

خود، مانند زمان تأیید سریع‌تر یا هزینه‌های تراکنش کمتر، بهینه کنند. برای توسعه‌دهندگان نیز بسیار مهم است که بهینه‌سازی قراردادهای هوشمند را در طول توسعه و اجرای برنامه‌های غیرمتمرکز اولویت‌بندی کنند. این به این دلیل است که گاز مورد استفاده برای فراخوانی توابع از قرارداد هوشمند کاملاً به بهینه بودن کد بستگی دارد. اگر کد به طور ناکارآمد نوشته شود، هزینه‌های اجرا برای کاربران می‌تواند بسیار بالا باشد؛ بنابراین، نوشتن کد به شیوه‌ای بهینه باهدف کاهش این هزینه‌ها بسیار مهم است.

2-3-6 رویداد¹

رویدادها یکی از امکانات قراردادهای هوشمند مبتنی بر شبکه‌ی اتریوم هستند و امکان استفاده از لاگ‌های EVM را به نهادهای خارج از شبکه‌ی بلاکچین می‌دهند. Dapps می‌توانند در صورت نیاز به رویدادهای منتشر شده توسط یک قرارداد هوشمند گوش دهند و بر اساس آن واکنش نشان دهند.

2-4 قرارداد هوشمند

در حوزه‌ی توکنیزه کردن املاک و مستغلات به‌صورت غیرمتمرکز، قراردادهای هوشمند نقش حیاتی را برعهده دارند. این قراردادها با عمل به‌عنوان یک واسطه، فرایند معاملات بین دو طرف را خودکار می‌کنند. برخلاف قراردادهای مرسوم، قراردادهای هوشمند اقدامات لازم را به‌صورت خودکار و در صورت برآورده شدن شرایط از پیش تعیین شده انجام می‌دهند، بنابراین نیاز به واسطه‌ها را حذف کرده و آن‌ها را به ابزاری حیاتی در تضمین اعتماد و شفافیت در معاملات تجاری مانند املاک و مستغلات تبدیل می‌کند. در زمینه بلاکچین و توکنیزه کردن، قراردادهای هوشمند را می‌توان به این صورت تعریف کرد: قرارداد هوشمند یک برنامه رایانه‌ای است که در یک محیط امن اجرا شده و مستقیماً دارایی‌های دیجیتال را کنترل می‌کند. این تعریف را می‌توان به بخش‌های زیر گسترش داد:

- **برنامه رایانه‌ای:** یک برنامه رایانه‌ای شامل دستورالعمل‌هایی است که رایانه را در انجام یک کار خاص راهنمایی می‌کند.
- **محیط امن:** محیط‌های امن به اجرای دقیق، محرمانه بودن و یکپارچگی کد و داده‌ها نیاز دارند که در بلاکچین اتریوم تحت عنوان EVM که در بخش 2-3-1 توضیح داده شد، ظاهر می‌شوند.
- **کنترل مستقیم:** جنبه «کنترل مستقیم» قراردادهای هوشمند، به اجرای خودکار شرایط قرارداد بدون دخالت واسطه‌ها اشاره دارد. در قراردادهای هوشمند، برنامه با کنترل مستقیم، اجرای توافق را مدیریت می‌کند و نیاز به اعتماد به یک نهاد متمرکز را از بین می‌برد.
- **دارایی‌های دیجیتال:** دارایی‌های دیجیتال اقلامی هستند که در قالب دیجیتال وجود دارند و شامل

¹ Event

حق استفاده می‌شوند.

1-4-2 مقایسه قراردادهای هوشمند و قراردادهای قانونی مرسوم

قراردادهای هوشمند، در مقایسه با قراردادهای مرسوم، از ویژگی‌های منحصربه‌فردی برخوردارند که کارایی آن‌ها را در زمینه‌های خاص برجسته می‌کند. جدول زیر، تفاوت‌های کلیدی این دو نوع قرارداد را در حوزه‌های کاربرد، هزینه، سرعت، مکانیزم‌های اجرا و محدودیت‌های حقوقی به طور خلاصه نشان می‌دهد.

جدول 1-2 مقایسه قراردادهای هوشمند با قراردادهای مرسوم

قراردادهای هوشمند	قراردادهای مرسوم
مناسب ادعاهای عینی (قابل ارزیابی از طریق ریاضی)	مناسب ادعاهای ذهنی (نیاز به قضاوت انسانی)
کم هزینه	پر هزینه
سریع و خودکار	ممکن است نیاز به پروسه قانونی طولانی داشته باشد
متکی به وثیقه / سپرده‌های امنیتی	متکی به جریمه
بین‌المللی	محدود به حوزه قضایی

قراردادهای هوشمند در مدیریت ادعاهای عینی¹ با مبنای ریاضی بسیار خوب عمل می‌کنند و گزینه‌ای ساده و مقرون‌به‌صرفه را ارائه می‌دهند. اجرای خودکار این قراردادها، تضمین‌کننده‌ی سرعت و حذف فرایندهای قانونی پیچیده و زمان‌بر مرتبط با قراردادهای مرسوم است که اغلب نیازمند دخالت و نظر انسان و تفسیرهای ذهنی² هستند. قراردادهای هوشمند اغلب به سپرده‌های امنیتی (رمزارز یا توکن دیجیتال) در قالب وثیقه متکی هستند، بنابراین جایگزینی متمایز برای ماهیت متمرکز بر جریمه قراردادهای حقوقی مرسوم را ارائه می‌دهند. علاوه‌برآن، قراردادهای هوشمند پتانسیل عبور از مرزهای بین‌المللی را دارند و دامنه احتمالاً جهانی را فراهم می‌کنند در صورتی که قراردادهای حقوقی سنتی با محدودیت‌های قضایی یک کشور یا ایالت محدود می‌شوند. در نتیجه، قراردادهای هوشمند جایگزین قابل توجهی برای قراردادهای حقوقی مرسوم هستند؛ به‌ویژه در شرایطی که شامل ادعاهای عینی مبتنی بر ریاضیات است و نیاز به اجرای سریع و دسترسی جهانی در اولویت است.

5-2 توکن

توکن‌ها را می‌توان به‌عنوان واحدهای دیجیتال قابل مبادله توصیف کرد که نه تنها ارزش مالی و اقتصادی

¹ Objective

² Subjective

دارند، بلکه جنبه‌های دیگری مانند حق نشر، حق رأی، اعتبار، حق مالکیت و امتیاز دسترسی به خدمات را در بر می‌گیرند. در بلاکچین اتریوم، توکن‌ها از طریق قراردادهای هوشمند ایجاد و مدیریت می‌شوند که در جلوتر به استانداردهای آن خواهیم پرداخت. خارج از مقوله‌ی استانداردها، توکن‌ها را می‌توان در سه دسته‌بندی توکن‌های کاربردی¹، توکن‌های امنیتی² (اوراق بهادار) و توکن‌های دارایی³ طبقه‌بندی کرد. توکن‌های کاربردی به‌عنوان یک ابزار و کاربرد خاص (مانند رأی‌گیری) در یک سامانه استفاده می‌شوند و اهداف مالی و اقتصادی را می‌توانند به دنبال نداشته باشند. توکن‌های امنیتی شبیه اوراق قرضه یا سهام هستند و مستقیماً به شرکت صادرکننده آن‌ها مرتبط و محدود می‌شوند و نمی‌توان به‌صورت آزادانه آن‌ها را مبادله کرد و در نهایت، توکن‌های دارایی، نمایندگی و حق مالکیت دیجیتالی یک دارایی فیزیکی را در بر می‌گیرند.

اگرچه اصطلاحات ارز دیجیتال یا رمزارز⁴ و توکن⁵ اغلب به‌جای یکدیگر استفاده می‌شوند، اما در واقع به دو مفهوم متمایز اشاره می‌کنند. ارزهای دیجیتال (مانند بیت‌کوین و اتر)، دارایی‌های اصلی و بومی یک شبکه‌ی بلاکچین خاص هستند که موارد استفاده‌ی مختلفی از قبیل مبادلات و ذخیره ارزش، پرداخت هزینه‌ی کارمزد تراکنش و حفظ امنیت شبکه (عملکردهای جایزه و جریمه در الگوریتم‌های PoS و PoW توضیح داده شده در بخش‌های قبل) را ارائه می‌دهند. توکن‌ها در مقابل، بر اساس یک پروتکل از پیش تعریف شده عمل می‌کنند و به‌صورت غیربومی بر روی بلاکچین‌های موجود پیاده‌سازی می‌شوند.

2-5-1 استانداردهای توکن

توکن‌های شبکه‌ی اتریوم، بر اساس خاصیت قابل‌معاوضه بودن آن‌ها در دسته‌بندی‌های قابل‌معاوضه، غیرقابل‌معاوضه و نیمه قابل‌معاوضه تعریف می‌شوند که هر کدام استاندارد خاص خود را در شبکه‌ی اتریوم (و سایر شبکه‌های مبتنی بر اتریوم) دارند [11].

2-5-1-1 توکن قابل‌معاوضه⁶

توکن‌های قابل‌معاوضه یا تعویض‌پذیر، نوعی توکن با ارزش برابر هستند که می‌توانند با یکدیگر مبادله شوند. به‌عنوان مثال، یک واحد توکن USDT با یک واحد دیگر از توکن USDT دارای ارزش یکسانی است و می‌توان آن‌ها را بدون هیچ‌گونه تفاوت ارزشی معاوضه کرد. توکن‌های قابل‌معاوضه بر اساس

¹ Utility Tokens

² Security Tokens

³ Asset Tokens

⁴ Cryptocurrency

⁵ Token

⁶ Fungible Token

استاندارد ERC-20 عمل می‌کند که این استاندارد، در شبکه‌ی اتریوم و تمام شبکه‌های مبتنی بر اتریوم، به‌عنوان یک رابط¹ سراسری عمل می‌کند. استاندارد ERC-20، توابع و رویدادهایی را که یک قرارداد هوشمند معرف توکن قابل‌معاوضه باید با آن‌ها مطابقت داشته باشد را تعریف می‌کند. از جمله این توابع می‌توان به مواردی مانند نمایش موجودی، انتقال وجه و تأیید اشاره کرد. این توابع و رویدادها قراردادهای هوشمند مختلف را قادر می‌سازند تا با یکدیگر بر روی شبکه‌ی اتریوم در تعامل باشند و ارتباط برقرار کنند. از توکن‌های قابل‌معاوضه می‌توان به‌عنوان ارز و امتیاز و کاربردهای دیگری مانند حق رأی استفاده کرد [11].

2-1-5-2 توکن غیرقابل معاوضه² (NFT)

توکن‌های غیرقابل‌معاوضه یا NFTها در مقابل توکن‌های قابل‌معاوضه قرار می‌گیرند، زیرا هر توکن منحصربه‌فرد و متمایز است و این امر باعث می‌شود که ارزش‌های یکسانی نداشته و از این رو قابل‌معاوضه با یکدیگر نباشند. استاندارد ایجاد و انتقال این توکن‌ها در بلاکچین اتریوم ERC-721 است که به مشابه استاندارد قبلی، مجموعه‌ای از قوانین و عملکردها را تعریف می‌کند [17]. NFTها دارای ویژگی‌های خاص زیر هستند:

- **منحصربه‌فرد:** هر NFT توسط یک شناسه‌ی منحصربه‌فرد با عنوان شناسه‌ی توکن³ شناسایی می‌شود و آن را از سایر توکن‌های موجود تعریف شده در همان قرارداد هوشمند متمایز می‌کند. علاوه‌برآن، NFTها ممکن است دارای ویژگی‌های متمایزی مانند نام، توضیحات و تصویر در قالب فراداده⁴ باشند که ارزش یا مطلوبیت آن‌ها را در مقایسه با سایر توکن‌ها افزایش می‌دهد [17].
- **تقسیم‌ناپذیری:** NFTها را در چارچوب استاندارد ERC-721 نمی‌توان به واحدهای کوچک‌تر تقسیم کرد. هر NFT یک موجودیت کل و غیرقابل‌تقسیم است که توسط مالک یا اشخاص دارای مجوز می‌تواند به‌صورت یکجا منتقل شود.
- **مالکیت:** توکن‌های غیرقابل‌معاوضه با استفاده از رمزنگاری در قالب قراردادهای هوشمند ایمن می‌شوند و مالک هر توکن، کنترل و حقوق کاملی بر آن توکن دارد. مالکین و افراد دارای مجوز توسط مالک می‌توانند آزادانه و بدون هیچ واسطه یا محدودیتی توکن‌های خود را انتقال دهند، بفروشند و یا حتی آن‌ها را به دیگری در قالب وام قرض دهند.
- **قابلیت تأیید:** NFTها در بلاکچین به‌صورت شفاف قابل‌ردیابی هستند که این امر هر کسی را

¹ Interface

² Non-fungible Token

³ Token ID

⁴ Metadata

قادر می‌سازد که اصالت، مالکیت و سابقه یک NFT را در شبکه تأیید کند و از جعل یک توکن جلوگیری شود.

توکن‌های غیرقابل معاوضه (NFT) نوعی دارایی دیجیتال محسوب می‌شوند و می‌توانند موارد منحصربه‌فرد و کمیاب مانند آثار هنری و کلکسیون، کلیدهای دسترسی، بلیت‌های مختلف یک کنسرت یا رویداد ورزشی، بازی، دامنه (ENS¹) یا حق مالکیت و مدرک را نشان دهند [18].

2-5-1-3 توکن نیمه قابل معاوضه²

توکن‌های نیمه قابل معاوضه که به آن‌ها نیمه مثلی نیز می‌گویند نمایانگر هر دو نوع توکن قابل معاوضه و غیرقابل معاوضه هستند. این توکن‌ها دارای ویژگی‌های مشترک خاصی هستند اما می‌توانند به طور هم‌زمان ویژگی‌های منحصربه‌فردی نیز در مقایسه با سایر توکن‌های یک قرارداد داشته باشند که این ویژگی باعث انعطاف‌پذیرتر شدن این نوع توکن‌ها می‌شود. این توکن‌ها با استاندارد ERC-1155 به‌گونه‌ای تعریف می‌شوند که تمام عملکردهای توکن‌های ERC-20 و ERC-721 را به طور هم‌زمان شامل می‌گردند و باعث کارآمدتر شدن این استانداردها می‌شوند [11]. چنین توکن‌هایی در سناریوهایی کاربرد پیدا می‌کنند که حفظ تعادل بین قابلیت تعویض‌پذیری و منحصربه‌فرد بودن موردنظر است.

2-5-2 توکنیزه کردن³ و مزایای آن

توکنیزه کردن یک ایده و مفهوم نوین در حوزه فناوری بلاکچین است. این امر، به تبدیل حقوق یک دارایی، غالباً حقوق یک دارایی فیزیکی مانند املاک و مستغلات، به یک توکن دیجیتال در یک بلاکچین اشاره می‌کند [5].

فرایند توکنیزه کردن، بر اساس ماهیت بلاکچین مزایای زیر را برای ما فراهم می‌کند [5]:

- **دسترسی جهانی به سرمایه‌گذاری:** توکنیزه کردن به افراد کشورهای مختلف با زمینه‌های اقتصادی مختلف اجازه می‌دهد تا از طریق مالکیت جزئی و معاملات بدون واسطه در فرصت‌های سرمایه‌گذاری مختلف شرکت کنند.
- **نقدشوندگی بیشتر:** دارایی‌های توکن شده، به دلیل مالکیت جزئی و دسترسی گسترده‌تر به بازار، افزایش نقدشوندگی را نشان می‌دهند. برخلاف دارایی‌های فیزیکی، تجارت توکن‌های دیجیتال ساده‌تر است و بازاری پویاتر و روان‌تری را تشکیل می‌دهد و سرمایه‌گذاری با وجوه کمتر را برای سرمایه‌گذاران خرد ممکن می‌سازد.

¹ Ethereum Name Service

² Semi-Fungible Tokens

³ Tokenization

- **حذف نیاز به واسطه:** توکنیزه کردن دخالت واسطه‌هایی مانند بانک‌ها، وکلا یا کارگزاران را در معاملات دارایی‌ها کاهش داده و یا به طور کامل حذف می‌کند و در نتیجه کارمزدها و موانع مرتبط با واسطه‌های مرسوم را کاهش می‌دهد.
- **سرعت بیشتر:** توکنیزه کردن امکان تراکنش‌های فوری دارایی‌ها در بلاکچین را از طریق قراردادهای هوشمند و استانداردهای توکن فراهم می‌کند. انتقال توکن شامل هیچ کاغذبازی، دوره انتظار طولانی یا تأیید دستی نیست.
- **شفافیت:** توکنیزه کردن در بلاکچین، تضمین می‌کند که تمام تراکنش‌ها و سوابق مالکیت دارایی‌ها از طریق بلاکچین به طور عمومی قابل مشاهده و تأیید هستند.

6-2 سازمان غیرمتمرکز خودگردان¹ (DAO)

سازمان غیرمتمرکز خودگردان یا DAO، شکل جدیدی از سازمان است که بر پایه فناوری بلاکچین استوار می‌باشد. یک سازمان غیرمتمرکز خودگردان می‌تواند به طور مستقل و بدون هیچ‌گونه مرجع مرکزی یا سلسله‌مراتب مدیریتی عمل کند. تمامی قوانین و تصمیمات این سازمان‌ها در قالب قراردادهای هوشمند نوشته شده و بر اساس مکانیزم اجماع بر روی بلاکچین اجرا می‌گردند [19].

یک DAO دارای ویژگی‌های منحصربه‌فردی است که آن را از سازمان‌های معمولی متمایز می‌کند. DAO توزیع شده و غیرمتمرکز است و براین اساس هیچ نقطه کنترل یا نقطه‌ی تکی شکستی وجود ندارد. علاوه بر آن، DAO به طور مستقل و خودکار عمل می‌کند و بر یک منطق از پیش تعریف شده (در قالب قرارداد هوشمند) پایبند است و نیازی به دخالت مستقیم انسان ندارد. قوانین پیاده‌سازی شده در قراردادهای هوشمند با تخصیص دقیق مسئولیت‌ها، مشارکت‌ها و پاداش‌ها هماهنگی و نظم را در عملیات درون سازمان تضمین می‌کنند. DAO کاربردهای مختلفی دارد؛ می‌توان از آن برای ایجاد و مدیریت ارزهای دیجیتال، پلتفرم‌های عمومی، شبکه‌های اجتماعی، اینترنت اشیا و غیره استفاده کرد [19].

مزایای استفاده از DAO به شرح زیر می‌باشد:

- **بدون نیاز به اعتماد:** یک DAO بدون نیاز به نهاد مدیریت مرکزی به کار خود ادامه می‌دهد.
- **بدون تعطیلی:** یک DAO به صورت شبانه‌روزی و در تمام روزهای هفته در شبکه‌ی بلاکچین به کار خود ادامه می‌دهد و نمی‌توان آن را بدون اکثریت آرا سازمان تعطیل کرد.
- **متن‌باز²:** کدهای قراردادهای هوشمند یک DAO به صورت کاملاً شفاف در دسترس تمامی اعضا قرار دارد و قابل تأیید و بررسی است و این امر امکان تقلب در این سامانه‌ها را نزدیک به صفر

¹ Decentralized Autonomous Organization

² Open Source

می‌کند.

- توزیع شده و غیرمتمرکز: DAOها بر بستر بلاکچین پیاده‌سازی می‌شوند و یک شبکه‌ی بلاکچین به صورت ذاتی توزیع شده و غیرمتمرکز می‌باشد.
- برابری میان اعضا: هر توکن برابر یک رأی می‌باشد و به طور عمومی در DAOها هر عضو تنها یک توکن دارد. البته در برخی سازمان‌ها، حق رأی به فروش گذاشته شده و کسانی که قدرت خرید بیشتری دارند تعادل آرا را میان اعضا از بین می‌برند.

7-2 مرور ادبیات بلاکچین در املاک و مستغلات

همان‌طور که در بخش 1-4 بیان شد، تحقیقات مرتبط مبتنی بر بلاکچین در املاک و مستغلات به چهار دسته‌ی اصلی مدیریت اراضی، معاملات املاک و مستغلات، توکنیزه کردن و مدیریت این دارایی‌ها تقسیم می‌شود [6]. برخی از این تحقیقات به استفاده از توکن‌های امنیتی قابل معاوضه و توکن‌های ناهمتای غیرقابل معاوضه (NFT) برای توکنیزه کردن دارایی‌ها اشاره کرده‌اند که در همه‌ی موارد نقش قراردادهای هوشمند پر رنگ بوده است. از آنجایی که این پایان‌نامه در راستای توسعه و پیاده‌سازی قراردادهای هوشمند به جهت توکنیزه کردن و مدیریت املاک و مستغلات عمل می‌کند، تمرکز اصلی این بخش بررسی مقالات مبتنی بر پیاده‌سازی قراردادهای هوشمند برای توکنیزه کردن یا مدیریت املاک و مستغلات خواهد بود.

2-7-1 توکنیزه کردن املاک و مستغلات

توکنیزه کردن املاک و مستغلات یک مفهوم نسبتاً جدید در دنیای سرمایه‌گذاری املاک و مستغلات است که در نتیجه پیشرفت فناوری بلاکچین پدیدار شده است. چندین مطالعه، توکنیزه کردن املاک و مستغلات را به عنوان وسیله‌ای برای حل مشکلات بازار مرسوم مانند عدم شفافیت، ناکارآمدی، تقلب و فساد، هزینه‌های بالا و مشکلات مربوط به اعتماد معرفی کرده‌اند [6]. مقالات موجود مرتبط با توکنیزه کردن املاک و مستغلات را می‌توان بر اساس نوع توکن مورد استفاده به دو بخش تقسیم کرد که در زیر به آن‌ها خواهیم پرداخت.

2-7-1-1 توکنیزه کردن املاک و مستغلات بر اساس توکن‌های قابل معاوضه

Gupta و همکاران در مقاله‌ای با عنوان «توکنیزه کردن املاک و مستغلات با استفاده از فناوری بلاکچین» سامانه‌ای را برای توکنیزه کردن دارایی‌های املاک و مستغلات مبتنی بر بلاکچین اتریوم و قراردادهای هوشمند پیشنهاد می‌کنند [20]. این سامانه به صاحبان دارایی اجازه می‌دهد تا پس از احراز هویت در سامانه بتوانند از طریق قراردادهای هوشمند یک ابزار ویژه مالی (SPV^1) ایجاد کنند که این SPV به عنوان

¹ Special Purpose Vehicle

مالک قانونی این دارایی عمل می‌کند و وظیفه‌ی صادرکردن توکن‌ها و مدیریت ملک را بر عهده دارد. توکن‌های این سامانه از استاندارد ERC-777 پیروی می‌کنند و می‌توانند در بازارهای ثانویه معامله شوند. استاندارد ERC-777 یک استاندارد توکن قابل معاوضه است که قابلیت‌های استاندارد ERC-20 را بهبود می‌دهد [11]. توکن‌های ERC-777 ساخته شده در این سامانه، حکم سهام از SPV و مالکیت یک ملک را دارند و در فاز عرضه توکن‌ها (STO¹) توسط خریداران احراز هویت شده قابل خرید هستند. این سامانه همچنین توزیع ماهیانه سود سهام به دارندگان توکن را با استفاده از یک قرارداد هوشمند خودکار می‌کند.

2-1-7-2 توکنیزه کردن املاک و مستغلات بر اساس توکن‌های غیرقابل معاوضه

در مقاله‌ای تحت عنوان «توکنیزه کردن املاک و مستغلات از طریق توکن‌های غیرقابل معاوضه»، رویکرد جدیدی برای توکنیزه کردن دارایی‌های املاک و مستغلات با استفاده از توکن‌های غیرقابل معاوضه (NFT) در یک بلاکچین خصوصی توسط Will Serrano پیشنهاد می‌شود [21]. این مقاله نشان می‌دهد که چگونه NFTها و فناوری بلاکچین می‌توانند به طور بالقوه توکنیزه کردن املاک و مستغلات را با افزایش نقدشوندگی، دسترسی و شفافیت بازار همراه کنند. سهم پژوهش این مقاله در سه بخش ارائه‌ی توکنی جدید و سفارشی مبتنی بر NFT برای املاک و مستغلات، ارائه یک بلاکچین خصوصی برای مدیریت NFT و یک ارزیابی تجربی از مدل پیشنهادی خلاصه می‌شود. NFT سفارشی شده در این مقاله یک دارایی املاک و مستغلات فیزیکی و داده‌های مربوط به آن را نشان می‌دهد و می‌تواند در یک بازار ثانویه بین کاربران و ذی‌نفعان مختلف مبادله شود. زیرساخت بلاکچین خصوصی NFTها را در خود جای داده و دقت و شفافیت داده‌های آنها را با اجرای یک الگوریتم اجماع ترکیبی حفظ می‌کند. این مقاله همچنین با نشان دادن نتایج تجربی استفاده از مدل، قابلیت پیش‌بینی و مقیاس‌پذیری مدل NFT را برای مدل‌های پیچیده‌تر مشخص می‌کند.

فرایند اجرا شده برای ایجاد NFT در املاک و مستغلات این مقاله به شرح زیر است:

- یک توکن خصوصی در یک بلاکچین خصوصی با استفاده از زبان Java توسعه یافته است. این توکن از استاندارد ERC-721 برای NFTها پیروی می‌کند و حاوی اطلاعات ثابت و پویا مرتبط با دارایی فیزیکی است. اطلاعات ثابت شامل اسناد املاک، معاملات، اجاره‌نامه‌ها و قراردادها می‌باشد و اطلاعات پویا، تغییرات سالانه مانند هزینه‌های بیمه و مدیریت، سوابق مدیریت تسهیلات و حساب‌های دارایی را پوشش می‌دهد.
- NFTها از طریق یک شناسه توکن و آدرس حساب که مالکیت را به آن اختصاص می‌دهد، به دارایی‌های فیزیکی مرتبط می‌شوند. هر زمان که اطلاعات دارایی یا مالکیت تغییر کند، توکن

¹ Security Token Offering

به صورت دستی یا خودکار به روزرسانی می شود.

- توکن در یک بازار ثانویه توسط ذی نفعان مختلف از جمله مالکان، خریداران، سرمایه گذاران، ساکنان و بیمه گذاران معامله می شود. مدیران توکن ها مسئولیت به روزرسانی و اعتبارسنجی اطلاعات توکن ها در شبکه ی بلاکچین را بر عهده دارند.
- پروتکل اجماع این بلاکچین شامل ترکیبی از مکانیزم های اثبات کار و اثبات سهم است که در آن تأییدکنندگان شبکه به طور تصادفی بر اساس میزان سهامشان انتخاب می شوند. همچنین قانون انتخاب زنجیره، بر اساس طولانی ترین زنجیره با بالاترین سطح درجه سختی تجمیع شده می باشد.

2-7-2 سایر کاربردهای بلاکچین در املاک و مستغلات

جدا از کاربردهای توکنیزه کردن که باهدف افزایش نقدشوندگی و دسترسی صورت می گیرد، بلاکچین کاربردهای مختلف دیگری را در زمینه ی املاک و مستغلات دارد، در این بخش به طور خلاصه به دو کاربرد دیگر که به اهداف این پایان نامه نزدیک تر است خواهیم پرداخت.

1-2-7-2 اجاره ی خانه ی هوشمند مبتنی بر بلاکچین

Arcenegui و همکارانش راه حلی را برای اجاره خانه های هوشمند با استفاده از توکن های غیرقابل معاوضه (NFT) بر اساس استاندارد ERC-4519 پیشنهاد کرده اند [22]. این استاندارد اجازه می دهد تا دارایی های فیزیکی، توکن شده و کانال های ارتباطی امنی بین دارایی، مالک و کاربر استفاده کننده از آن ایجاد شود. برای رسیدن به این هدف، تراشه ی Pycom Wipy 3.0 به عنوان دروازه¹ خانه هوشمند برای تعامل با بلاکچین اتریوم استفاده و یک قرارداد هوشمند برای اجرای فرایند اجاره ایجاد شده است. در قرارداد هوشمند، شرایط اجاره NFT شامل قیمت، مدت زمان، مقدار سپرده و کاربر (مستأجر)، توسط مالک قرارداد هوشمند (صاحب خانه) تعیین می شود. همچنین قیمت و میزان مصرف منابع خدماتی مانند آب و گاز و برق توسط دروازه ی خانه ی هوشمند در قرارداد هوشمند مشخص می شود. علاوه بر آن، دروازه خانه هوشمند وضعیت ملک را کنترل می کند و هر گونه مشکل را به صاحب خانه گزارش می دهد. در نهایت نویسندگان این مقاله با استقرار قراردادهای هوشمند خود در شبکه آزمایشی Goerli و با اندازه گیری مدت زمان و هزینه تراکنش ها، امکان سنجی و کارایی راه حل خود را با موفقیت نشان داده اند.

2-2-7-2 مدیریت اراضی مبتنی بر بلاکچین

روشی کاربردی برای استفاده از قراردادهای هوشمند برای مدیریت معاملات و تراکنش های سامانه مدیریت اراضی توسط Stefanovic و همکاران پیشنهاد شده است [23]. نویسندگان مقاله سه مشکل اصلی را در سامانه های کنونی اداره ی اراضی شناسایی می کنند: دست کاری داده ها، کلاهبرداری از طریق فروش مجدد

¹ Gateway

و روند طولانی فرایند ثبت. آن‌ها یک قرارداد هوشمند مبتنی بر توکن ERC-721 طراحی کرده‌اند که می‌تواند طیف گسترده‌ای از سناریوهای مالکیت و انتقال زمین، از جمله اشتراک و تقسیم مالکیت و محدودکردن انتقالات املاک را در خود جای دهد. آن‌ها همچنین در مورد زیرساخت سامانه، روش پیاده‌سازی و محدودیت‌های راه‌حل خود بحث کرده‌اند. به‌طورکلی این مقاله روشی را نشان می‌دهد که در آن فناوری بلاکچین می‌تواند کارایی، امنیت، شفافیت و دقت سامانه‌های مدیریت اراضی را افزایش دهد. راه‌حل و روش پیشنهادی این مقاله موارد زیر را شامل می‌شود:

- **مالکیت اشتراکی:** قرارداد هوشمند چندین طرف را قادر می‌سازد تا بخشی از توکن املاک و مستغلات را که نماینده‌ی یک ملک انحصاری است، در اختیار داشته باشند. سهم مالکیت به‌صورت کسری یا درصدی بیان می‌شود. این مقاله مالکیت اشتراکی را از طریق تعریف توابع جدیدی مانند `shareOf` و `ownersOf` در قرارداد هوشمند ERC-721 که توابعی خارج از استاندارد NFT هستند، انجام می‌دهد.
- **انتقال جزئی:** مالکان می‌توانند از قرارداد هوشمند برای انتقال مقداری کمتر از ۱۰۰ درصد مالکیت خود به‌طرف دیگر استفاده کنند. این نوع انتقال در شرایطی مانند ارث یا شراکت رایج است. تراکنش انتقال با هش مختص به خود به‌عنوان سند بر روی بلاکچین ذخیره می‌شود. پیاده‌سازی این امر با تعریف پارامتر جدید `share_` در تابع انتقال استاندارد ERC-721 صورت گرفته است.
- **محدودیت انتقال:** قرارداد هوشمند می‌تواند توکن‌هایی را تحت شرایط قانونی یا مقرراتی خاصی مانند توقیف، کلاهبرداری یا تروریسم به‌صورت غیرقابل انتقال علامت‌گذاری کند. این محدودیت‌های انتقال ممکن است موقت یا دائمی باشند و می‌توانند توسط نهادهای مجاز حذف شوند. محدودکردن انتقال با پیاده‌سازی دو تابع جدید `isTransferable` و `setTransferable` در قرارداد هوشمند ERC-721 پیاده‌سازی شده است.
- **انتقال توسط شخص ثالث:** قرارداد هوشمند شخص ثالثی را قادر می‌سازد تا در سناریوهایی مانند حکم دادگاه، تأییدیه دفتر اسناد رسمی یا معامله باطل، علاوه بر مالک، مالکیت را منتقل کند. در شرایط ارثیه نیز از این نوع انتقال بر اساس حکم دادگاه یا دفتر اسناد رسمی استفاده می‌شود. انتقال توسط شخص ثالث در استاندارد ERC-721 با توابع `approve`، `transferFrom` و `setApprovalForAll` از پیش تعریف شده است.
- **تقسیم و ادغام املاک و مستغلات:** کاربرد تقسیم در این سامانه مطابق شرایطی است که یک مالک می‌خواهد ملک خود را به‌صورت جزئی، منتهی نه در حالت مالکیت اشتراکی، به یک یا چند نفر به فروش برساند و ادغام در شرایطی کاربرد دارد که یک مالک چندین زمین در کنار هم دارد و می‌خواهد آن‌ها را در قالب یک توکن جدید ادغام کند. در شرایط تقسیم، شناسه‌ی توکن

نماینده‌ی ملک به طور کل از سامانه حذف شده و توکن‌های جدیدی با شناسه‌های جدید تولید می‌شوند. در شرایط ادغام نیز چند شناسه‌ی ملک که نماینده‌ی ملک‌های مختلفی هستند حذف شده و یک توکن با شناسه‌ای جدید که نماینده‌ی یک ملک جدید است تولید می‌شود.

این مقاله از پلتفرم بلاکچین اتریوم و زبان برنامه‌نویسی Solidity برای پیاده‌سازی و استقرار قراردادهای هوشمند استفاده می‌کند. روش پیشنهادی این مقاله شامل پیاده‌سازی قابلیت‌های سامانه مدیریت اراضی در یک قرارداد هوشمند مبتنی بر استاندارد ERC-721 می‌شود. همچنین، قراردادهای هوشمند این مقاله در شبکه‌ی آزمایشی Ropsten مورد آزمایش قرار گرفته‌اند. شایان‌ذکر است که از اواخر سال ۲۰۲۲ و در زمان نگارش این پایان‌نامه، شبکه‌ی آزمایشی Ropsten منسوخ شده است [24].

2-7-3 یافته‌های تحقیقات موجود

ادبیات موجود پیشرفت‌های قابل توجهی در کشف پتانسیل فناوری بلاکچین در صنعت املاک و مستغلات داشته‌اند. یافته‌های کلیدی شامل موارد زیر است:

1. **توکنیزه کردن:** فناوری بلاکچین این پتانسیل را دارد که دارایی‌های املاک و مستغلات را توکنیزه کند و دسترسی و نقدشوندگی آن‌ها را افزایش دهد. این امر به طور بالقوه می‌تواند مشکلات بازارهای مرسوم مانند عدم شفافیت، ناکارآمدی، تقلب و فساد، زمان‌بر بودن، هزینه‌های بالا و مسائل مربوط به اعتماد را حل کند.
2. **مدیریت اراضی:** فناوری بلاکچین پتانسیل بهبود کارایی، امنیت، شفافیت و دقت سامانه‌های مدیریت زمین را دارد. علاوه‌بر آن، این امکان را دارد که مالکیت مشترک، نقل و انتقالات جزئی و همچنین تقسیم و ادغام املاک و مستغلات را تسهیل کند.
3. **قراردادهای هوشمند:** قراردادهای هوشمند نقشی حیاتی در پیاده‌سازی راه‌حل‌های مبتنی بر بلاکچین ایفا می‌کنند. آن‌ها فرایندهای متعددی را خودکار می‌سازند و نیاز به واسطه‌ها را کاهش و کارایی را افزایش می‌دهند.
4. **کاربردهای بالقوه:** علاوه بر توکنیزه کردن و مدیریت اراضی، فناوری بلاکچین کاربردهای بالقوه دیگری مانند سامانه‌های واسطه‌ای و اجاره، معاملات و سرمایه‌گذاری، تأمین مالی، مدیریت ساخت و ساز و تعمیر و نگهداری املاک و مستغلات را دارد [25].

این یافته‌ها یک زمینه محکمی برای تحقیق و توسعه‌های آینده در مورد فناوری بلاکچین و املاک و مستغلات ایجاد می‌کند. با این وجود، مقابله با شکاف‌ها و محدودیت‌های شناسایی شده در بخش بعدی، برای به فعلیت رساندن کامل پتانسیل بلاکچین در املاک و مستغلات بسیار مهم است.

2-7-4 شکاف‌ها و محدودیت‌های تحقیقات موجود

درحالی‌که تحقیقات پیشین بیش از ارزشمندی را در مورد اجرای فناوری بلاکچین در املاک و مستغلات ارائه می‌دهند، برخی از شکاف‌ها و محدودیت‌ها نیاز به توجه دارند:

- **دامنه‌ی محدود:** اکثر مطالعات صرفاً بر روی مواردی از کاربردهای بلاکچین در املاک و مستغلات مانند توکنیزه کردن یا اجاره به طور خاص متمرکز هستند و تحقیق جامعی که بر روی پیاده‌سازی یک بستر جامع برای استفاده از بلاکچین در املاک و مستغلات تمرکز داشته باشد یافت نشد.
- **محدودیت‌های فنی:** تحقیقات کنونی عمدتاً بر استفاده از بلاکچین اتریوم و زبان برنامه‌نویسی Solidity متمرکز است. با این وجود، پلتفرم‌های بلاکچین و زبان‌های برنامه‌نویسی دیگری نیز وجود دارند که ممکن است مزایای متمایزی داشته باشند. برای بررسی این گزینه‌ها، تحقیقات بیشتری مورد نیاز است.
- **چالش‌های مقرراتی:** پیامدهای قانونی و مقرراتی پیاده‌سازی فناوری بلاکچین در املاک و مستغلات به طور کامل بررسی نشده است. با توجه به مسائل حقوقی احتمالی که می‌تواند از توکنیزه کردن و دیگر کاربردهای بلاکچین در املاک سرچشمه گیرد، این امر یک حوزه حیاتی است که به بررسی بیشتر نیاز دارد.
- **عدم استفاده از ظرفیت توکن‌های جدید:** یکی از شکاف‌های برجسته فقدان استفاده از پروتکل‌های استاندارد شده و جدید برای توکنیزه کردن املاک و مستغلات است. درحالی‌که استانداردهای ERC-20 و ERC-721 به طور گسترده مورد استفاده قرار گرفته است، استفاده از استانداردهای ترکیبی و جدید دیگری مانند ERC-1155 پیشنهاد نشده است [22]. همچنین استاندارد استفاده شده در [20]، یعنی ERC-777، در سال ۲۰۲۱ در مستندات بلاکچین اتریوم و همچنین سایر کتابخانه‌های شناخته شده به دلایل امنیتی (حساسیت به انواع مختلف حمله) منسوخ شده و استفاده از آن دیگر پیشنهاد نمی‌گردد [11]، [26].
- **نگرانی‌های امنیتی:** با وجود شهرت فناوری بلاکچین به دلیل ویژگی‌های امنیتی ذاتی آن، خطرات و آسیب‌پذیری‌های بالقوه مرتبط با کاربرد آن در حوزه املاک و مستغلات به طور کامل مورد واکاوی قرار نگرفته‌اند. علاوه بر آن، در هیچ یک از مطالعات پیشین ذکر شده در این بخش، به ارزیابی امنیتی سامانه‌ی پیشنهادی پرداخته نشده است.
- **پذیرش و آموزش کاربران:** پذیرش کاربر یک عامل مهم در موفقیت برنامه‌های املاک و مستغلات مبتنی بر بلاکچین است. تأکید کافی بر آموزش سهام‌داران و ذی‌نفعان، مانند صاحبان املاک، سرمایه‌گذاران و عموم مردم، در مورد مزایا و پیچیدگی‌های سامانه‌های مبتنی بر بلاکچین انجام نشده است. تحقیقات آینده باید راهبردهای مؤثری را برای ارتقای پذیرش کاربر و توسعه درک

عمیق‌تر از فناوری بلاکچین در بخش املاک و مستغلات را بررسی کنند.

- **ادغام با سامانه‌های مرسوم:** مهاجرت از سامانه‌های املاک و مستغلات مرسوم به راه‌حل‌های مبتنی بر بلاکچین، نیازمند یکپارچگی روان و هموار است. تحقیقات می‌توانند روش‌هایی را برای پر کردن شکاف بین شیوه‌های فعلی و فناوری نوظهور بلاکچین را مورد بررسی قرار دهند و یک انتقال یکپارچه و بدون مشکل را برای کاربران مختلف تضمین کنند.

8-2 جمع‌بندی

این فصل به بررسی عمیق ادبیات موجود در زمینه فناوری بلاکچین و کاربردهای نوآورانه آن در صنعت املاک و مستغلات پرداخت. مفاهیم کلیدی بلاکچین مانند معماری، مکانیزم‌های اجماع، مزایا و محدودیت‌های آن شرح داده شد. همچنین پلتفرم اتریوم، شامل EVM، وب 3 و DApps، مورد بررسی قرار گرفت. قراردادهای هوشمند و تفاوت‌های اساسی آن با قراردادهای مرسوم، توکن‌ها، توکنیزه کردن و استانداردهای آن و DAOها نیز به طور مفصل بیان شدند. در ادامه، مطالعات پیشین مرتبط با کاربردهای بلاکچین در املاک و مستغلات، از جمله توکنیزه کردن املاک، اجاره هوشمند و مدیریت اراضی مورد بررسی قرار گرفتند. در نهایت، شکاف‌های تحقیقاتی و یافته‌های کلیدی از این مطالعات ارائه داده شد. این بررسی جامع از ادبیات موجود، نه تنها بستری محکم برای درک عمیق فناوری بلاکچین، ویژگی‌ها و کاربردهای بالقوه آن در صنعت املاک و مستغلات فراهم می‌کند، بلکه زمینه‌هایی را که نیازمند کاوش و تحقیق بیشتر هستند نیز برجسته می‌سازد و همچنین به عنوان نقطه شروعی قوی برای بخش‌های بعدی این پایان‌نامه عمل می‌کند.

فصل سوم

روش پیشنهادی

1-3 مقدمه

املاک و مستغلات به عنوان یکی از بازارهای پایدار و باارزش سرمایه‌گذاری، همواره به دنبال روش‌های جدید برای بهبود نقدشوندگی و دسترسی سرمایه‌گذاران بوده است. با ظهور فناوری بلاکچین، امکان تبدیل دارایی‌های فیزیکی به توکن‌های دیجیتال، در این حوزه فراهم شده است. در حالی که تحقیقات و پروژه‌های مختلفی تاکنون به کاربرد بلاکچین در توکنیزه کردن املاک پرداخته‌اند، بسیاری از آن‌ها تنها به یک جنبه خاص از این موضوع متمرکز شده‌اند که در بخش 2-7 به چهار مورد از این تحقیقات اشاره شد. این روش‌ها اغلب با محدودیت‌هایی مانند عدم پوشش کامل چرخه زندگی دارایی‌ها و عدم تسهیل مدیریت و پرداخت هزینه‌های اجاره و خدمات خصوصاً در مورد واحدهای اشتراکی مواجه هستند.

این فصل از پایان‌نامه به ارائه یک روش جامع برای حل این مسائل می‌پردازد. روش پیشنهادی این پایان‌نامه شامل طراحی و پیاده‌سازی یک سامانه مبتنی بر بلاکچین برای توکنیزه کردن املاک و مستغلات با استفاده از قراردادهای هوشمند است. نوآوری این پژوهش در ارائه یک رویکرد ترکیبی چند توکنه برای مدیریت سرمایه‌گذاری در املاک و مستغلات نهفته است. برای دستیابی به هدف تسهیل سرمایه‌گذاری و مدیریت املاک و مستغلات، روش پیشنهادی شامل ایجاد یک سامانه با چندین ویژگی کلیدی است. در مرحله اول، این سامانه، فروش، پیش‌فروش و عرضه اولیه املاک را به صورت شفاف و ایمن از طریق قراردادهای هوشمند استانداردهای مختلف توکن امکان‌پذیر می‌کند. علاوه بر آن، این سامانه مدیریت دارایی را از طریق استفاده از سازمان غیرمتمرکز خودگردان (DAO) تسهیل می‌کند. در ادامه، این سامانه جمع‌آوری هزینه‌های ساختمان را با استفاده از قراردادهای هوشمند ساده کرده و از این طریق، جمع‌آوری و مدیریت کارآمد و شفاف هزینه‌ها را در بستر بلاکچین تضمین می‌کند. در این فصل، ابتدا یک شمای کلی از مراحل عرضه املاک را توضیح می‌دهیم و در ادامه به پیاده‌سازی این بخش و سایر بخش‌های این سامانه مانند DAO و جمع‌آوری هزینه‌های ساختمان (اجاره و سرانه‌ی خدمات)، در قالب قراردادهای هوشمند خواهیم پرداخت.

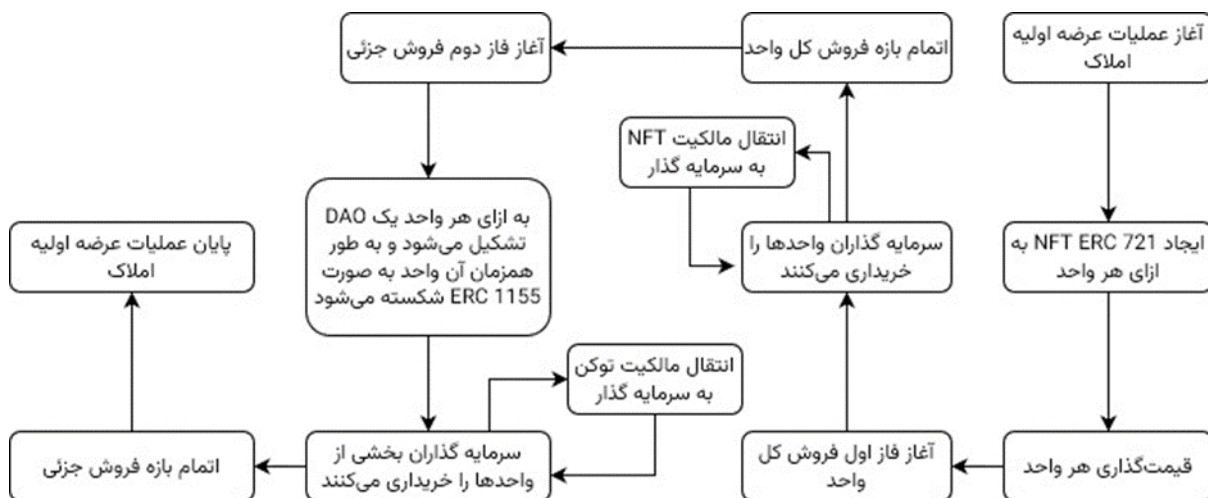
2-3 معرفی سامانه پیشنهادی

سامانه پیشنهادی این پایان‌نامه بر اساس فرایندهای اصلی به سه بخش مجزا تقسیم می‌شود: عرضه اولیه از طریق توکنیزه کردن، مدیریت واحدهای تقسیم شده از طریق DAO و مدیریت و جمع‌آوری هزینه‌های سرانه خدمات ساختمان. در این بخش صرفاً به معرفی اجمالی این سه بخش پرداخته می‌شود و جزئیات هر یک در بخش‌های بعدی به طور کامل تشریح خواهد شد.

1-2-3 عرضه اولیه املاک

عرضه اولیه املاک در سامانه پیشنهادی حول مفهوم توکنیزه کردن پیاده‌سازی می‌گردد. در توکنیزه کردن، کل واحدهای ساختمان به توکن‌های دیجیتالی تبدیل می‌شوند و بستری برای مالکیت جزئی و افزایش

نقدشوندگی فراهم می‌کنند. فرایند توکنیزه کردن این سامانه از دو استاندارد ERC-721 و ERC-115 که در بخش 2-5-1 توضیح داده شد، تبعیت می‌کند. توکنیزه کردن ابتدا از طریق عرضه اولیه املاک و در دو مرحله انجام می‌شود. شکل 3-1 شمای کلی روش پیشنهادی برای توکنیزه کردن را نشان می‌دهد. هدف از پیشنهاد دوم مرحله‌ای برای توکنیزه کردن املاک و مستغلات، ارائه یک رویکرد منعطف و همه‌کاره است که خواسته‌ها و الزامات مختلف سرمایه‌گذاران را بر اساس شرایط بازار و توانایی مالی آن‌ها برآورده سازد. در مرحله اول، هر واحد ساختمان به طور کامل و به عنوان یک نهاد واحد به بازار عرضه می‌شود و به خریداران بالقوه‌ای که قصد مالکیت کامل را دارند، فرصت خرید آن را می‌دهد. مرحله بعدی که مالکیت کسری را ارائه می‌کند، سرمایه‌گذاری در املاک را برای سرمایه‌گذاران خرد ممکن می‌سازد.



شکل 3-1 شمای کلی روش پیشنهادی جهت عرضه اولیه املاک

3-1-2-1 گام اول عرضه اولیه: فروش کل واحد

در گام اول، برای هر واحد در ساختمانی که قرار است برای فروش یا پیش‌فروش عرضه شود، یک توکن NFT منحصر به فرد (ERC-721) ایجاد و توکنیزه می‌شود. هر NFT نمایانگر یک واحد کامل از ساختمان است و برای خرید در سامانه به ذی‌نفعان ارائه می‌شود. سرمایه‌گذاران و خریداران می‌توانند با مراجعه به سامانه، NFTها را بررسی و با پرداخت قیمت واحد، توکن را خریداری کنند. قیمت هر توکن از قبل توسط مالک ملک تعیین شده است.

3-1-2-2 گام دوم عرضه اولیه: فروش جزئی

پس از اتمام مرحله توکنیزه کردن اولیه و فروش کلی واحدها، فرایند فروش جزئی واحدهای باقی‌مانده آغاز می‌شود. این مرحله با استفاده از استاندارد ERC-1155 امکان مالکیت جزئی را فراهم می‌کند. در این مرحله، واحدهای باقی‌مانده از ساختمان به بخش‌های مجزا در قالب توکن‌های ERC-1155 تقسیم می‌شوند. هر توکن نیمه‌مثلی، بیانگر سهمی از واحد NFT اصلی است؛ به این معنی که سرمایه‌گذاران می‌توانند با توجه به نیاز سرمایه‌گذاری یا توان مالی خود، بخش‌هایی از یک واحد در ساختمان را خریداری

و در سرمایه‌گذاری مشارکت کنند.

2-2-3 مدیریت و اجاره‌ی واحدهای تقسیم شده

در سامانه پیشنهادی، پس از فروش یک واحد تقسیم‌شده در ساختمان به چندین سرمایه‌گذار، یک سازمان غیرمتمرکز خودگردان (DAO) برای مدیریت آن واحد تشکیل می‌شود. در واقع، هنگامی که در گام 2-1-2-3 یک ملک به سهم‌های کوچک‌تر تقسیم می‌شود، یک DAO بر روی بستر بلاکچین برای مدیریت توکن اصلی NFT ایجاد می‌گردد. سرمایه‌گذاران جدیدی که توکن‌های نیمه قابل معاوضه ERC-1155 (سهم‌های خرد شده NFT) را خریداری می‌کنند، به طور خودکار عضو این DAO می‌شوند. قدرت رأی‌گیری هر عضو در تصمیم‌گیری‌های DAO بر اساس میزان موجودی توکن او (سهم او از NFT اصلی) تعیین می‌شود. به عبارتی، DAO به‌عنوان نماینده کلیه سهام‌داران عمل می‌کند و مسئول تصمیم‌گیری در مورد مدیریت و اجاره واحد خواهد بود.

برای اجاره واحد، DAO با رأی اکثریت اعضای خود مستأجر جدیدی را انتخاب می‌کند و مبلغ اجاره واحد بر اساس سهم هر عضو در DAO بین آن‌ها تقسیم می‌شود. برای جلوگیری از سوءاستفاده، می‌توان سامانه را به‌گونه‌ای طراحی کرد که قیمت پایه اجاره توسط کارشناسان سامانه تعیین شود و DAO فقط مجاز به تعیین قیمت اجاره در بازه‌ی قیمت کارشناسی شده باشد.

3-2-3 مدیریت و جمع‌آوری هزینه سرانه‌ی خدمات ساختمان

به‌منظور حفظ کیفیت و کارایی ساختمان، از تمام ساکنان برای خدمات مشترکی مانند نظافت، نگهداری و تعمیرات، هزینه دریافت می‌شود. در این سامانه، مدیریت و پرداخت این هزینه‌ها به طور خودکار با استفاده از قراردادهای هوشمند انجام می‌شود. این هزینه از همه ساکنان جمع‌آوری و در یک حساب امن و شفاف در بلاکچین نگهداری می‌شود. این حساب که در واقع یک قرارداد هوشمند است، قبل از فرایند توکنیزه‌کردن ساختمان (پیش از گام 1-1-2-3) برای کل ساختمان ایجاد می‌شود. در صورت نیاز به خدمات یا تعمیرات، وجوه لازم توسط مدیر ساختمان، که فرایند انتخابات او نیز در سامانه و در بستر بلاکچین صورت می‌گیرد، از این حساب برداشت و برای پرداخت هزینه‌ها استفاده می‌شود.

این رویکرد مزایای زیر را به همراه دارد:

- **کارایی و شفافیت:** ضمانت می‌شود که هزینه‌های خدمات به طور کامل جمع‌آوری و به طور مؤثر مورد استفاده قرار گیرند. تمامی تراکنش‌ها برای شفافیت کامل در بلاکچین ثبت می‌شوند.
- **آسانی استفاده:** نیاز به فرایندهای صورت‌حساب و پرداخت دستی را از بین می‌برد و این امر را برای ساکنین آسان‌تر می‌کند.
- **کاهش اختلافات:** با ثبت شفاف کلیه تراکنش‌ها در بلاکچین، از بروز اختلافات در مورد

پرداخت‌ها جلوگیری می‌شود.

3-3 بلاکچین و فناوری‌های مورد استفاده

سامانه توکنیزه کردن املاک و مستغلات پیشنهادی بر بستر بلاکچین اتریوم توسعه خواهد یافت. درحالی‌که بسیاری از شبکه‌های بلاکچینی موجود امکان پیاده‌سازی قراردادهای هوشمند را فراهم می‌کنند، اتریوم به دلیل پایداری بالا، پشتیبانی از زبان برنامه‌نویسی Solidity و همچنین مستندات جامع آن انتخاب شده است. این انتخاب، اجرای امن، شفاف و خودکار فرایندهای مختلف در سامانه را تضمین می‌کند و در نتیجه هزینه‌های توسعه را کاهش می‌دهد. در ادامه به بررسی هر یک از فناوری‌های مورد استفاده در پیاده‌سازی و اجرای این پایان‌نامه خواهیم پرداخت.

3-3-1 شبکه‌های آزمایش و توسعه اتریوم

بلاکچین اتریوم به دلیل سابقه اثبات شده و پذیرش گسترده‌ی آن در حوزه‌ی برنامه‌های غیرمتمرکز (DApps) و قراردادهای هوشمند، به‌عنوان پلتفرم بلاکچین جهت سامانه پیشنهادی توکنیزه کردن املاک و مستغلات انتخاب شده است. توضیحات و ویژگی‌های این بلاکچین در بخش 2-3 آمده است. استفاده از بلاکچین اتریوم و استانداردهای آن، سازگاری سامانه را با سایر بلاکچین‌های موجود مبتنی بر اتریوم تضمین می‌کند و قابلیت تعامل با سایر برنامه‌ها و خدمات غیرمتمرکز را تسهیل می‌کند. اتریوم استانداردهایی را برای ایجاد یک دانش مشترک ارائه می‌دهد که ارتباط بین کاربران، برنامه‌ها، کیف پول‌ها و قراردادهای هوشمند را تسهیل و مستند می‌کند. این استانداردها که به‌عنوان درخواست اتریوم برای اظهارنظر (ERC¹) شناخته می‌شوند، به طور مداوم توسط جامعه اتریوم توسعه می‌یابند. پیشنهادهای بهبود اتریوم (EIP²) نیز برای معرفی ویژگی‌های فنی جدید و بهبود ویژگی‌های موجود استفاده می‌شود [10]. در دسترس بودن این استانداردها، مستندات و همچنین توسعه مستمر اتریوم، آن را به یک سامانه ایدئال برای توکن‌سازی املاک و مستغلات تبدیل می‌کند و راهی مطمئن و کارآمد برای مدیریت دارایی‌ها ارائه می‌دهد.

در طول مراحل توسعه و آزمایش سامانه پیشنهادی، داشتن یک رویکرد امن و منعطف در استفاده از شبکه‌ی اتریوم ضروری است. بسیار مهم است که قراردادهای هوشمند و برنامه‌های کاربردی غیرمتمرکز در حال توسعه را در محیطی شبیه به محیط اصلی اتریوم، بدون به خطر انداختن سرمایه‌ها یا دارایی‌های واقعی و بدون نیاز به پرداخت هزینه‌های کارمزد، مستقر و آزمایش کرد. برای این منظور، از دو نوع شبکه‌ی مختلف استفاده می‌شود که هر کدام اهداف مشخصی را دنبال می‌کنند.

¹ Ethereum Request for Comments

² Ethereum Improvement Proposal

شبکه‌های آزمایشی:

اتریوم چندین شبکه‌ی آزمایشی مانند Goerli یا Sepolia را ارائه می‌دهد که شبکه‌ی اصلی اتریوم را شبیه‌سازی می‌کنند، با این تفاوت که در این شبکه‌ها، به‌جای اتر واقعی از اتر آزمایشی استفاده می‌شود. استفاده از شبکه‌های آزمایشی، توسعه‌دهندگان را قادر می‌سازد تا قراردادهای برنامه‌های خود را در یک محیط شبیه‌سازی شده و مشابه محیط اصلی، بدون داشتن پیامدهای مالی آزمایش کرده و قبل از استقرار در شبکه اصلی اطمینان حاصل کنند که همه چیز مطابق انتظار کار می‌کند. علاوه‌بر آن، شبکه‌های آزمایشی برای اشکال‌زدایی و عیب‌یابی ارزشمند هستند، زیرا راهی برای بازتولید مشکلات بدون تأثیرگذاری بر شبکه اصلی ارائه می‌کنند.

شبکه‌های محلی:

برای توسعه محلی، ابزارهایی مانند شبکه Ganache و Hardhat وجود دارند که توسعه‌دهندگان را قادر می‌سازند تا یک شبکه محلی مشابه بلاکچین اتریوم را بر روی دستگاه خود راه‌اندازی کنند. این شبکه‌ها به‌صورت محلی عمل کرده و محیطی سریع و کارآمد برای استقرار و آزمایش قراردادهای هوشمند فراهم می‌کنند. این امر به‌خصوص برای توسعه و آزمایش قراردادهای هوشمند به‌صورت خودکار مفید است. توسعه‌دهندگان می‌توانند قبل از استقرار در شبکه‌های آزمایشی یا اصلی، به‌سرعت سناریوهای مختلف را تکرار و آزمایش کرده و قراردادهای هوشمند را در یک محیط کنترل شده اشکال‌زدایی کنند.

در فرایند توسعه سامانه پیشنهادی، از شبکه‌های آزمایشی و محلی ذکر شده استفاده خواهیم کرد تا قبل از استقرار در شبکه اصلی اتریوم، عملکرد صحیح و بدون نقص قراردادهای هوشمند توسعه‌یافته شده را در محیطی امن و کنترل‌شده مشابه بلاکچین اتریوم، تضمین کنیم. این امر به ما امکان می‌دهد تا قبل از بروز هرگونه مشکل یا خطایی که ممکن است به کاربران آسیب مالی برساند، آن‌ها را شناسایی و برطرف نماییم.

2-3-3 قراردادهای هوشمند Solidity

قراردادهای هوشمندی که بر فرایندهای توکنیزه‌کردن و مدیریت املاک و مستغلات حاکم هستند، با استفاده از زبان برنامه‌نویسی Solidity که یک زبان برنامه‌نویسی برای قراردادهای هوشمند است، نوشته می‌شوند. Solidity ویژگی‌ها و عملکردهای منطق‌های تجاری پیچیده موردنیاز ما را فراهم می‌کند و یکپارچگی و قابلیت اطمینان عملیات سامانه را تضمین می‌کند.

3-3-3 محیط توسعه Hardhat

سامانه پیشنهادی از ابزار توسعه‌ی اتریوم Hardhat برای اهداف توسعه و آزمایش استفاده می‌کند. Hardhat با ارائه مجموعه‌ای کامل از ابزارها برای کامپایل، استقرار و آزمایش قراردادهای هوشمند، فرایند توسعه را ساده می‌کند [27]. توسعه در این محیط با استفاده از زبان‌های برنامه‌نویسی JavaScript و TypeScript

صورت گرفته و آن را به گزینه‌ای محبوب برای پروژه‌های مبتنی بر اتریوم تبدیل کرده است.

4-3-3 سامانه پرونده IPFS¹

همان‌طور که در بخش 2-3-5 ذکر شد، تمام عملیات‌های بلاکچین مشمول هزینه‌ی کارمزد می‌باشد و هرچقدر که عملیات پیچیده‌تر باشد هزینه‌ی بیشتری باید پرداخت گردد. در سامانه پیشنهادی، ما نیاز به یک سامانه پرونده داریم تا اطلاعات مربوط به املاک و مستغلات را در آن ذخیره کنیم و همان‌طور که مسلم است، استفاده از بلاکچین به‌عنوان سامانه پرونده به دلیل حجم و تعداد زیاد اطلاعات، بسیار پرهزینه است. از طرفی استفاده از سایر سامانه‌های پرونده‌ی متمرکز در تضاد با ماهیت غیرمتمرکز بلاکچین می‌باشد. برای حل این مشکل، از سامانه پرونده‌ی IPFS برای ذخیره‌سازی غیرمتمرکز و توزیع‌شده‌ی داده‌های ضروری، به‌ویژه فراداده‌های مرتبط با توکن‌های املاک استفاده خواهیم کرد. IPFS یک راه‌حل پایدار و مقاوم در برابر تغییر مبتنی بر هش را برای ذخیره حجم زیادی از داده‌ها به‌طوری‌که با اصول اساسی بلاکچین، مانند تمرکززدایی و تغییرناپذیری همسو باشد ارائه می‌دهد. این تضمین می‌کند که اطلاعات حیاتی دارایی‌های املاک و مستغلات به‌طور ایمن، کارآمد و غیرقابل‌تغییر و به‌صورت غیرمتمرکز ذخیره می‌شوند. ادغام IPFS با سامانه پیشنهادی، با اصول غیرمتمرکز فناوری بلاکچین همسو می‌باشد و به انعطاف‌پذیری و قابلیت اطمینان کلی سامانه توکنیزه کردن املاک و مستغلات کمک می‌کند.

4-3-4 نمای کلی سامانه

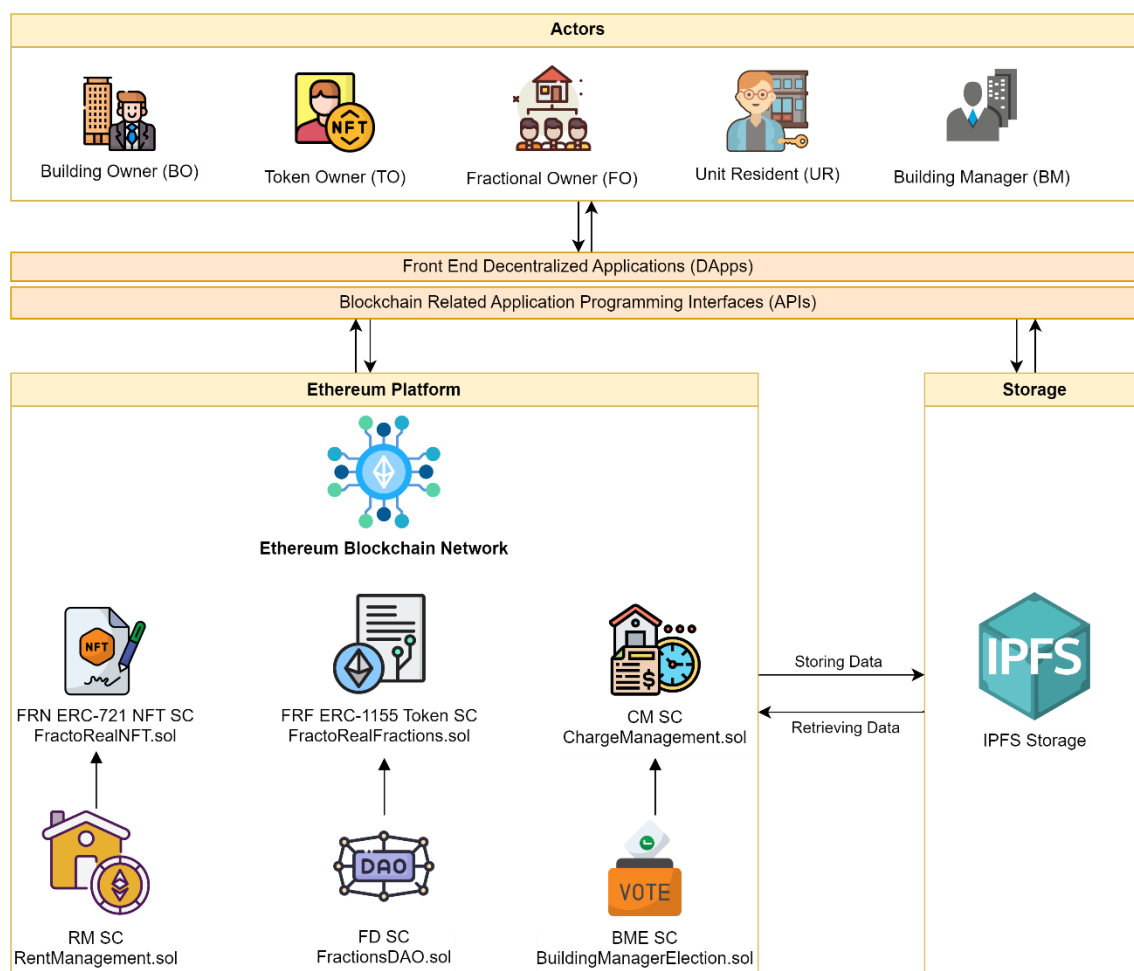
سامانه توکنیزه کردن املاک و مستغلات پیشنهادی از قراردادهای هوشمند مبتنی بر اتریوم برای ایجاد یک دفترکل امن و شفاف برای ثبت جزئیات مالکیت دارایی‌های املاک استفاده می‌کند. معماری این سامانه به‌گونه‌ای طراحی شده است که تا با بهره‌گرفتن از استانداردهای مرتبط، به‌طور کامل در دسترس عموم باشد. نهادهای مختلف از طریق رابط فرانت‌اند برنامه‌های کاربردی غیرمتمرکز (DApps) با قراردادهای هوشمند در تعامل هستند. هر بازیگر این سامانه از یک آدرس اتریوم منحصربه‌فرد (EOA) استفاده می‌کند که از طریق هش کلید عمومی آن‌ها ایجاد می‌گردد. توجه داشته باشید که این پایان‌نامه به‌طور خاص بر جنبه بلاکچین و پیاده‌سازی قراردادهای هوشمند سامانه توکنیزه کردن و مدیریت املاک و مستغلات تمرکز دارد و شامل بخش طراحی رابط کاربری نمی‌شود. دلیل این امر آن است که عناصر وب ۲ مانند توسعه بک‌اند و فرانت‌اند هیچ ارزش، سهم یا نوآوری در راستای هدف این پایان‌نامه ندارند و می‌توان آن‌ها را به‌طور آزادانه و به هر روشی برای تعامل با قراردادهای هوشمند سامانه پیشنهادی طراحی کرد.

در معماری روش پیاده‌سازی سامانه پیشنهادی، رویدادهایی نیز توسط توابع قرارداد هوشمند ایجاد و از طریق رابط‌های برنامه‌نویسی کاربردی شبکه بلاکچین (API) به تمام مشارکت‌کنندگان شبکه ارسال

¹ InterPlanetary File System

می‌شوند. با مشاهده و رصد این رویدادها، می‌توان از هر تغییر کلیدی و مهم در وضعیت بلاکچین به‌صورت لحظه‌ای آگاه شد و اقدامات دلخواه را بر اساس آن‌ها انجام داد. جزئیات و فهرست کامل این رویدادها در بخش‌های بعدی به‌طور مفصل شرح داده خواهد شد.

پیاده‌سازی پیشنهادی این پایان‌نامه بر توکنیزه کردن واحدهای یک ساختمان با قابلیت تقسیم آن‌ها به قطعات ریزتر برای مالکیت اشتراکی تمرکز دارد. این قطعات توکن شده را می‌توان با استفاده از سازمان‌های مستقل غیرمتمرکز (DAO) مدیریت کرد و همچنین می‌توان آن‌ها را به واحدهای کامل و بالعکس تبدیل کرد. شکل 2-3 معماری روش پیشنهادی سامانه را نشان می‌دهد که بازیگران کلیدی از جمله مالک ساختمان، مالک واحد، مالک جزئی واحد و مدیر ساختمان و همچنین قراردادهای هوشمند این سامانه را شامل می‌شود.



شکل 2-3 شمای کلی از معماری روش سامانه پیشنهادی

1-4-3 بازیگران سامانه

موفقیت و کارایی یک سامانه غیرمتمرکز به نقش‌ها و مسئولیت‌هایی که به بازیگران مختلف در شبکه تخصیص داده می‌شود بستگی دارد. در سامانه توکنیزه کردن املاک و مستغلات پیشنهادی، چندین بازیگر

کلیدی نقش‌های محوری در راه‌اندازی، عملکرد و مدیریت اکوسیستم دارند. در ادامه به شرح وظایف هر نقش در این سامانه می‌پردازیم.

3-1-4-1 مالک ساختمان¹ (BO)

مالک ساختمان فرایند توکنیزه کردن را آغاز و بر آن نظارت می‌کند. این عملیات شامل استقرار قراردادهای هوشمند، پیکربندی قیمت و متراژ واحدها و آپلود اطلاعات هر توکن در IPFS است. مسئولیت‌ها و توانایی‌های مالک ساختمان به شرح زیر است:

1. استقرار قرارداد هوشمند: مالک ساختمان کلیه قراردادهای هوشمند را در شبکه مستقر می‌کند.
2. پیکربندی توکن‌ها: شامل تعیین قیمت، متراژ و آپلود اطلاعات هر واحد در IPFS است.
3. توکنیزه کردن: مالک روند کلی توکنیزه کردن را آغاز و بر آن نظارت می‌کند.

3-1-4-2 مالک توکن² (TO)

مالک توکن به مالک هر توکن NFT در سامانه اشاره دارد. این مالکان یک واحد را به طور کامل خریداری کرده‌اند و مسئولیت مدیریت و انتقال آن واحد را به‌تنهایی بر عهده دارند. این وظایف شامل موارد زیر می‌شود:

1. خرید کامل: می‌تواند در زمان عرضه اولیه یا در بازارهای ثانویه، مالکیت کامل یک یا چند واحد را خریداری کند.
2. مدیریت اجاره: اختیار تعیین مستأجر، تعیین قیمت اجاره و دریافت آن را دارد.
3. قابلیت تقسیم: می‌تواند واحد خود را برای مالکیت اشتراکی به قطعات کوچک‌تر تقسیم کند.
4. پرداخت هزینه سرانه‌ی خدمات: موظف به پرداخت ماهانه هزینه‌های خدمات واحد است. در صورت وجود مستأجر، این وظیفه بر عهده مستأجر است.
5. انتخابات مدیر ساختمان: می‌تواند در انتخابات مدیر ساختمان خود را به‌عنوان نامزد معرفی کرده و در فرایند رای‌گیری شرکت کند.

3-1-4-3 مالک جزئی³ (FO)

مالک جزئی به مالک هر توکن کسری (ERC-1155) در سامانه اشاره دارد. این مالکان مالکیت یک توکن NFT را به‌صورت اشتراکی در اختیار دارند و می‌توانند با مشارکت سایرین و از طریق DAO، قابلیت‌ها و

¹ Building Owner

² Token Owner

³ Fractional Owner

توانایی‌های یک مالک کلی (TO) را به اشتراک بگذارند. مسئولیت‌ها و توانایی‌های مالک جزئی به شرح زیر است:

1. **خرید جزئی:** می‌تواند در مرحله فروش ثانویه یا بعد از آن در بازارهای ثانویه، مالکیت جزئی یک یا چند واحد را به مقدار دلخواه خود خریداری کند.
2. **مشارکت در مدیریت:** می‌تواند با رأی‌گیری و مشارکت در DAO، در وظایف ذکر شده در بخش مالک توکن (TO) مشارکت کند.

4-1-4-3 ساکن واحد¹ (UR)

ساکن واحد به افرادی گفته می‌شود که توسط مالکان توکن برای سکونت در واحدها تعیین می‌شوند. ساکنان مسئولیت‌های مالی مرتبط با ملک را نیز بر عهده دارند. مسئولیت‌ها و اختیارات ساکن واحد به شرح زیر است:

1. **سکونت:** می‌تواند توسط مالک توکن به عنوان ساکن واحد تعیین شوند.
2. **پرداخت اجاره:** موظف به پرداخت ماهانه اجاره بها است.
3. **پرداخت هزینه‌ی سرانه‌ی خدمات:** موظف به پرداخت ماهانه سهم خود از هزینه‌های خدمات واحد است.
4. **انتخابات مدیر ساختمان:** می‌تواند در انتخابات مدیر ساختمان شرکت کند.

5-1-4-3 مدیر ساختمان² (BM)

مدیر ساختمان از میان مالکان (TO) واحدها در یک فرایند انتخاباتی بر بستر بلاکچین و با رأی اکثریت ساکنین یا مالکان واحدها انتخاب می‌شود. مدیر ساختمان مسئولیت مدیریت مالی هزینه‌های مشترک خدمات جمع‌آوری شده از ساکنین یا مالکان را بر عهده دارد. مسئولیت‌های مدیر ساختمان به شرح زیر است:

1. **مدیریت مالی:** تخصیص و انتقال وجوه مربوط به هزینه‌های مشترک خدمات که توسط ساکنین یا مالکان در قرارداد هوشمند مدیریت هزینه‌های خدمات پرداخت می‌شود.
2. **تسهیل وظایف مدیریت:** انجام سایر وظایف یک مدیر ساختمان، مانند رسیدگی به درخواست‌های تعمیرات و حل و فصل اختلافات ساکنین. این وظایف خارج از شبکه بلاکچین انجام می‌شود.

¹ Unit Resident

² Building Manager

3-4-2 معرفی قراردادهای هوشمند سامانه

این بخش به معرفی مختصری از قراردادهای هوشمند کلیدی که بر سامانه توکنیزه کردن املاک و مستغلات حاکم هستند، می‌پردازد. جزئیات پیاده‌سازی و معماری پیاده‌سازی هر قرارداد هوشمند در بخش‌های بعدی ارائه خواهد شد. به‌طور کلی، قراردادهای هوشمند این سامانه را می‌توان به دودسته زیر تقسیم کرد:

1. **قراردادهای توکنیزه کردن:** این قراردادها وظایف مربوط به فرایند توکنیزه کردن، مانند ایجاد توکن‌ها، مدیریت مالکیت و انتقال توکن‌ها را بر عهده دارند.

2. **قراردادهای مدیریتی:** این قراردادها وظایف مربوط به مدیریت و اداره دارایی‌ها، مانند مدیریت هزینه‌های اجاره و خدمات، مدیریت اشتراکی واحدها از طریق DAO و برگزاری انتخابات مدیر ساختمان را بر عهده دارند.

با وجود این دسته‌بندی دوگانه، هنگام استقرار قراردادها در شبکه بلاکچین، با استفاده از وراثت در برنامه‌نویسی شیء‌گرا، تنها سه قرارداد اصلی در شبکه مستقر می‌شوند. سایر قراردادها که قابلیت‌های بیشتری را به این سه قرارداد اصلی اضافه می‌کنند، انتزاعی هستند و به‌جای استقرار مجزا، به همراه قراردادهای اصلی در شبکه مستقر می‌شوند و قابلیت‌های جدیدی را به آن‌ها اضافه می‌کنند. در ادامه به معرفی قراردادهای هوشمند دو دسته‌بندی ذکر شده خواهیم پرداخت.

3-4-2-1 قراردادهای هوشمند توکنیزه کردن

طبق استانداردهای توکن که در بخش 2-5-1 معرفی شده‌اند، این قراردادها به دودسته توکن غیرقابل معاوضه (NFT) بر پایه ERC-721 و توکن نیمه قابل معاوضه (ERC-1155) تقسیم می‌شوند. هر دو این قراردادها جزو قراردادهای اصلی شبکه هستند و به صورت مجزا در شبکه مستقر می‌شوند.

3-4-2-1-1 مالکیت کلی¹ (FRN)

این قرارداد هوشمند به‌عنوان پایه و اساس فرایند توکنیزه کردن عمل می‌کند و واحدهای منحصربه‌فرد یک ساختمان را به‌عنوان توکن‌های غیرقابل معاوضه (NFT) نشان می‌دهد. بر اساس استاندارد ERC-721، این قرارداد وظایف مربوط به مالکیت، انتقال و مدیریت اجاره و تعیین مستأجر را مطابق با عملکرد بازیگران سامانه پیشنهادی، مدیریت می‌کند. علاوه‌بر آن، گام فروش اولیه و همچنین آغاز عملیات فروش ثانویه (مالکیت اشتراکی) به طور کامل در این قرارداد انجام می‌شود.

¹ FractoRealNFT.sol

2-1-2-4-3 مالکیت اشتراکی¹ (FRF)

این قرارداد پس از قرارداد FRN در شبکه مستقر می‌شود و عملیات فروش ثانویه که توسط قرارداد FRN آغاز می‌گردد را شامل می‌شود. این قرارداد هوشمند، طبق استاندارد ERC-1155، وظایف مربوط به ایجاد، مالکیت و انتقال واحدهای تقسیم شده را مدیریت می‌کند. علاوه بر آن، عملکردهای DAO مرتبط با مدیریت مالکیت اشتراکی نیز در این قرارداد هوشمند پیاده‌سازی شده است.

2-2-4-3 قراردادهای هوشمند مرتبط با مدیریت واحدها

قراردادهای این دسته‌بندی، شامل قراردادهای مرتبط با جمع‌آوری هزینه‌های سرانه‌ی خدمات، مدیریت و جمع‌آوری اجاره، مدیریت واحدهای اشتراکی از طریق DAO و همچنین انتخابات مدیر ساختمان می‌شود. از بین این قراردادهای، تنها قرارداد مدیریت هزینه‌های خدمات در کنار قراردادهای مرتبط با توکنیزه کردن، جزو سه قرارداد اصلی شبکه است و به طور مجزا در شبکه مستقر می‌شود. سایر قراردادهای هوشمند این دسته‌بندی توسط قراردادهای هوشمند اصلی ارث‌بری می‌شوند.

1-2-2-4-3 مدیریت هزینه‌های سرانه‌ی خدمات² (CM)

این قرارداد هوشمند وظایف مربوط به مدیریت و جمع‌آوری هزینه‌های سرانه‌ی خدمات یک ساختمان، مانند هزینه‌های تعمیر و نگهداری و سایر هزینه‌های مشترک را بر عهده دارد. مدیریت مالی و انتقال وجوه موجود در این قرارداد تنها در اختیار مدیر ساختمان منتخب ساکنین یا مالکان واحدها است. فرایند انتخاب مدیر ساختمان توسط قرارداد هوشمند دیگری که در 4-2-2-4-3 معرفی می‌شود صورت می‌گیرد.

2-2-2-4-3 مدیریت اجاره³ (RM)

این قرارداد هوشمند توسط قرارداد هوشمند FRN به ارث برده می‌شود و ویژگی‌ها و عملکردهایی را برای جمع‌آوری و مدیریت اجاره واحدهای توکنیزه شده ارائه می‌دهد. این عملکردها شامل توابعی مانند پرداخت و برداشت اجاره، مشخص کردن مستأجر و قیمت‌گذاری اجاره می‌شود.

3-2-2-4-3 مدیریت واحد اشتراکی⁴ (FD)

این قرارداد هوشمند به عنوان یک سازمان غیرمتمرکز خودگردان (DAO) باهدف مدیریت واحدهایی که مالکیت آن‌ها در قرارداد هوشمند FRF به صورت توکن‌های ERC-1155 تقسیم شده است، توسط قرارداد هوشمند FRF به ارث‌بری شده و به همراه آن در شبکه مستقر می‌شود. عملکردهای این قرارداد هوشمند

¹ FractoRealFractions.sol

² ChargeManagement.sol

³ RentManagement.sol

⁴ FractionsDAO.sol

شامل توابعی مانند ارسال و ثبت پیشنهاد، رأی دادن به یک پیشنهاد و اجرای یک پیشنهاد تأیید شده توسط آرا است. اعضای این DAO در واقع همان مالکان یک توکن FRF خاص هستند و قدرت رأی آن‌ها در فرایند تصمیم‌گیری، توسط تعداد توکن‌های FRF که در اختیار دارند، تعیین می‌شود. به این معنا که هر چه تعداد توکن‌های FRF یک مالک بیشتر باشد، قدرت رأی او نیز بیشتر خواهد بود.

3-4-2-4 انتخابات مدیر ساختمان¹ (BME)

این قرارداد هوشمند توسط قرارداد هوشمند مدیریت هزینه‌های سرانه (CM) وراثت شده و فرایند انتخابات مدیر ساختمان را که شامل وظایفی مانند معرفی نامزدها، ثبت رأی، و اعلام و ثبت برنده به‌عنوان مدیر ساختمان است، مدیریت می‌کند. در این قرارداد هوشمند، مالکان یا ساکنان یک واحد می‌توانند خود را به‌عنوان نامزد برای شرکت در انتخابات معرفی کنند. میزان موجودی توکن FRN هر فرد، قدرت رأی او را در انتخابات تعیین می‌کند. در نهایت، نامزدی که بیشترین رأی را به دست آورد، توسط قرارداد هوشمند در شبکه بلاکچین به‌عنوان مدیر ساختمان ثبت می‌شود.

3-5 پیاده‌سازی قراردادهای هوشمند

معماری روش سامانه پیشنهادی برای ادغام یکپارچه فناوری بلاکچین، قراردادهای هوشمند و برنامه‌های کاربردی غیرمتمرکز (DApps) جهت توکنیزه کردن و مدیریت کارآمد املاک و مستغلات، به طور دقیق گسترش داده شده است. این روش در هسته‌ی خود به بلاکچین اتریوم متکی است و از زبان برنامه‌نویسی Solidity برای توسعه قراردادهای هوشمند خود استفاده می‌کند. این ترکیب شفافیت، امنیت و تعامل‌پذیری² را در اکوسیستم غیرمتمرکز تضمین می‌کند. در بخش 3-3 و فصل دوم به طور مفصل در مورد زیرساخت بلاکچین مورد استفاده در این پیاده‌سازی، یعنی بلاکچین اتریوم، صحبت شده است، از این رو، در این بخش و در ادامه به جزئیات پیاده‌سازی قراردادهای هوشمند و ارتباط آن‌ها با نقش‌ها و بازیگران مختلف سامانه می‌پردازیم.

سامانه پیشنهادی بر پایه قراردادهای هوشمندی طراحی شده است که برای بلاکچین اتریوم نوشته شده‌اند. این قراردادهای هوشمند، قوانین و منطق حاکم بر جنبه‌های مختلف فرایند توکنیزه کردن و مدیریت دارایی‌های املاک و مستغلات را در بر می‌گیرند. در بخش 3-4-2، هر یک از این قراردادها به طور خلاصه معرفی شدند. در این بخش، ابتدا اطلاعات مربوط به توابع و رویدادهای هر یک از قراردادهای هوشمند اصلی و قراردادهای هوشمند مرتبط با آن را در قالب جداول جمع‌آوری می‌کنیم. سپس، نحوه پیاده‌سازی توابع اصلی را به طور جداگانه شرح خواهیم داد تا درک کاملی از نقش و عملکرد آن‌ها در فرایندهای

¹ BuildingManagerElection.sol

² Interoperability

مختلف داشته باشیم.

3-5-1 قرارداد هوشمند FractoRealNFT.sol یا FRN

قرارداد هوشمند FRN، اولین قرارداد هوشمندی است که توسط مالک ساختمان (BO) بر روی شبکه‌ی بلاکچین مستقر می‌شود. این قرارداد برای توکنیزه کردن واحدهای ساختمان از استاندارد ERC-721 تبعیت کرده و امکاناتی مانند توکنیزه کردن، انتقال و مدیریت اجاره‌ی واحدهای ساختمانی را فراهم می‌کند.

3-5-1-1 توابع

این بخش، توابع اصلی ارائه شده توسط FRN و قراردادهای هوشمند ارث‌بری شده توسط آن را نشان داده و اهداف و محدودیت‌های دسترسی هر تابع را شرح می‌دهد. شایان ذکر است که کدهای پیاده‌سازی شده مرتبط با قرارداد هوشمند FRN و قرارداد هوشمند RM به ترتیب در پیوست‌های الف و ب آورده شده است.

جدول 3-1 توابع قرارداد هوشمند FRN

نام	دسترسی ۱	توضیحات
setMeterages(tokenIds, metrages)	BO	متر از هر واحد بر حسب سانتی‌متر تنظیم می‌شود، این مقدار برای عمل تقسیم کردن واحد بسیار مهم است.
mint(to, tokenId) batchMint(to, tokenIds)	BO	یک یا چند توکن را برای یک TO ضرب می‌کند.
setPhaseTwoStartTime(timestamp) setPhaseOneStartTime(timestamp)	BO	مه‌های زمانی شروع گام‌های مختلف را تنظیم می‌کند.
phaseOneMint(signature, tokenId, price)	عمومی	این تابع توسط یک TO بالقوه صدا زده می‌شود، اگر شرایط خاص ذکر شده در 3-5-1-3-1 مهیا باشد، توکن مشخص شده برای او ضرب می‌گردد.
setErc1155Address(address)	BO	آدرس قرارداد FRF را تنظیم می‌کند.
startPhaseTwoMint()	عمومی	مرحله‌ی دوم فرایند فروش را آغاز می‌کند.
fractionize(from, tokenId)	صاحب توکن یا شخص دارای مجوز	یک توکن FRN را بر اساس متر از به توکن‌های FRF تقسیم می‌کند.
withdraw()	BO	امکان برداشت وجوه پرداخت شده توسط TOها را فراهم

1 در این جدول و جدول‌های دیگر این بخش، از نام خلاصه‌ی نقش‌های ذکر شده در 3-4-1 استفاده شده است.

می‌کند.		
آدرس پوشه‌ی IPFS که شامل اطلاعات هر واحد هست را تنظیم می‌کند.	BO	setBaseURI(baseUrl)

جدول 2-3 توابع اصلی قرارداد هوشمند استاندارد ERC-721 ارث‌بری شده توسط FRN [28]

نام	دسترسی	توضیحات
balanceOf(owner)	عمومی	تعداد توکن‌های موجود در حساب مالک را بر می‌گرداند.
ownerOf(tokenId)	عمومی	آدرس مالک یک توکن مشخص را بر می‌گرداند.
transferFrom(from, to, tokenId)	صاحب توکن یا شخص دارای مجوز	tokenId را از from به to منتقل می‌کند.
approve(to, tokenId)	صاحب توکن یا شخص دارای مجوز	به to برای انتقال و مدیریت tokenId مجوز اعطا می‌کند.
tokenURI(tokenId)	عمومی	آدرس IPFS که اطلاعات مرتبط با tokenId در آن ذخیره شده است را بر می‌گرداند.
totalSupply()	عمومی	تعداد کل توکن‌های ذخیره شده در قرارداد هوشمند را بر می‌گرداند.

جدول 3-3 توابع قرارداد هوشمند RentManagement.sol ارث‌بری شده توسط FRN

نام	دسترسی	توضیحات
payRent(tokenId)	شخص ساکن در واحد	برای پرداخت اجاره استفاده می‌گردد.
withdrawRent(tokenId)	صاحب توکن یا شخص دارای مجوز	تمام مبلغ اجاره مرتبط با tokenId را به حساب شخص فراخوان‌کننده‌ی تابع واریز می‌کند.
setRentFee(tokenId, rentAmount)	صاحب توکن یا شخص دارای مجوز	قیمت اجاره مرتبط با tokenId را تنظیم می‌کند.
setResident(tokenId, resident)	صاحب توکن یا شخص دارای مجوز	ساکن (مستأجر) را برای tokenId مشخص می‌کند.

2-1-5-3 رویدادها

این بخش به معرفی رویدادهای مهمی می‌پردازد که قراردادهای هوشمند مرتبط در طول اجرای توابع خود منتشر می‌کنند. این رویدادها اطلاعات مهمی را در مورد وضعیت و پیشرفت فرایند توکنیزه‌کردن ارائه

می‌کنند و به نهادهای خارجی اجازه می‌دهند تا بر اساس آن واکنش نشان دهند. توسعه‌دهندگان و DApp‌ها می‌توانند برای به‌روزرسانی‌های بلادرنگ و هماهنگ با قراردادهای هوشمند این سامانه، در محیط خارج از شبکه بلاکچین به این رویدادها گوش داده و منطق‌های خاص خود را بر اساس این رویدادها اعمال کنند.

جدول 3-4 رویدادهای قرارداد هوشمند FractoRealNFT.sol

نام	توضیحات
phaseOneStartTimeSet(startTime)	هنگامی که زمان شروع گام اول توسط BO تنظیم می‌شود، منتشر می‌شود.
phaseTwoStartTimeSet(startTime)	هنگامی که زمان شروع گام دوم توسط BO تنظیم می‌شود، منتشر می‌شود.
phaseTwoStarted()	هنگامی که گام دوم فرایند توکنیزه کردن آغاز می‌گردد، منتشر می‌شود.

جدول 3-5 رویدادهای اصلی قرارداد هوشمند استاندارد ERC-721 ارث‌بری شده توسط FRN [17]

نام	توضیحات
Transfer(from, to, tokenId)	هنگامی که توکن tokenId از from به to منتقل می‌شود، منتشر می‌گردد.
Approval(owner, approved, tokenId)	زمانی که مجوز مدیریت توکن tokenId توسط owner به approved اعطا می‌گردد، منتشر می‌شود.

جدول 3-6 رویدادهای RentManagement.sol ارث‌بری شده توسط FRN

نام	توضیحات
RentPaid(tokenId, payer, amount)	هنگامی که اجاره برای یک شناسه توکن خاص پرداخت می‌شود، منتشر می‌گردد.
RentWithdrawn(tokenId, to, amount)	هنگامی که مبلغ اجاره یک توکن توسط مالک یا شخص دارای مجوز برداشت می‌شود، منتشر می‌گردد.
RentFeeSet(tokenId, amount)	هنگامی که هزینه اجاره برای یک توکن خاص تنظیم می‌شود، منتشر می‌گردد.
ResidentSet(tokenId, resident)	هنگامی که یک ساکن برای یک شناسه توکن خاص مشخص می‌گردد، منتشر می‌شود.

3-1-5-3 پیاده‌سازی

قرارداد هوشمند FractoRealNFT.sol، از کتابخانه‌ی Openzeppelin برای استفاده از توابع استاندارد ERC-721 استفاده می‌کند و همچنین امکانات مرتبط با مدیریت اجاره را توسط قرارداد هوشمند

RentManagement.sol که توسط این پایان نامه پیشنهاد شده است به ارث می برد. در زیر توابع مهم این قرارداد شرح داده خواهند شد.

1-3-1-5-3 تابع phaseOneMint

تابع phaseOneMint از قرارداد هوشمند FRN، یک جز اصلی در فرایند توکنیزه کردن املاک و مستغلات می باشد که به خریداران بالقوه اجازه می دهد تا در مرحله ی اول فروش توکن ها، واحدها را به طور کامل خریداری کنند. این تابع، این امکان را به کاربر می دهد تا یک توکن NFT جدید را در زمان مناسب ضرب کند. این تابع نیاز به امضای معتبر از طرف BO دارد.

این تابع به سه پارامتر ورودی نیاز دارد: signature، tokenId و priceToPay. مورد اول یعنی امضا، یک پیام امضا شده از طرف مالک قرارداد یا همان BO است که در جلوتر توضیح داده می شود. tokenId یک شناسه ی منحصر به فرد برای توکن جدید است که برابر با شماره ی واحد ساختمان می باشد. و در نهایت priceToPay مقدار اتری است که از کاربر انتظار می رود برای ضرب کردن توکن پرداخت کند.

این تابع ابتدا بررسی می کند که آیا مهر زمانی فعلی در پنجره زمانی مشخص شده برای گام اول عرضه اولیه قرار می گیرد یا خیر، و از ضرب کردن توکن در زمان های دیگر جلوگیری می کند. سپس مقدار اتر ارسال شده را با مبلغ مورد انتظار مطابقت می دهد تا از پرداخت با مبلغ اشتباه جلوگیری گردد.

برای پیاده سازی فرایند اعتبارسنجی پارامترهای ارسال شده مانند قیمت و یا آدرس خریدار، دو راه وجود دارد. راه اول این است که قیمت تمامی واحدها و آدرس های خریداران بالقوه ی معتبر بر روی بلاکچین به صورت مجزا ذخیره گردد که این امر مشمول پرداخت هزینه ی گاز زیاد جهت ذخیره کردن اطلاعات می شود. برای صرفه جویی در مصرف گاز، این صحت سنجی را می توان با استفاده از رمزگذاری نامتقارن به صورت امضا کردن خارج از بلاکچین¹ با کلید خصوصی BO و تأیید این امضا بر روی بلاکچین² پیاده سازی کرد. پیش از این در 1-2-2 شرح داده شد که شبکه های بلاکچین از رمزگذاری نامتقارن برای اعتبارسنجی هویت کاربر و رمزگذاری و تأیید تراکنش ها استفاده می کنند. همچنین در 3-3-2 گفته شد که بلاکچین اتریوم نیز برای مدیریت و آدرس دهی حساب های EOA از رمزگذاری نامتقارن استفاده می کند. الگوریتم رمزگذاری نامتقارن استفاده شده در بلاکچین اتریوم ECDSA³ نام دارد. اتریوم با استفاده از این الگوریتم رمزگذاری اطمینان حاصل می کند که وجوه اتر فقط توسط صاحبان آنها خرج می شود. این الگوریتم همچنین روش ترجیحی برای ایجاد کلیدهای عمومی و خصوصی مربوط به تولید آدرس حساب (EOA) و تأیید تراکنش ها می باشد [10]. از آنجایی که تمامی کاربران بلاکچین اتریوم از جمله BO

¹ Off-chain Sign

² On-chain Verify

³ Elliptic Curve Digital Signature Algorithm

به صورت پیش فرض دارای کلیدهای خصوصی و عمومی (آدرس حساب) مرتبط با الگوریتم ECDSA هستند، این تابع از این الگوریتم نامتقارن برای صحت سنجی اطلاعات استفاده می کند. این تابع با استفاده از آدرس شخص ارسال کننده تراکنش، آدرس قرارداد هوشمند FRN، شناسه ی توکن و همچنین قیمت آن یک هش ایجاد می کند. سپس از روی این هش و با استفاده از مقدار امضای ورودی تابع، آدرس شخص امضاکننده ی پیام را به دست آورده و آن را با آدرس BO مطابقت می دهد. در طول این فرایند اگر کوچک ترین تغییری در اطلاعات مانند قیمت واحد یا آدرس شخص خریدار رخ دهد، این تطبیق صورت نگرفته و تراکنش با خطا مواجه شده و برگشت داده می شود. قابل توجه است که مقدار امضای ورودی تابع از روی خروجی یک فرایند هش مشابه خارج از شبکه ی بلاکچین توسط کلید خصوصی BO محاسبه می شود. در نهایت، اگر همه بررسی ها تأیید شوند، تابع mint داخلی که در کتابخانه ی openzeppelin وجود دارد فراخوانی می شود تا مالکیت توکن مشخص شده را به خریدار منتقل کرده و فرایند ضرب را تکمیل کند. تابع phaseOneMint مکانیزمی امن، قابل تأیید و کارآمد برای کاربران فراهم می کند تا در مرحله اولیه فروش توکن املاک شرکت کنند.

2-3-1-5-3 تابع startPhaseTwoMint

این تابع، مرحله مهم دیگر در فرایند توکنیزه کردن املاک و مستغلات می باشد که گام دوم فروش واحد، یعنی فروش جزئی را آغاز می کند. این تابع را فقط می توان بعد از رسیدن به زمان شروع تعیین شده برای گام دو فراخوانی کرد؛ چرا که این تابع ابتدا با چک کردن مهر زمانی از فراخوانی های پیش از موعد جلوگیری می کند. پس از تأیید شرایط آغازشدن گام دوم، این تابع تعداد توکن های ضرب نشده در گام اول را محاسبه کرده و سپس آن توکن ها را برای تبدیل شدن به توکن های ERC-1155 آماده سازی می کند. فرایند آماده سازی به این صورت است که ابتدا توکن هایی را که در مرحله اول ضرب نشده اند شناسایی می شوند و سپس هر توکن شناسایی شده را به صورتی که مالک آن آدرس قرارداد FRF باشد، به صورت NFT ضرب می کند. دلیل اینکه مالک توکن های NFT تقسیم شده، قرارداد هوشمند FRF است این است که در آینده بتوان از طریق DAO پیاده سازی شده در FRF، این توکن ها را به طور کامل به صورت اشتراکی مدیریت کرد. در نهایت، تابع mintBatch از قرارداد هوشمند FRF فراخوانی شده تا تمامی توکن های شناسایی شده در مرحله قبل بر اساس مترائز آن ها برای BO به صورت ERC-1155 ضرب گردند. مالکیت توکن های تقسیم شده در این گام ابتدا به BO واگذار می گردد تا بتواند آن ها را به فروش برساند. در انتها نیز رویداد متناسب برای اطلاع رسانی سرمایه گذاران بالقوه ی خرد منتشر می شود تا آن ها را از آغازشدن گام دوم مطلع کند.

3-3-1-5-3 تابع fractionize

تابع fractionize، مالکان واحدها را قادر می سازد تا مالکیت خود را به بخش های کوچک تر تقسیم کنند.

این تابع کار خود را با انتقال مالکیت توکن مشخص شده به قرارداد هوشمند FRF آغاز می‌کند. دلیل این عمل در توضیحات تابع قبلی که مرتبط با شروع گام دوم است بیان شده است. همچنین این انتقال تضمین می‌کند که تنها مالک قانونی یا یک طرف تأیید شده توسط مالک می‌تواند فرایند تقسیم‌بندی را آغاز کند و از اقدامات غیرمجاز محافظت می‌شود. پس از انتقال مالکیت، تابع mint از قرارداد هوشمند FRF فراخوانی می‌شود تا توکن NFT مشخص شده را بر اساس متراژ آن به توکن‌های ERC-1155 تقسیم کند. مالکیت توکن‌های جدید تقسیم شده، مالک اصلی توکن NFT تقسیم شده می‌باشد و آن مالک می‌تواند در قرارداد هوشمند FRF، مالکیت خود را به صورت اشتراکی و بر اساس مقدار دلخواه خود به سایرین واگذار کرده و یا به فروش بگذارد.

3-5-2 قرارداد هوشمند FractoRealFractions.sol یا FRF

این قرارداد هوشمند، روشی را برای مدیریت مالکیت جزئی توکن‌های ERC-721 (NFT) ارائه می‌دهد. در این قرارداد هوشمند، از استاندارد ERC-1155 برای نشان دادن کسری از توکن NFT استفاده می‌شود. این قرارداد هوشمند برای کار با یک قرارداد هوشمند خاص ERC-721 (FRN) طراحی شده است و همچنین از طریق ارث‌بردن قرارداد هوشمند FD، عملکردهای DAO را برای مدیریت اشتراکی پیاده‌سازی می‌کند. این قرارداد هوشمند پس از استقرار قرارداد هوشمند FRN، توسط BO بر روی شبکه مستقر می‌شود؛ چرا که برای استقرار این قرارداد به آدرس قرارداد هوشمند FRN نیاز است. در بخش‌های بعدی، عملکردها و قابلیت‌های کلیدی قرارداد هوشمند FRF را بررسی خواهیم کرد.

3-5-2-1 توابع

توابع قرارداد هوشمند FRF به شرح زیر می‌باشد. در بخش‌های بعدی جزئیات پیاده‌سازی توابع مهم به صورت مفصل آورده شده است. همچنین برای مشاهده‌ی کدهای مرتبط با پیاده‌سازی قرارداد هوشمند FRF و FD به پیوست‌های پ و ت مراجعه شود.

جدول 3-7 توابع قرارداد هوشمند FRF

نام	دسترسی	توضیحات
mint(to, id, amount) mint(to, ids, amounts)	FRN	این توابع توسط FRN فراخوانی شده و مقدار مشخص شده از توکن‌ها را برای FO ضرب می‌کنند.
rebuildNFT(tokenId)	FO	این تابع به صورت عکس تابع fractionize از FRN عمل کرده و به توکن‌ها اجازه می‌دهد تا در صورتی که کاملاً متعلق به یک FO باشند، در یک توکن NFT واحد جمع شوند.
splitRent(tokenId)	عمومی	اجاره انباشته شده برای شناسه توکن مشخص شده برداشته شده و بر اساس سهام

مالکیت مربوطه بین سهام‌داران توکن توزیع می‌شود.		
مبلغ تقسیم شده توسط تابع بالا را به حساب شخص فراخوان‌کننده‌ی تابع واریز می‌کند.	عمومی	withdrawRent(tokenId)
آدرس پوشه‌ی IPFS که شامل اطلاعات هر واحد هست را تنظیم می‌کند.	BO	setURI (newuri)

جدول 3-8 توابع اصلی قرارداد هوشمند استاندارد ERC-1155 ارث‌بری شده توسط FRF [29]

نام	دسترسی	توضیحات
balanceOf(account, id) balanceOfBatch(accounts, ids)	عمومی	تعداد توکن‌های مشخص‌شده‌ی موجود در حساب مالک را بر می‌گرداند.
safeTransferFrom(from, to, id, value, data) safeBatchTransferFrom(from, to, ids, values, data)	صاحب توکن یا شخص دارای مجوز	برای انتقال مقدار مشخص از یک یا چند توکن مشخص شده استفاده می‌شود.
setApprovalForAll(operator, approved)	صاحب توکن یا شخص دارای مجوز	به operator برای انتقال و مدیریت توکن‌های شخص صدا زنده‌ی تابع مجوز اعطا می‌کند.
isApprovedForAll(account, operator)	عمومی	مشخص می‌کند که آیا operator مجوز مدیریت توکن‌های account را دارد یا خیر.
uri(id)	عمومی	آدرس IPFS که اطلاعات مرتبط با توکن مشخص شده در آن ذخیره شده است را بر می‌گرداند.
totalSupply() totalSupply(id)	عمومی	تعداد کل توکن‌های موجود در قرارداد هوشمند یا تعداد توکن‌های توکن مشخص شده را بر می‌گرداند.
exists(id)	عمومی	مشخص می‌کند که شناسه‌ی توکن موردنظر در قرارداد هوشمند وجود دارد یا خیر.

جدول 3-9 توابع قرارداد هوشمند FractionsDAO.sol ارث‌بری شده توسط FRF

نام	دسترسی	توضیحات
submitProposal(tokenId, voteThreshold, targetAddress, data, description, voteEndTimestamp)	FO	پیشنهاد جدیدی را برای رأی‌گیری به DAO ارسال می‌کند.
castVote(proposalId, vote)	FO	به اعضای DAO اجازه می‌دهد تا رأی موافق یا مخالف خود را در مورد یک پیشنهاد خاص ثبت کنند.

پیشنهادی که رأی کافی برای اجرا آورده باشد را اجرا و اعمال می‌کند.	عمومی	executeProposal(proposalId)
بررسی می‌کند که آیا توکن در حال حاضر در DAO قفل شده است یا خیر. FRF تا زمانی که قفل آن باز نشود، از انتقال یا معامله آن جلوگیری می‌کند.	عمومی	isTokenLocked(tokenId)

3-5-2 رویدادها

رویدادهای منتشر شده در قراردادهای هوشمند FRF و قراردادهای هوشمند مرتبط با آن به صورت زیر است:

جدول 3-10 رویدادهای قرارداد هوشمند FractoRealFractions.sol

نام	توضیحات
RentSplited(tokenId, rentAmount)	نشان می‌دهد که پول اجاره بین سهام‌داران برای یک توکن خاص تقسیم شده است.
RentWithdrawn(tokenId, to, rentAmount)	هنگامی که یک FO سهم خود را از اجاره برداشت می‌کند منتشر می‌شود.
Received(from, amount)	هنگامی که FRF مقداری اتر دریافت می‌کند منتشر می‌شود.

جدول 3-11 رویدادهای اصلی قرارداد هوشمند استاندارد ERC-1155 ارث‌بری شده توسط FRF [30]

نام	توضیحات
TransferSingle(_operator, _from, _to, id, value) TransferBatch(_operator, _from, _to, _ids, -values)	هنگام انتقال یک یا چند توکن منتشر می‌شوند.
ApprovalForAll(_owner, _operator, _approved)	زمانی که مجوز operator برای مدیریت توکن‌های owner تغییر می‌کند منتشر می‌شود.
URI(_value, _id)	هنگامی که آدرس URI یک توکن مشخص تغییر می‌کند منتشر می‌شود.

جدول 3-12 رویدادهای FractionsDAO.sol ارث‌بری شده توسط FRF

نام	توضیحات
ProposalSubmitted(proposalId, tokenId, proposer, description)	نشان می‌دهد که یک پیشنهاد جدید در بلاکچین ثبت شده است.
Voted(proposalId, voter, vote)	هنگامی که یک FO به یک پیشنهاد رأی می‌دهد، منتشر می‌شود.
ProposalExecuted(proposalId)	هنگامی که یک پیشنهاد با موفقیت به طور کامل اجرا می‌شود

منتشر می شود.	
هنگامی که یک پیشنهاد حداقل رأی مثبت کافی برای اجرا را می آورد صدا زده می شود.	ProposalPassed(proposalId)
هنگامی که یک پیشنهاد به تعداد مشخص شده رأی منفی می آورد و رد شده محسوب می شود منتشر می شود.	ProposalRejected(proposalId)

3-2-5-3 پیاده سازی

قرارداد هوشمند FractoRealFractions.sol، از کتابخانه‌ی Openzeppelin برای استفاده از توابع استاندارد ERC-1155 استفاده می کند و همچنین امکانات مرتبط با مدیریت اشتراکی توسط DAO را توسط قرارداد هوشمند FractionsDAO.sol که توسط این پایان نامه پیاده سازی شده است به ارث می برد. در زیر توابع مهم این قرارداد شرح داده خواهد شد.

rebuildNFT تابع 1-3-2-5-3

تابع rebuildNFT در قرارداد FRF به عنوان مکانیزمی برای بازسازی یک NFT تکه تکه شده پیاده سازی شده است و این کار را با سوزاندن تمام توکن های ERC-1155 مرتبط با آن NFT و انتقال مالکیت آن NFT به شخص مورد نظر انجام می دهد. این تابع در واقع عکس تابع fractionize قرارداد هوشمند FRN که در بخش 3-3-1-5-3 توضیح داده شد عمل می کند.

هنگام فراخوانی، تابع ابتدا بررسی می کند که آیا توکن مورد نظر در قرارداد هوشمند FRF در حال حاضر وجود دارد یا خیر. اگر توکن مورد نظر وجود نداشته باشد، تابع با برگرداندن خطای مرتبط به اجرای خود خاتمه می دهد. متعاقباً، این تابع تأیید می کند که آیا شخص صدا زننده تابع همه ی توکن های FRF مرتبط با شناسه مشخص شده را در اختیار دارد یا خیر. اگر فراخوان کننده، مالک همه توکن های مشخص -ERC-1155 نباشد، تابع با خطا مواجه شده و از ادامه ی عملیات جلوگیری می شود. اگر هر دو شرط برآورده شوند، تابع کار خود را با سوزاندن¹ تمام توکن های FRF مرتبط با شناسه مشخص شده ادامه می دهد. پس از سوزاندن موفقیت آمیز توکن ها، مالکیت توکن ERC-721 مرتبط با شناسه مشخص شده از قرارداد هوشمند FRF به شخص فراخوان کننده تابع منتقل می شود و عملاً NFT در اختیار او قرار می گیرد.

در صورت اجرای موفق آمیز این تابع، رویداد TransferSingle از قرارداد هوشمند FRF که در اینجا بیانگر عملیات سوزاندن است و رویداد Transfer از قرارداد هوشمند FRN که مشخص کننده ی انتقال NFT مورد نظر است در شبکه منتشر می شود و سایرین را از این اتفاق آگاه می سازد.

¹ Burn

3-2-5-2 splitRent تابع

تابع splitRent قرارداد FRF توزیع عادلانه اجاره مرتبط با یک توکن خاص را در بین سهامدارانش امکان‌پذیر می‌کند. تابع تنها یک پارامتر به نام tokenId را می‌پذیرد که نشان‌دهنده شناسه منحصر به فرد توکنی است که اجاره‌ی آن قرار است تقسیم شود. این تابع با فراخوانی تابع برداشت اجاره از قرارداد هوشمند RM کار خود را آغاز می‌کند. هنگامی که قرارداد هوشمند FRF تابع برداشت اجاره از RM را فراخوانی می‌کند، تمامی مبلغ اجاره‌ی برداشت‌نشده‌ی واحد موردنظر از RM، به قرارداد هوشمند FRF که در واقع مالک NFT توکن موردنظر است واریز می‌شود. در هنگام عملیات واریز، رویدادهای RentWithdrawn از RM و Received از FRF در شبکه منتشر می‌گردند. پس از این، این تابع وارد یک حلقه شده و به‌ازای هر سهام‌دار توکن که در واقع یک FO محسوب می‌شود، مقدار اجاره را بر اساس میزان سهم او محاسبه کرده و به آن تخصیص می‌دهد تا بتواند بعداً آن را برداشت کند. تابع این محاسبات را با تقسیم سهم مالکیت سهام‌دار بر تعداد کل سهام واحد مشخص و ضرب حاصل در کل مبلغ اجاره انجام می‌دهد. اگر مجموع تمام سهام سهام‌داران که اجاره به آن‌ها تخصیص یافته است با تعداد کل سهام مرتبط با توکن FRF برابر نباشد، به این معنی است که در مرحله‌ی فروش جزئی آن واحد به طور کامل فروش نرفته و BO مالک آن بخش‌ها می‌باشد و در نتیجه مقداری اجاره تخصیص داده نشده باقی‌مانده است. در چنین مواردی، تابع، اجاره باقیمانده را در متغیری خاص به‌صورت مجزا به BO تخصیص می‌دهد که BO می‌تواند در آینده توسط تابع withdrawNonSharesRents که مختص به او است، این مقدار را برداشت کند. دلیل پیاده‌سازی سهم اجاره‌ی BO به روش ذکر شده، صرفاً جلوگیری از ذخیره‌ی اطلاعات تکراری در بلاکچین و صرفه‌جویی در مصرف گاز می‌باشد. پس از اتمام عملیات تقسیم اجاره، تابع رویداد RentSplited را منتشر می‌کند. این امر اجرای موفقیت‌آمیز فرایند توزیع اجاره را برای توکن مشخص شده نشان می‌دهد و FOها را مطلع می‌کند که می‌تواند اجاره‌ی خود را با صدازدن تابع withdrawRent که در بخش بعدی توضیح می‌دهیم برداشت کنند.

3-2-5-3 withdrawRent تابع

این تابع به FOها اجازه می‌دهد تا اجاره‌ی انباشته‌شده‌ی خود را برای یک توکن خاص برداشت کنند. هنگامی که تابع فراخوانی می‌شود، مبلغ اجاره‌ای که در تابع splitRent به FO اختصاص داده شده است بازیابی شده و به او انتقال پیدا می‌کند. این بازیابی با دسترسی به مقدار ذخیره شده برای دارنده توکن در نگاشت tokenIdShareHolders انجام می‌شود. متعاقباً، این تابع پیش از اینکه مقدار را برای FO واریز کند برای جلوگیری از حملات احتمالی ورود مجدد¹ که منجر به برداشت چندباره‌ی پول می‌شود، مانده

¹ Reentrancy Attack

اجاره را برای او صفر می‌کند. این تضمین می‌کند که مبلغ اجاره برداشت نشده تنها یک‌بار قابل برداشت است و از اقدامات غیرمجاز جلوگیری می‌کند. در نهایت پس از انتقال وجه به شخص صدا زننده‌ی تابع، برای اطمینان از شفافیت، این تابع رویداد RentWithdrawn را منتشر می‌کند و سایرین را از این برداشت مطلع می‌سازد.

submitProposal تابع 4-3-2-5-3

این تابع یکی از سه تابع اصلی فرایندهای مرتبط با DAO پیاده‌سازی شده در قرارداد هوشمند FractionsDAO می‌باشد. پیش از این که به توضیح این تابع بپردازیم، بهتر است که از ساختار هر پیشنهاد مطلع شویم. در معماری روش پیشنهادی، هر پیشنهاد در DAO ساختار ذکر شده در جدول 3-13 را شامل می‌شود.

جدول 3-13 پارامترهای هر پیشنهاد در FD

نام فیلد	توضیحات
id	شناسه منحصر به فرد هر پیشنهاد
tokenId	شناسه توکن FRF مرتبط (ERC-1155)
proposer	آدرس شخص ارائه‌دهنده‌ی پیشنهاد
voteThreshold	حداقل قدرت رأی لازم برای تصویب یا رد این پیشنهاد
voteEndTimestamp	مهر زمانی نشان‌دهنده پایان دوره رأی‌گیری
targetAddress	آدرس قرارداد هوشمندی که در هنگام اجرای موفق پیشنهاد فراخوانی می‌گردد.
data	هنگام اجرای موفق بر روی targetAddress فراخوانی می‌شود.
passed	مقدار بولی که نشان می‌دهد آیا پیشنهاد تصویب شده است یا خیر.
rejected	مقدار بولی که نشان می‌دهد آیا پیشنهاد رد شده است یا خیر.
executed	مقدار بولی که نشان می‌دهد آیا پیشنهاد اجرا شده است یا خیر.
votesFor	آرای موافق پیشنهاد
votesAgainst	آرای مخالف پیشنهاد
description	شرح پیشنهاد

قابل توجه است که بسته به نیاز می‌توان فیلدهای دلخواه را به این ساختار اضافه کرد. دو پارامتر اصلی هر پیشنهاد که در زمان اجرا شدن آن مورد استفاده است، پارامتر targetAddress و data می‌باشد. این دو

پارامتر ماهیت اصلی هر پیشنهاد را شامل می‌شوند. در واقع، فرایند رأی‌گیری هر پیشنهاد بر روی اجرا یا اجرائشدن data موردنظر بر روی قرارداد هوشمندی به آدرس targetAddress اتفاق می‌افتد. به طور مثال، برای مشخص کردن مستأجر یا قیمت اجاره‌ی یک واحد اشتراکی توسط DAO، باید پیشنهادی با مقدار data برابر اسم تابع و پارامترهای تابع موردنظر انتخاب مستأجر یا تعیین قیمت اجاره، و targetAddress قرارداد هوشمند FRN را ایجاد و برای رأی‌گیری در DAO ثبت کرد. در صورت رسیدن آرای موافق به مقدار voteThreshold، تابع موردنظر با پارامترهای ثبت شده بر روی قرارداد هوشمند هدف، توسط FRF فراخوانی خواهد شد.

در تابع submitProposal، هر FO می‌تواند برای یک توکن FRF یک پیشنهاد جدید را برای بررسی و رأی‌گیری توسط DAO در بلاکچین ثبت کند. هنگام فراخوانی، این تابع به برخی از پارامترهای ذکر شده در جدول 3-13 نیاز دارد. این پارامترها شامل شناسه‌ی توکن، آستانه‌ی رأی، آدرس قرارداد هوشمند و داده‌های موردنیاز برای اجرای پیشنهاد، شرح پیشنهاد و مهر زمانی پایان دوره‌ی رأی‌گیری پیشنهاد می‌شوند. برای اطمینان از اینکه فقط دارندگان توکن می‌توانند پیشنهاد ارسال کنند، ابتدا تابع تأیید می‌کند که آیا پیشنهاددهنده دارای توکن‌های مرتبط با شناسه توکن مشخص شده است یا خیر. اگر شرط مالکیت برآورده نشود، تابع با یک پیام خطای مناسب برمی‌گردد و از ثبت پیشنهاد جلوگیری می‌کند. پس از تأیید موفق، تابع یک شناسه پیشنهاد منحصر به فرد (proposalID) ایجاد می‌کند و برای آن شناسه، اطلاعات ورودی تابع را ثبت و در بلاکچین ذخیره می‌کند. سپس برای جلوگیری از انتقال توکن‌ها هنگام فرایند رأی‌گیری، تعداد پیشنهاد‌های فعال برای توکن موردنظر را در نگاشت مرتبط افزایش می‌دهد. در نهایت، برای اطلاع‌دادن به ذی‌نفعان و ایجاد شفافیت در مورد ارسال پیشنهادها، تابع رویداد ProposalSubmitted را منتشر می‌کند. این رویداد شامل جزئیات مربوطه مانند شناسه پیشنهاد، آدرس شخص پیشنهاددهنده، شناسه توکن مرتبط و توضیحات پیشنهاد می‌شود.

5-3-2-5-3 تابع castVote

فرایند اصلی رأی‌گیری یک پیشنهاد در DAO در این تابع صورت می‌گیرد. تابع ثبت رأی، به FOها اجازه می‌دهد تا در فرایند رأی‌گیری برای یک پیشنهاد خاص در DAO شرکت کنند. هنگام فراخوانی، این تابع به شناسه منحصر به فرد پیشنهاد و نوع رأی (موافق یا مخالف) نیاز دارد. برای اطمینان از درستی فرایند رأی‌گیری، این تابع چهار الزام و شرط زیر را بررسی و اعمال می‌کند:

1. پیشنهاد قبلاً نباید اجرا شده باشد.
2. کاربر نباید قبلاً به این پیشنهاد مشخص رأی داده باشد.
3. بازه‌ی زمانی رأی‌گیری باید مطابق با زمان کنونی باشد.
4. کاربر باید دارای موجودی کافی از توکن موردنظر مرتبط با پیشنهاد باشد.

اگر تمامی الزامات بالا برآورده شوند، تابع تعداد آرا را بر اساس ترجیح کاربر (موافق یا مخالف) و به تعداد موجودی آن کاربر از توکن مشخص شده مرتبط با پیشنهاد، به روز می کند. در این حین، رویداد Voted نیز در شبکه منتشر می شود و اطلاعاتی مرتبط با ثبت رأی کاربر بر روی نگاشت hasVoted در شبکه ثبت می شود تا از رأی گیری تکراری کاربر بر روی پیشنهاد مشخص در آینده جلوگیری گردد. در انتها پس از ثبت رأی، تابع ارزیابی می کند که آیا پیشنهاد از تعداد آرای کافی برای تصویب یا رد شدن بر اساس حدنصاب از پیش تعریف شده در هنگام ثبت پیشنهاد برخوردار است یا خیر. اگر پیشنهاد آرای مثبت کافی دریافت کند، به عنوان تصویب شده علامت گذاری و رویداد ProposalPassed در شبکه منتشر می شود. به طور عکس، اگر پیشنهاد به اندازه کافی رأی منفی دریافت کند، رد شده تلقی می شود و رویداد ProposalRejected منتشر می شود. در هر دو مورد، تعداد پیشنهادهای فعال مرتبط با توکن کاهش می یابد چرا که فرایند رأی گیری به پایان رسیده است.

6-3-2-5-3 تابع executeProposal

این تابع، آخرین تابع در فرایند ثبت و اجرای یک پیشنهاد در DAO است. تابع اجرای پیشنهاد، مسئول اجرای پیشنهادی است که فرایند رأی گیری در DAO را با موفقیت پشت سر گذاشته باشد. هنگام فراخوانی، تابع به شناسه منحصر به فرد پیشنهاد نیاز دارد. قبل از اجرای پیشنهاد، تابع الزامات و اعتبارسنجی های زیر را بر روی پیشنهاد مورد نظر اعمال می کند:

1. پیشنهاد نباید پیش از این اجرا شده باشد.
2. پیشنهاد نباید در فرایند رأی گیری رد شده تلقی شده باشد.
3. از همه مهم تر، در فرایند رأی گیری پیشنهاد باید به صورت تصویب شده علامت گذاری شده باشد. پس از برآورده شدن این شرایط، با فراخوانی قرارداد هدف (targetAddress) به همراه پارامترهای ثبت شده (data) در پیشنهاد، پیشنهاد را اجرا می کند. اگر اجرای پیشنهاد موفقیت آمیز باشد، تابع پیشنهاد را به صورت اجرا شده علامت گذاری کرده و رویداد ProposalExecuted را در شبکه منتشر می کند.

3-5-3 قرارداد هوشمند ChargeManagement.sol یا CM

آخرین قرارداد هوشمندی که در شبکه مستقر می شود، قرارداد هوشمند مدیریت هزینه های سرانه ی خدمات ساختمان می باشد. این قرارداد هوشمند به عنوان یک صندوق امن برای جمع آوری و مدیریت هزینه های ماهانه پرداخت شده توسط ساکنان و مالکان برای خدمات مختلف عمل می کند. مدیر ساختمان منصوب شده مسئولیت مدیریت و تخصیص این هزینه ها را بر عهده دارد. هدف اصلی این قرارداد ارائه یک رویکرد روشن و کارآمد برای نظارت بر استفاده از هزینه ها و حصول اطمینان از تخصیص مناسب آن ها برای حفظ و بهبود زیرساخت ها و خدمات ساختمان است. مدیر ساختمان از طریق یک فرایند رأی گیری دقیق مبتنی

بر بلاکچین توسط قرارداد هوشمند رأی‌گیری که توسط این قرارداد به ارث برده می‌شود انتخاب می‌گردد. مالکان واحدهای ساختمان (TO) می‌توانند در این قرارداد خود را به‌عنوان نامزد معرفی کرده و در فرایند رأی‌گیری که توسط تمامی ساکنان یا مالکان ساختمان صورت می‌گیرد رقابت می‌کنند.

3-5-1 توابع

توابع این قرارداد هوشمند به صورت زیر است. برای بررسی کدهای پیاده‌سازی شده مرتبط با قراردادهای هوشمند CM و BME به پیوست‌های 3 و 4 جیم مراجعه شود.

جدول 3-14 توابع قرارداد هوشمند CM

نام	دسترسی	توضیحات
setFeeAmount(newFeeAmount)	BM	مبلغ هزینه‌ی سرانه‌ی خدمات را تنظیم می‌کند.
payFee(tokenId)	TO یا UR	به ساکن یا مالک واحد اجازه می‌دهد تا هزینه ماهانه مربوط به خود را پرداخت کند.
spendFee(amount, to)	BM	به مدیر ساختمان اجازه می‌دهد تا مقدار amount را از موجودی CM به آدرس to انتقال دهد.

جدول 3-15 توابع قرارداد هوشمند BuildingManagerElection.sol ارث‌بری شده توسط CM

نام	دسترسی	توضیحات
startElection()	عمومی	فرایند انتخابات را با تغییر به وضعیت گام قبول نامزدها آغاز می‌کند.
registerCandidate()	TO	آدرس فراخوان‌کننده‌ی تابع را به‌عنوان نامزد در انتخابات مدیر ساختمان ثبت می‌کند.
startVoting(votingEndTime)	عمومی	پس از گام اعلام نامزدها، فرایند رأی‌گیری را آغاز می‌کند.
castVote(tokenId, candidate)	UR یا TO	به ساکنین یا مالکان واحدها اجازه می‌دهد تا رأی خود را به یک نامزد بدهند.
finalizeElection()	عمومی	پس از اتمام بازه رأی‌گیری، نامزدی که بیشترین آرا را کسب کرده به‌عنوان مدیر ساختمان منصوب می‌کند.

3-5-2 رویدادها

رویدادهای منتشر شده در قراردادهای هوشمند CM و BME به صورت زیر می‌باشد:

جدول 3-16 رویدادهای قرارداد هوشمند ChargeManagement.sol

نام	توضیحات
FeeAmountChanged(newFeeAmount)	نشان می‌دهد که مقدار هزینه سرانه‌ی خدمات تغییر کرده است.
FeePaid(tokenId, payer, amount)	بیانگر این است که هزینه سرانه‌ی خدمات واحد مربوطه پرداخت شده است.
FeeSpent(to, amount)	بیانگر این است که میزان amount از موجودی CM توسط BM به آدرس to واریز شده است.

جدول 3-17 رویدادهای قرارداد هوشمند BuildingManagerElection.sol ارث‌بری شده توسط CM

نام	توضیحات
StateChanged(newState)	نشان‌دهنده تغییر وضعیت در فرایند انتخابات است. فهرست وضعیت‌ها در جدول 3-18 قابل مشاهده می‌باشد.
CandidateRegistered(candidate)	نشان‌دهنده ثبت نام یک نامزد می‌باشد.
Voted(tokenId,voter,candidate)	هنگام ثبت رأی توسط ساکن یا مالک واحد مربوطه به نامزد موردنظر منتشر می‌شود.
BuildingManagerElected(newBuildingManager)	بیانگر پایان فرایند انتخابات و انتخاب مدیر ساختمان جدید می‌باشد.

3-3-5-3 پیاده‌سازی

پیاده‌سازی قرارداد هوشمند ChargeManagement.sol آخرین مرحله در ایجاد یک اکوسیستم مدیریت ساختمان است که به‌عنوان یک صندوق برای مدیریت هزینه یک ساختمان عمل می‌کند. این قرارداد هوشمند بر اساس قرارداد هوشمند BME بنا شده است و عملکردهای مرتبط با فرایند انتخاب مدیر ساختمان را از این قرارداد به ارث می‌برد. قرارداد از طریق توابعی که دارد عملکردهای حیاتی برای نظارت بر جمع‌آوری و تخصیص هزینه‌های سرانه‌ی خدمات پرداخت شده توسط ساکنان یا مالکان را پوشش می‌دهد. کارکردهای اصلی این قرارداد شامل پیاده‌سازی توابعی برای تنظیم، پرداخت و تخصیص هزینه‌های سرانه‌ی خدمات ساختمان می‌باشد. امکانات مرتبط با تنظیم و تخصیص صرفاً برای مدیر ساختمان محدود می‌شود. هنگام مستقر کردن این قرارداد هوشمند در شبکه، آدرس قرارداد هوشمند FRN نیز باید به‌عنوان یک پارامتر به شبکه ارسال گردد. با این ادغام، قرارداد هوشمند CM می‌تواند از اطلاعات مالکیت یا سکونت یک توکن FRN مطلع شود که این اطلاعات در فرایندهای مرتبط با انتخابات قرارداد هوشمند BME و پرداخت سرانه‌ی خدمات مورداستفاده قرار می‌گیرند.

3-3-5-3 تابع setFeeAmount

تابع setFeeAmount از قرارداد هوشمند CM به مدیر ساختمان این امکان را می‌دهد که مقدار هزینه سرانه‌ی خدمات یک ساختمان را با ارائه یک پارامتر که شامل مقدار جدید است تغییر دهد. این تابع تضمین می‌کند که آدرس فراخوان کننده‌ی تابع فقط برابر با آدرس مدیر ساختمان تعیین شده توسط قرارداد هوشمند BME باشد. در نهایت پس از تغییر مقدار، رویداد FeeAmountChanged منتشر می‌شود تا ذی‌نفعان را از این تغییر مطلع کند. علاوه بر آن، شایان ذکر است که درحالی که پیاده‌سازی فعلی به مدیر ساختمان اجازه می‌دهد تا آزادانه میزان هزینه را تنظیم کند، شرایط محدودکننده‌تری را نیز می‌توان در این تابع اعمال کرد. برای مثال، شروطی را می‌توان برای محدود کردن تغییر میزان هزینه‌ی سرانه‌ی خدمات به فقط یکبار در هر شش ماه یا یک سال پیاده‌سازی کرد. این ملاحظات، انعطاف‌پذیری این قراردادهای هوشمند را نشان می‌دهد و امکان پیشرفت‌های آتی در پیاده‌سازی نهایی را متناسب با الزامات خاص حاکمیتی و ترجیحات جامعه فراهم می‌کند.

3-3-5-3 تابع payFee

این تابع ساکن یا مالک یک واحد را قادر می‌سازد تا برای یک توکن خاص مرتبط با خود، هزینه‌های سرانه‌ی خدمات را به قرارداد CM واریز کنند. این تابع شناسه‌ی یک توکن FRN را به عنوان پارامتر ورودی می‌گیرد و مطمئن می‌شود که فقط ساکن یا مالک آن واحد می‌تواند این تابع را فراخوانی کند. سپس بررسی می‌کند که مقدار اتر ارسال شده با مقدار هزینه‌ی سرانه‌ی خدمات تنظیم شده در تابع قبلی مطابقت دارد یا خیر. اگر این مقدار مطابقت نداشته باشد، تراکنش با یک خطای مشخص برگردانده می‌شود. در غیر این صورت، مقدار هزینه‌ی پرداخت شده به مقدار مرتبط آن واحد که در نگاشت مشخص ذخیره شده است اضافه می‌گردد. از این نگاشت می‌توان متوجه شد که هر واحد چه میزان مبلغ سرانه‌ی خدمات را پرداخت کرده است. در نهایت رویداد FeePaid که شامل شناسه‌ی واحد، آدرس شخص پرداخت کننده و هزینه‌ی سرانه‌ی خدمات است در شبکه منتشر می‌شود.

3-3-5-3 تابع spendFee

این تابع، آخرین تابع مرتبط با مدیریت هزینه‌های سرانه‌ی خدمات می‌باشد و باهدف انتقال مقدار مشخصی از اتر از موجودی قرارداد هوشمند CM به یک آدرس مشخص پیاده‌سازی شده است. این تابع به دو پارامتر نیاز دارد. این پارامترها شامل مقدار اتر موردنظر برای انتقال و آدرس گیرنده می‌باشد. قبل از انتقال، این تابع تضمین می‌کند که فقط مدیر ساختمان می‌تواند این تابع را فراخوانی کند و همچنین پس از انتقال موفق، رویداد FeeSpend که شامل آدرس گیرنده و مبلغ منتقل شده است برای اطلاع‌رسانی سایرین منتشر می‌شود.

4-3-3-5-3 registerCandidate تابع

تابع ثبت نام نامزد، بخشی از قرارداد هوشمند BME برای انتخابات مدیر ساختمان است. این تابع به یک نامزد اجازه می دهد تا برای انتخابات ثبت نام کند، اما تنها در صورتی که وضعیت انتخابات در حالت پذیرش نامزد تنظیم شده و همچنین شخص فراخوان کننده ی تابع یک TO، یعنی صاحب یک توکن FRN باشد. فهرست وضعیت های مختلف فرایند رأی گیری در جدول 3-18 قابل مشاهده است. اگر این شرایط برآورده نشود، تابع با یک خطای مرتبط برگردانده می شود. علاوه بر آن، اگر نامزد قبلاً و در طی فرایند انتخابات کنونی ثبت نام کرده باشد، تابع با خطا مواجه می گردد. اگر بررسی تمام شرایط موفقیت آمیز باشد، آدرس نامزد ذخیره شده و به عنوان ثبت نام شده به فهرست نامزدها اضافه می شود و در نهایت یک رویداد CandidateRegistered که شامل آدرس این نامزد است منتشر می گردد.

جدول 3-18 وضعیت های مختلف در فرایند انتخابات مدیر ساختمان

وضعیت	توضیحات
NotStarted	انتخابات هنوز شروع نشده است.
acceptCandidates	نامزدها می توانند در انتخابات ثبت نام کنند.
Voting	فرایند رأی گیری در حال انجام است.

5-3-3-5-3 castVote تابع

تابع ثبت رأی به ساکنین یا مالکان واحدها اجازه می دهد تا در طول فرایند انتخابات مدیر ساختمان، به یک نامزد رأی دهند. این تابع به دو پارامتر شناسه ی توکن و آدرس نامزد مورد نظر برای رأی نیاز دارد. شناسه ی توکن، شناسه ی منحصر به فرد یک توکن FRN و نمایانگر واحد مربوط به ساکن یا مالک شخص رأی دهنده می باشد. این تابع ابتدا تأیید می کند که فقط ساکنان یا صاحبان واحد می توانند در فرایند رأی گیری شرکت کنند و سپس وضعیت فعلی انتخابات را مطابق با جدول 3-18 بررسی می کند. اگر انتخابات در حالت «رأی گیری» نباشد، تراکنش را با خطای مرتبط بر می گرداند. همچنین با مقایسه ی مهر زمانی بلوک فعلی بلاکچین با متغیر اتمام زمان انتخابات مطمئن می شود که بازه ی زمانی فرایند رأی گیری به پایان نرسیده باشد. سپس، با جستجو در نگاشت مربوط به رأی دهندگان یک توکن در فرایند کنونی انتخابات بررسی می کند که آیا شخص فراخوان کننده ی تابع قبلاً رأی داده است یا خیر؛ تا در صورت رأی تکراری تراکنش با خطای مربوطه برگردانده شود. به عنوان آخرین پارامتر بررسی، تابع از طریق نگاشت مرتبط با نامزدهای ثبت نام شده در فرایند انتخابات کنونی، تأیید می کند که آدرس مورد رأی به عنوان نامزد در سامانه ثبت شده باشد، در غیر این صورت تراکنش با خطا مواجه می گردد. اگر همه ی شرایط مناسب باشد، تابع تعداد آرا را برای نامزد مربوطه افزایش می دهد و برای جلوگیری از رأی تکراری، شناسه توکن و آدرس رأی دهنده

را در نگاشت رأی‌دهندگان ثبت می‌کند. در نهایت یک رویداد Voted که شامل شناسه‌ی توکن شخص رأی‌دهنده، آدرس رأی‌دهنده و آدرس نامزد است منتشر می‌گردد.

6-3-3-5-3 finalizeElection تابع

تابع نهایی کردن انتخابات، آخرین مرحله از فرایند انتخابات مدیر ساختمان می‌باشد. این تابع طوری طراحی شده است که پس از پایان دوره رأی‌گیری فراخوانی شود و هدف آن نهایی کردن انتخابات با تعیین نامزد برنده و منصوب کردن آن به عنوان BM و تنظیم مجدد پارامترهای انتخابات کنونی است. تابع با بررسی وضعیت و بازه‌ی زمانی مشخص شده برای انتخابات کار خود را آغاز می‌کند. اگر وضعیت در حالت «رأی‌گیری» نباشد یا بازه‌ی انتخابات به اتمام نرسیده باشد تراکنش با خطای مرتبط برگردانده می‌شود. با برآورده شدن شروط، تابع برای تعیین نامزد برنده به کار خود ادامه می‌دهد و این کار را با پیمایش فهرست نامزدهای ثبت‌نام‌شده در طی انتخابات کنونی و یافتن شخصی که بیشترین رأی را دارد انجام می‌دهد. پس از تعیین نامزد برنده، تابع با فراخوانی تابعی مشخص در قرارداد هوشمند CM، آدرس نامزد برنده را به عنوان مدیر ساختمان تنظیم می‌کند. سپس برای آماده‌سازی قرارداد هوشمند BME برای انتخابات آینده، تابع متغیرهای مرتبط با فرایند انتخابات را بازنشانی می‌کند. این متغیرها شامل وضعیت انتخابات، فهرست نامزدها، نگاشت نامزدها به تعداد آرای هر نامزد، نگاشت رأی‌دهندگان و شناسه‌ی توکن مرتبط با هر رأی‌دهنده و همچنین مدت‌زمان بازه‌ی رأی‌گیری می‌باشد. در نهایت، این تابع با انتشار رویدادهای StateChanged و BuildingManagerElected که شامل آدرس مدیر ساختمان جدید است کار خود را به اتمام رسانده و فرایند انتخابات مدیر ساختمان را تکمیل می‌کند.

6-3 گردش کار و تعاملات سامانه

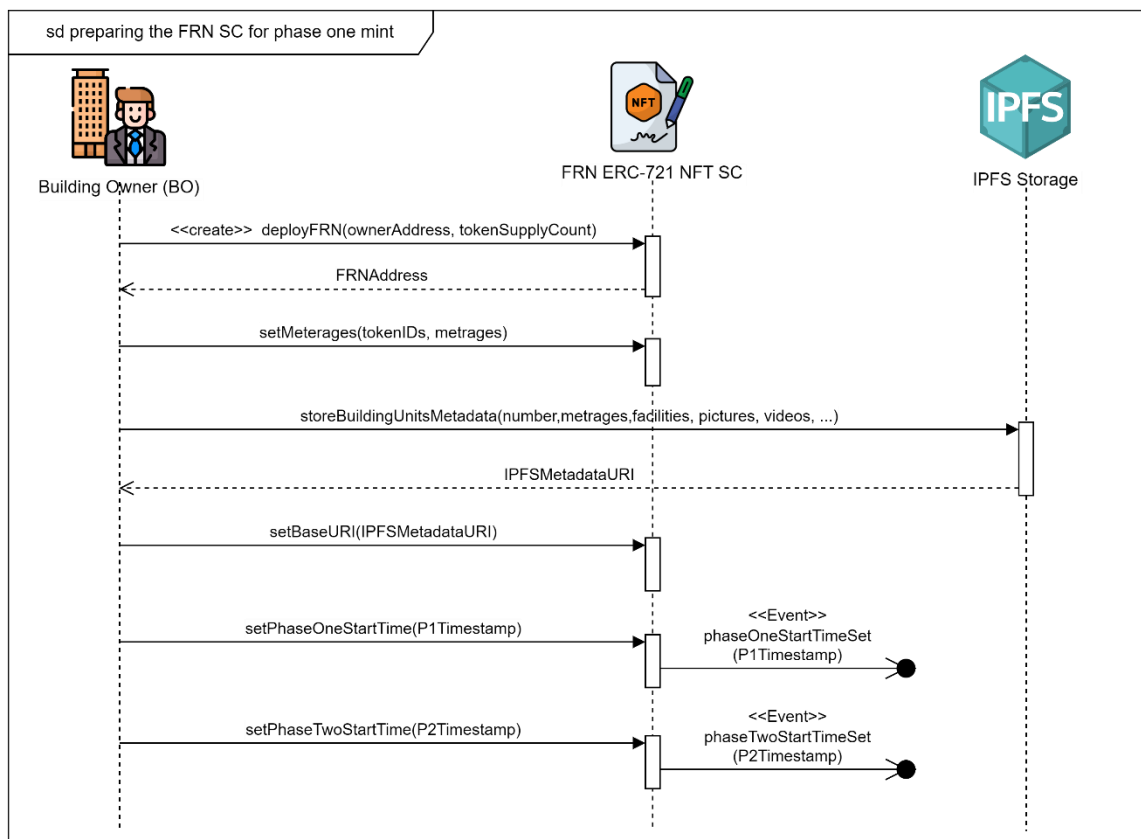
در این بخش، نمودارهای توالی تعاملات مهم بین بازیگران و قراردادهای هوشمند مختلف سامانه ارائه می‌شود. این نمودارها یک نمایش بصری از جریان پیام‌ها و تعاملات مختلف بین اجزای سامانه را در طول زمان ارائه می‌دهند. نمودارهای ارائه شده در این بخش بر تعاملات کلیدی سامانه پیشنهادی، از جمله روند توکنیزه کردن یک ساختمان، عملیات DAO، مدیریت اجاره و هزینه‌های سرانه‌ی خدمات و همچنین فرایند انتخابات مدیر ساختمان متمرکز خواهد بود.

3-6-1 توکنیزه کردن

توکنیزه کردن مهم‌ترین جنبه‌ی سامانه پیشنهادی است که به ما امکان نمایش دارایی‌های دنیای واقعی را به عنوان توکن‌های دیجیتال در بلاکچین می‌دهد. این بخش نمودارهای توالی مرتبط با روند ثبت یک ساختمان توسط BO در بلاکچین و همچنین دو گام اول و دوم فروش را ارائه می‌دهد.

شکل 3-3 نمودار توالی اولین فرایند مرتبط با سامانه پیشنهادی را شرح می‌دهد. این نمودار مراحل استقرار

و آماده‌سازی قرارداد هوشمند FRN برای گام اول فروش را تشریح می‌کند که شامل تعاملات بین مالک ساختمان (BO)، قرارداد هوشمند FRN و همچنین فضای ذخیره‌سازی IPFS می‌شود.

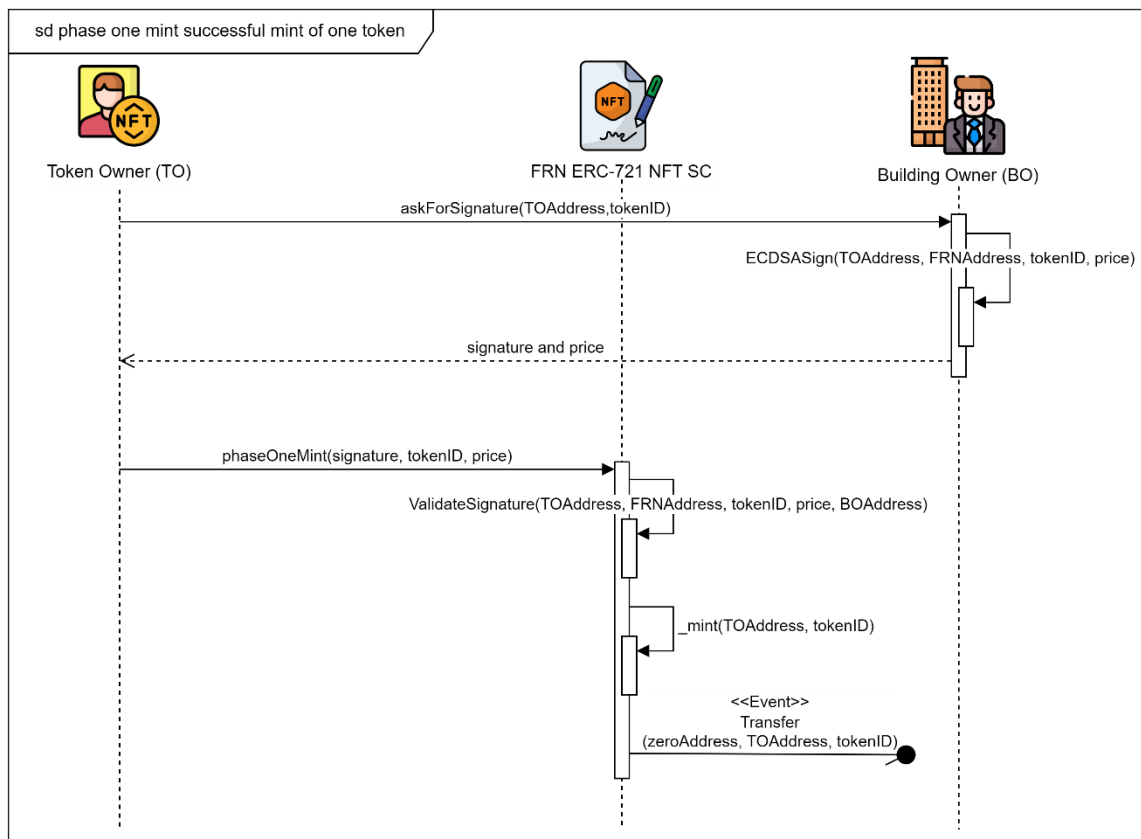


شکل 3-3 نمودار توالی آماده‌سازی قرارداد هوشمند FRN

مالک ساختمان قرارداد هوشمند FRN را با پارامترهای داده شده، یعنی آدرس خود و تعداد کل واحدهای ساختمان (تعداد توکن‌ها)، در بلاکچین مستقر می‌کند و آدرس قرارداد هوشمند FRN در شبکه‌ی اتریوم را دریافت می‌کند. سپس، BO چندین تابع را به ترتیب باهدف راه‌اندازی FRN برای گام اول فروش فراخوانی می‌کند. اولین تابع `setMeterages` است که متراژ هر واحد از ساختمان را به صورت سانتی‌متر در بلاکچین ذخیره می‌کند. سپس او فراداده‌های هر واحد ساختمان را در یک پوشه آماده کرده و آن پوشه را در IPFS ذخیره و آدرس پوشه را دریافت می‌کند. این فراداده‌ها شامل اطلاعاتی مانند تعداد خواب، متراژ، امکانات، تصاویر و ویدئوهایی از هر واحد است که بستگی به منطق تجارت این اطلاعات می‌تواند کم یا زیاد شود. سپس توسط فراخوانی تابع `setBaseURI`، آدرسی که از IPFS دریافت کرده است را در بلاکچین ذخیره می‌کند. در آینده از این آدرس برای دسترسی به اطلاعات فراداده‌ی هر توکن استفاده می‌شود. در نهایت با فراخوانی دو تابع ذخیره‌سازی زمان‌های گام اول و دوم فروش، زمان‌های مرتبط را در بلاکچین ذخیره کرده و قرارداد هوشمند را برای شروع گام اول فروش آماده می‌کند.

پس از آماده‌سازی قرارداد هوشمند توسط BO، یک TO بالقوه می‌تواند در گام اول فروش، توکن دلخواه

خود را ضرب کند. نمودار توالی این روند در شکل 4-3 آورده شده است.

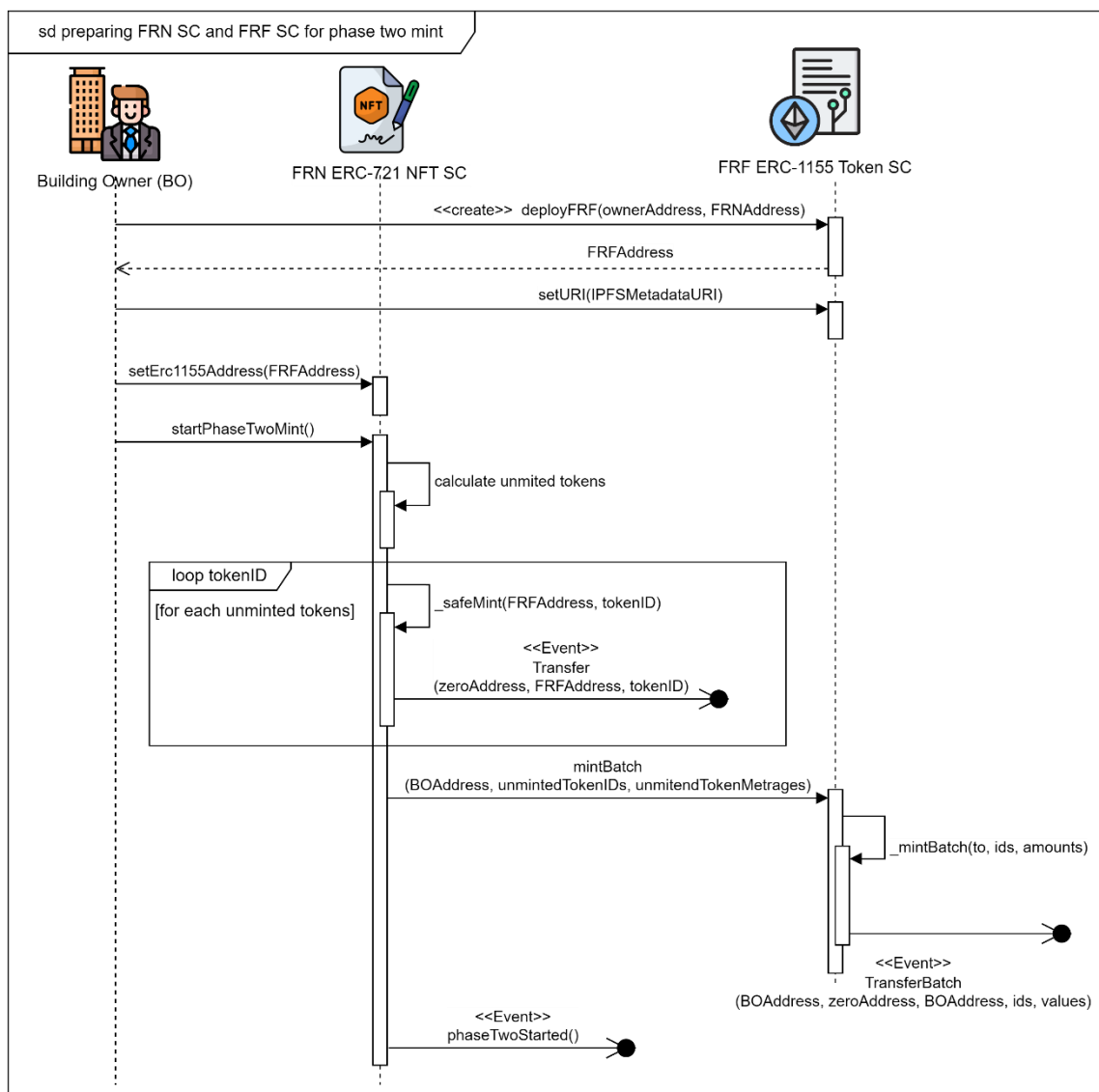


شکل 4-3 نمودار توالی فروش موفق یک توکن در گام اول

پیش از فراخوانی تابع مربوط به خرید توکن در گام اول، TO نیاز به یک تأییدیه در قالب امضا از سمت BO دارد. هنگام دریافت درخواست امضا، BO می‌تواند شروطی مبتنی بر اینکه آیا TO در سامانه شناسایی و احراز هویت شده است را بررسی کند. این شروط می‌تواند بر اساس منطق تجارت متغیر باشد. پس از تأیید، BO متنی مرتبط با درخواست خرید را امضا کرده و آن را به همراه قیمت توکن موردنظر به TO می‌گرداند. پیاده‌سازی این امضا در 1-3-1-5-3 شرح داده شده است. روند ارسال درخواست امضا، امضا و دریافت آن را می‌توان در سمت رابط کاربری و سرور DApp موردنظر پیاده‌سازی کرد. در نهایت، TO تابع مربوط به ضرب توکن در گام اول را با پارامترهای موردنظر فراخوانی می‌کند. سپس قرارداد هوشمند FRN بر اساس پارامترهای ورودی و آدرس BO، امضا را اعتبارسنجی می‌کند. در صورت تأیید، تابع `_mint` داخلی را فراخوانی کرده و توکن موردنظر را برای TO ضرب می‌کند. در انتها نیز رویداد انتقال از آدرس صفر (بیانگر ضرب شدن اولیه) به آدرس TO در شبکه منتشر می‌شود تا موفقیت‌آمیز بودن ضرب کردن توکن موردنظر را نشان دهد.

همانند آماده‌سازی قرارداد هوشمند FRN برای فروش توکن در گام اول، گام دوم نیز نیاز به آماده‌سازی دارد. آماده‌سازی قراردادهای هوشمند FRN و FRF و آغاز گام دوم فروش در شکل 5-3 شرح داده شده

است.

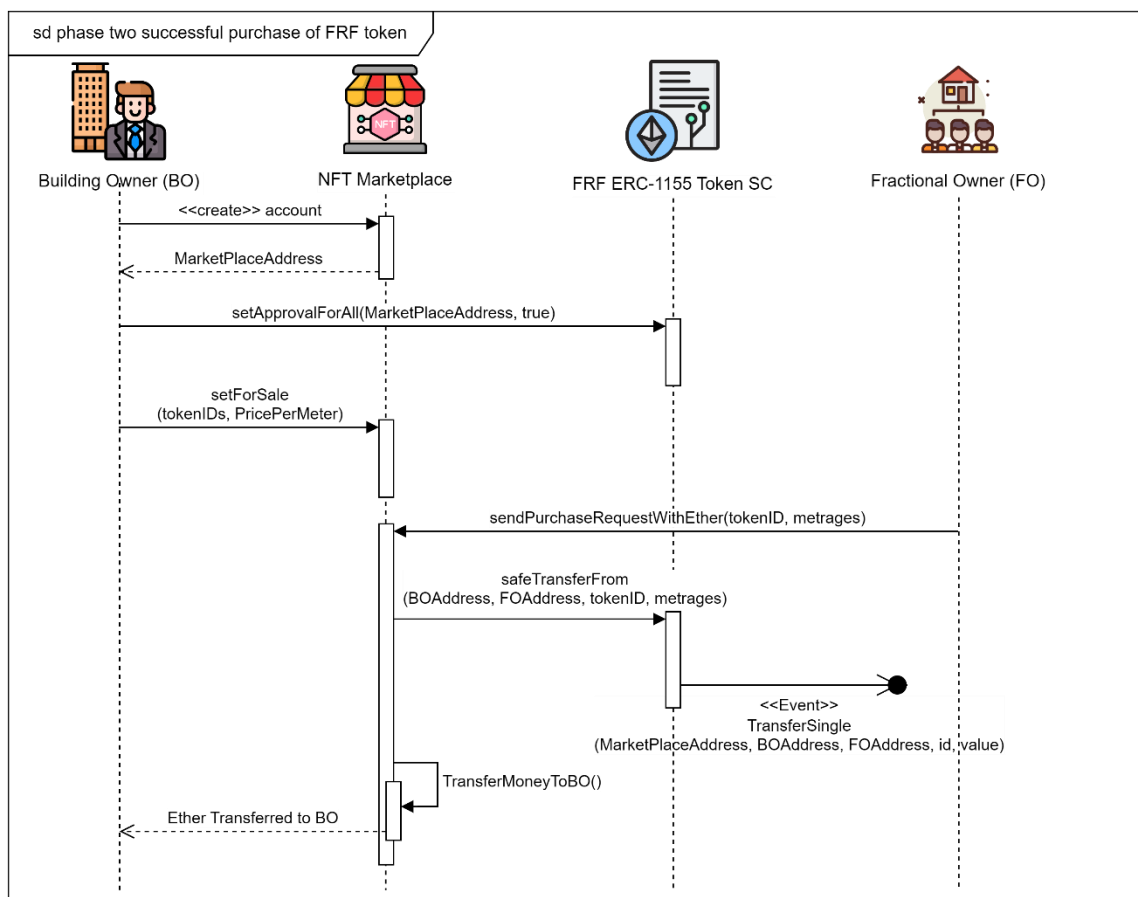


شکل 3-5 نمودار توالی آماده‌سازی قراردادهای هوشمند جهت گام دوم فروش

مالک ساختمان (BO) ابتدا قرارداد هوشمند FRF را با ورودی آدرس قرارداد هوشمند FRN در بلاکچین مستقر می‌کند. سپس همانند FRN، با فراخوانی تابع موردنظر به همراه آدرس IPFS که در مرحله‌ی آماده‌سازی قبلی به دست آورده، آدرس فراداده‌های مرتبط با توکن‌ها را در قرارداد هوشمند FRF تنظیم می‌کند. پس از استقرار و آماده‌سازی FRF، جهت پردازش‌های بعدی نیاز است که آدرس قرارداد هوشمند تازه مستقرشده‌ی FRF را در FRN ذخیره کند. مالک ساختمان این کار را با فراخوانی تابع تنظیم آدرس قرارداد هوشمند FRF انجام می‌دهد. در نهایت، BO با فراخوانی تابع مربوطه از قرارداد هوشمند FRN، گام دوم فروش را آغاز می‌کند. هنگام اجرای این تابع، FRN تعداد کل توکن‌های ضرب نشده را محاسبه کرده و آن‌ها را برای FRF ضرب می‌کند. سپس شناسه و متراژ آن توکن‌ها را برای ضرب شدن به صورت جزئی به قرارداد هوشمند FRF ارسال می‌کند. این روند با منتشر شدن رویدادهایی مبتنی بر ضرب شدن

توکن‌های جدید FRF و آغاز گام دوم فروش به پایان می‌رسد.

از آنجایی که در مرحله‌ی قبل، یعنی فرایند آغاز گام دوم فروش، تمامی توکن‌های FRF باقیمانده برای BO ضرب شده‌اند، برای فروش آن‌ها به یک بازار ثانویه نیاز داریم. شکل 3-6 نمودار توالی روند خرید و فروش توکن‌های FRF را در این بازار ثانویه نشان می‌دهد.



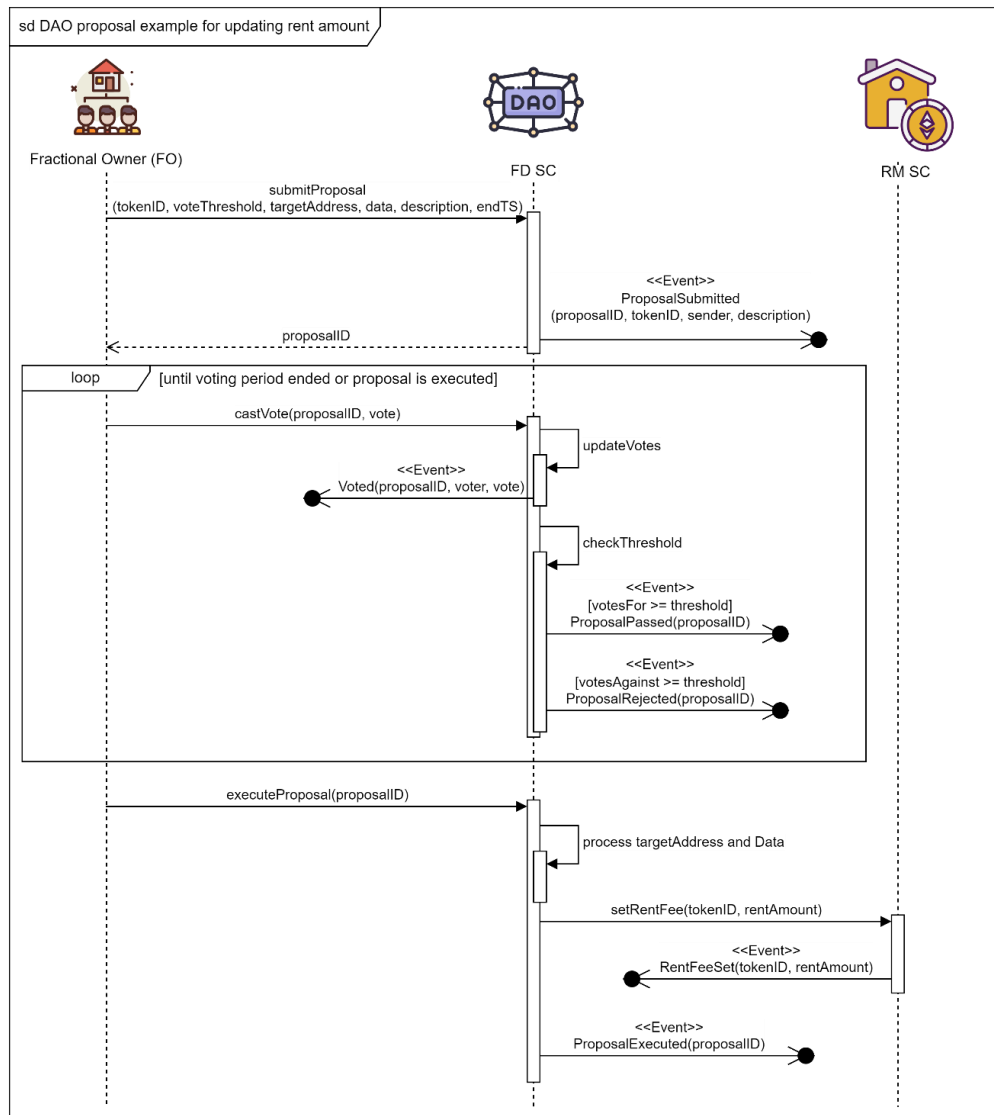
شکل 3-6 نمودار توالی خرید موفق یک توکن جزئی FRF

BO ابتدا در بازار ثانویه یک حساب کاربری ایجاد کرده و به آدرس قرارداد هوشمند بازار اجازه‌ی انتقال توکن‌های خود را می‌دهد. سپس BO توکن‌های FRF خود را به همراه قیمت برای فروش در بازار ثبت می‌کند. حال یک FO بالقوه می‌تواند جهت خرید یک توکن، مقدار موردنظر خود را به همراه قیمت ثبت شده به بازار ارسال کند. در این مرحله بازار به صورت هم‌زمان در یک تراکنش توکن موردنظر را در FRF به FO انتقال داده و مبلغ ارسال شده را به BO واریز می‌کند. در نهایت رویدادهای مبتنی بر انتقال توکن موردنظر در شبکه منتشر می‌شود.

2-6-3 عملیات DAO

پیاده‌سازی DAO برای اداره‌ی سامانه پیشنهادی جهت مدیریت واحدهای اشتراکی و تسهیل تصمیم‌گیری جمعی در مورد آن واحدها ضروری است. این بخش نمودارهای توالی دو عملیات کلیدی مرتبط با مدیریت

واحدهای اشتراکی، یعنی ارسال و اجرای یک نمونه پیشنهاد در DAO و همچنین برداشت و تقسیم اجاره مرتبط با یک واحد را نشان می‌دهد. شکل 3-7 نمودار توالی ارسال و اجرای یک پیشنهاد در DAO را شرح می‌دهد که در اینجا به صورت نمونه، پیشنهاد به روزرسانی قیمت اجاره‌ی یک واحد مطرح شده است.



شکل 3-7 نمونه فرایند ارسال و اجرای یک پیشنهاد در DAO

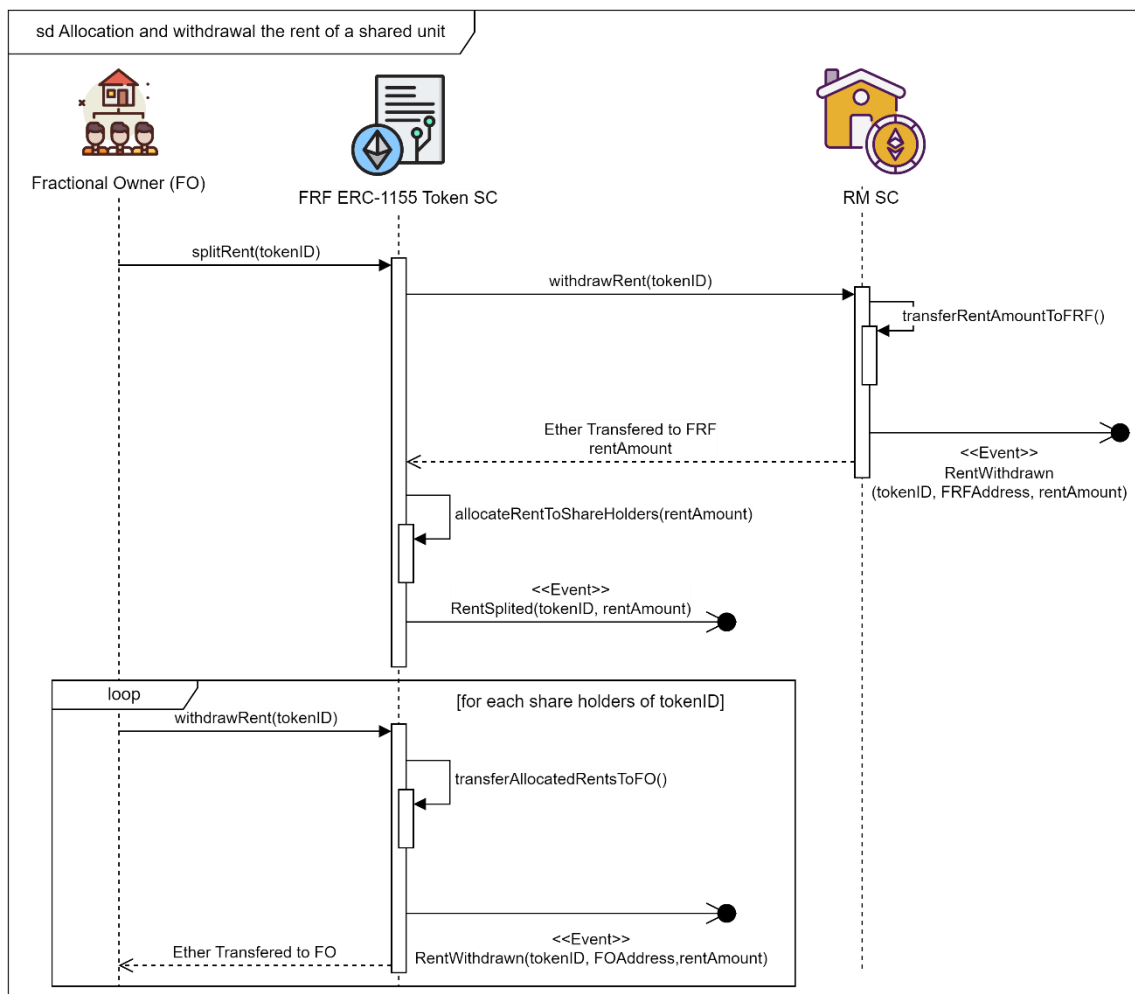
این نمودار توالی، تعاملات بین مالکان اشتراکی و قراردادهای هوشمند DAO و RM در طول ارائه و اجرای یک پیشنهاد به DAO را به طور زیر نشان می‌دهد:

1. شروع: فرایند با ارسال پیشنهاد از سمت یک FO به عنوان پیشنهاددهنده از طریق فراخوانی تابع مربوطه در FD به همراه پارامترهای موردنظر آغاز می‌شود. هنگام فراخوانی، آدرس قرارداد هوشمند RM به عنوان آدرس هدف و همچنین نوع تابع هدف به همراه پارامترهای ارسالی (تابع `setRentFee` و پارامترهایش یعنی `tokenID` و `rentAmount`) به صورت پارامتر `data` ارسال می‌گردد.

2. **دوره رأی گیری:** پس از ثبت پیشنهاد در DAO، FD با ارسال یک رویداد به تمامی بازیگران سامانه اطلاع رسانی می کند. در اینجا تمامی FOهای مرتبط با tokenID آن پیشنهاد می توانند در این رأی گیری شرکت و رأی موافق یا مخالف خود را در FD ثبت کنند. در همین حین اگر تعداد رأی موافق یا مخالف یک پیشنهاد از آستانه ی مورد نظر بگذرد، آن پیشنهاد به صورت تأیید و یا رد شده علامت گذاری می گردد و رویداد مرتبط با آن در شبکه منتشر می شود.

3. **اجرا:** در صورت تأیید پیشنهاد، با فراخوانی تابع مربوطه می توان پیشنهاد را اجرا کرد. در این مرحله DAO طبق آدرس و اطلاعات هدف پیشنهاد را اجرا می کند که در این نمونه، تابع به روزرسانی قیمت اجاره با شماره ی واحد و قیمت مورد نظر از طرف DAO فراخوانی می گردد. در نهایت پیشنهاد به صورت اجرا شده علامت گذاری گردیده و رویداد مرتبط منتشر می شود.

شکل 3-8 نیز نمودار توالی روند تخصیص و برداشت اجاره برای یک واحد اشتراکی را نشان می دهد. این فرایند توسط یک مالک جزئی (FO) با فراخوانی تابع مربوطه در FRF آغاز می شود. سپس قرارداد هوشمند FRF، تابع برداشت اجاره از قرارداد هوشمند RM را فراخوانی کرده تا تمامی مبلغ های اجاره برداشت نشده مرتبط با واحد مربوطه از RM به قرارداد هوشمند FRF واریز گردد. پس از واریز، مبالغ برداشت شده بین FOهای مختلف که سهام دار آن واحد هستند منطبق با میزان سهم آنها تخصیص می یابد. سپس هر FO که مبلغی به آن تخصیص داده شده، می تواند با فراخوانی تابع برداشت، آن مبلغ را از FRF برداشت و به آدرس خود منتقل کند. در حین این مراحل، رویدادهایی مرتبط برای اطلاع رسانی تمامی ذی نفعان سامانه منتشر می شود.



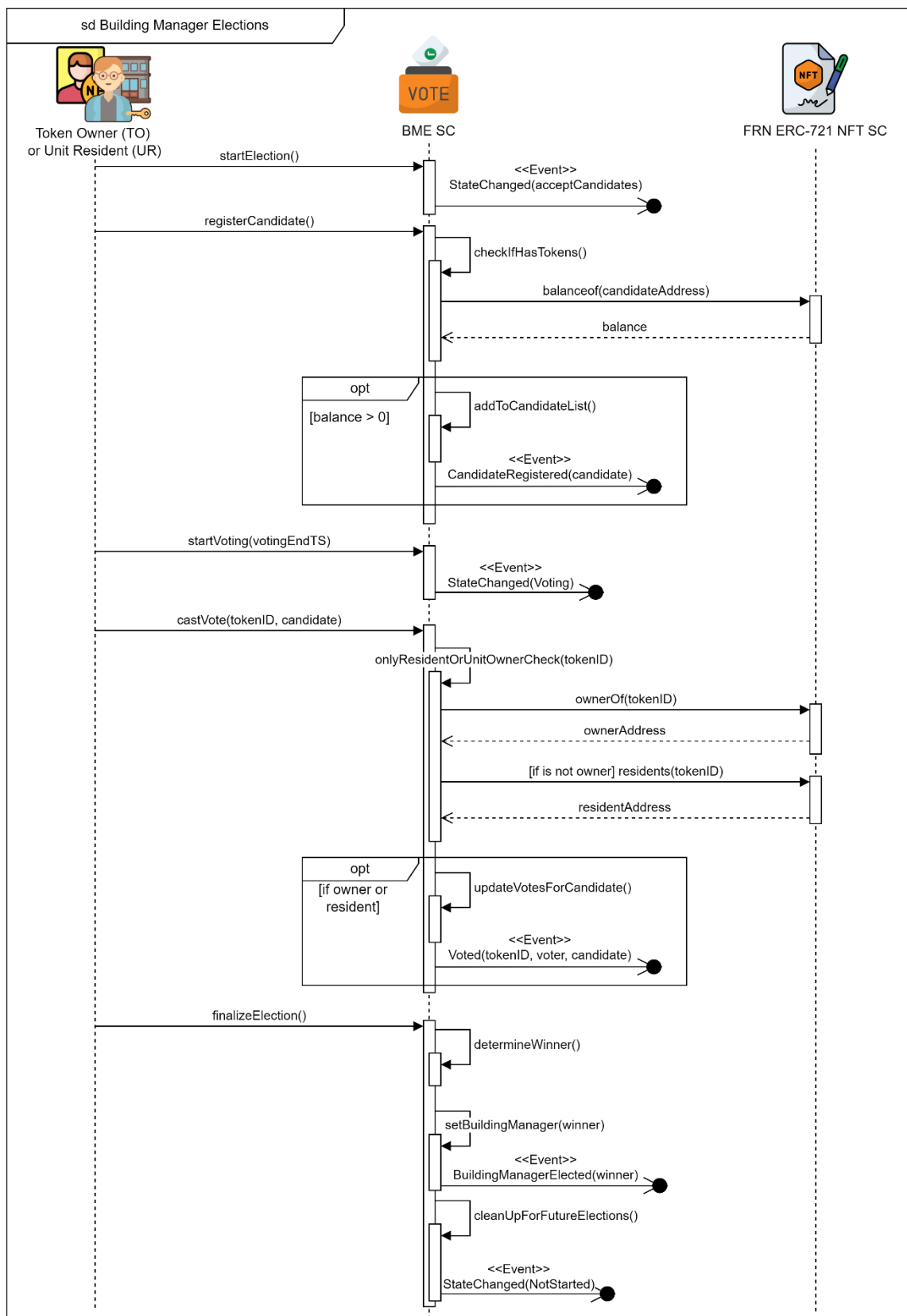
شکل 8-3 نمودار توالی تخصیص و برداشت اجاره یک واحد اشتراکی

3-6-3 مدیریت ساختمان

در این بخش به دو جنبه‌ی مدیریتی ساختمان در سامانه پیشنهادی، یعنی فرایند انتخابات مدیر ساختمان و جمع‌آوری و مدیریت هزینه‌های سرانه‌ی خدمات واحدها خواهیم پرداخت.

نمودار توالی ارائه شده در شکل 9-3، روند انتخابات مدیر ساختمان را تشریح می‌کند. به طور خلاصه، این فرایند را می‌توان به سه مرحله‌ی زیر تقسیم کرد:

1. ثبت نام: هر TO می‌تواند خود را به عنوان نامزد در شبکه معرفی کند.
2. رأی‌گیری: از هر واحد، یک TO یا UR می‌تواند به یک نامزد رأی بدهد.
3. نتایج: نامزدی که بیشترین آرا را داشته باشد به عنوان مدیر ساختمان (BM) انتخاب می‌شود.

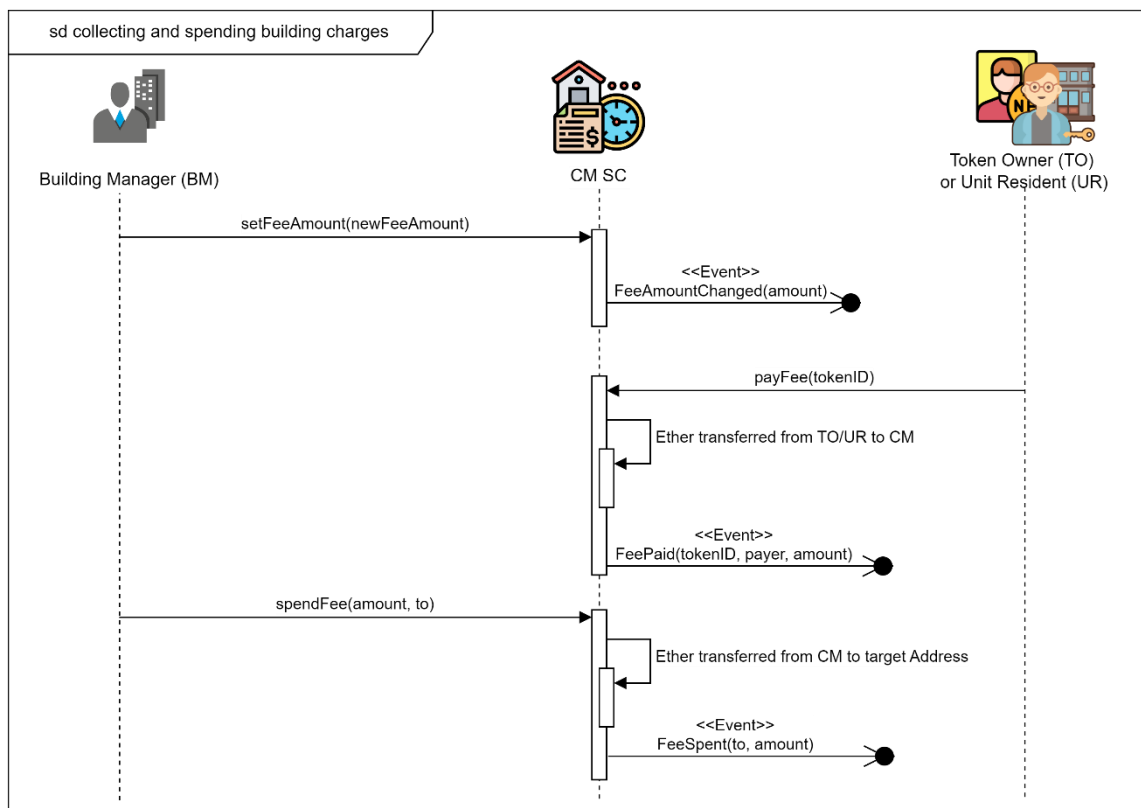


شکل 9-3 نمودار توالی انتخابات مدیر ساختمان

این فرایند با شروع انتخابات توسط مالک یک توکن (TO) یا ساکن یک واحد (UR) آغاز می‌گردد. سپس نامزدها به صورت داوطلبانه با فراخوانی تابع مربوطه خود را به شبکه معرفی می‌کنند. سپس به این نامزدها رأی داده می‌شود. قرارداد هوشمند BME از طریق فراخوانی تابعی از قرارداد هوشمند FRN، بررسی می‌کند که آیا نامزدها و رأی‌دهندگان از شرایط کافی برای شرکت در انتخابات برخوردار هستند یا خیر و آرا را بر اساس آن به‌روزرسانی می‌کند. پس از پایان رأی‌گیری، با مشخص شدن برنده بر اساس تعداد آرا و منصوب شدن آن به عنوان مدیر ساختمان، انتخابات به پایان می‌رسد و قرارداد هوشمند BME خود را برای انتخابات آینده آماده می‌کند.

پس از اینکه یک TO به عنوان BM انتخاب شد، آن BM می‌تواند هزینه‌های سرانه‌ی خدمات ساختمان را مدیریت کند. این مدیریت شامل تنظیم هزینه سرانه‌ی خدمات و تخصیص آن می‌باشد. شکل 3-10 نمودار توالی این فرایند را به صورت زیر توضیح می‌دهد:

1. **تنظیم:** مدیر ساختمان میزان هزینه را تعیین می‌کند.
2. **پرداخت:** مالک یا ساکن واحد هزینه‌ی سرانه‌ی خدمات مربوطه را به CM پرداخت می‌کند.
3. **تخصیص:** مدیر ساختمان با تعیین مبلغ مشخص و گیرنده، مبلغ موردنظر را از CM به گیرنده واریز می‌کند.



شکل 3-10 نمودار توالی جمع‌آوری و صرف هزینه‌های ساختمان

7-3 جمع‌بندی

این فصل شرح جامعی از سامانه توکنیزه‌کردن و مدیریت املاک و مستغلات پیشنهادی را ارائه کرد و به تشریح دقیق عملکردها، معماری روش و فناوری‌های اصلی آن پرداخت. این سامانه ویژگی‌های مختلف زیر را که برای سرمایه‌گذاری و مدیریت املاک و مستغلات حیاتی هستند ارائه می‌کند:

- **توکنیزه‌کردن و مالکیت جزئی:** این سامانه ایجاد و مدیریت توکن‌های تک مالکی (FRN) و چند مالکی (FRF) را با استفاده از استانداردهای موجود امکان‌پذیر می‌کند.
- **مدیریت غیرمتمرکز:** این سامانه دارای یک سازمان غیرمتمرکز خودگردان (DAO) می‌باشد که صاحبان توکن‌های اشتراکی را قادر می‌سازد تا از طریق رأی‌دادن به پیشنهادها در فرایندهای تصمیم‌گیری مشترک مرتبط با هر واحد شرکت کنند.
- **مدیریت اجاره و هزینه‌های سرانه‌ی خدمات:** قراردادهای هوشمند RM و CM، جمع‌آوری اجاره و سرانه‌ی خدمات در بین واحدهای یک ساختمان را تسهیل و در بستر بلاکچین امکان‌پذیر می‌کنند.
- **مدیریت ساختمان:** این سامانه انتخاب مدیر ساختمان (BM) را از طریق فرایندهای انتخابات موجود در قرارداد هوشمند BME ممکن کرده و عملکردهایی را برای مدیر ساختمان که جهت انجام وظایف مدیریتی ضروری است تعریف می‌کند.

سامانه پیاده‌سازی شده در این فصل شامل شش قرارداد هوشمند و پنج بازیگر مختلف می‌باشد و از زبان برنامه‌نویسی Solidity و محیط توسعه Hardhat جهت توسعه و IPFS برای ذخیره‌ی اطلاعات به‌صورت غیرمتمرکز استفاده کرد. بازیگران این سامانه شامل مالک ساختمان (BO)، مالک توکن (TO)، مالک جزئی (FO)، ساکن واحد (UR) و مدیر ساختمان (BM) می‌شوند و قراردادهای هوشمند پیاده‌سازی شده به‌صورت قراردادهای هوشمند مالکیت کلی و جزئی FRN و FRF جهت توکنیزه‌کردن، مدیریت هزینه‌های سرانه‌ی خدمات (CM) و اجاره (RM) جهت مدیریت کلی، قرارداد هوشمند مرتبط با DAO (FD) برای مدیریت اشتراکی و در نهایت انتخابات مدیر ساختمان (BME) می‌باشد. در ادامه، توضیحاتی در مورد توسعه و هدف هر قرارداد هوشمند ارائه و توابع و رویدادهای هر کدام به‌صورت جداگانه تشریح داده شد. علاوه‌برآن، گردش کار و تعاملات مهم سامانه از طریق نمودارهای توالی بیان شد که فرایندهای توکنیزه‌کردن، عملیات DAO و مدیریت ساختمان را نشان داد. در مجموع، این فصل پایه و اساس سامانه مبتنی بر بلاکچین املاک و مستغلات پیشنهادی ما را شامل می‌شود و زیرساخت‌های فناوری و نحوه‌ی پیاده‌سازی هر بخش را به‌صورت کامل تشریح می‌کند. در فصل بعدی، استقرار، ارزیابی و تجزیه و تحلیل سامانه را باهدف نشان‌دادن اثربخشی و امکان‌سنجی آن در بازار املاک و مستغلات، بررسی خواهیم کرد.

فصل چهارم

تحلیل و ارزیابی روش پیشنهادی

1-4 مقدمه

این فصل بر روی استقرار و ارزیابی قراردادهای هوشمند شرح داده شده در فصل قبل تمرکز دارد. این فصل با فرایند استقرار و تشریح جزئیات هر مرحله و در دسترس قراردادن آن‌ها در شبکه‌ی آزمایشی اتریوم شروع می‌شود. پس از استقرار، یک ارزیابی جامع از کد مستقر شده با در نظر گرفتن جنبه‌های مختلف مانند امنیت، هزینه و عملکرد انجام می‌شود. این ارزیابی شامل تجزیه و تحلیل آسیب‌پذیری، یکپارچگی داده‌ها، دردسترس بودن، مسئولیت‌پذیری و عدم انکار می‌شود. علاوه بر آن، هزینه‌های مربوط به استقرار و اجرای قراردادهای هوشمند از طریق ابزارهای مرتبط به نمایش گذاشته می‌شود. هدف این ارزیابی، به دست آوردن بینشی در مورد اثربخشی و استحکام قراردادهای هوشمند مستقر شده و شناسایی نقاط قوت و مناطق بالقوه برای بهبود است. همچنین با ارزیابی کامل کد مستقر شده در ابعاد مختلف، اطمینان حاصل خواهیم کرد که سامانه پیشنهادی استانداردهای مورد نیاز امنیت، مقرون به صرفه بودن و کارایی را برآورده می‌کند.

2-4 آزمون

پس از توسعه‌ی قراردادهای هوشمند و پیش از استقرار آن‌ها، آزمایش کامل برای اطمینان از صحت عملکرد و قابلیت اطمینان آن‌ها ضروری است. این بخش روش آزمایش بکار گرفته شده و سطح پوشش به دست آمده از این آزمایش‌ها را تشریح می‌کند.

محیط و ابزار آزمون:

1. **محیط توسعه:** چارچوب Hardhat قابلیت نوشتن آزمون که از پیش درون آن تعبیه شده است را به ما ارائه می‌دهد.

2. **زبان برنامه‌نویسی:** بر اساس توصیه Hardhat، از زبان برنامه‌نویسی TypeScript برای نوشتن آزمون استفاده شده است. چرا که این زبان جز زبان‌های Type-safe¹ محسوب می‌شود و تشخیص زودهنگام خطاهای احتمالی را در طول فرایند توسعه تسهیل می‌کند.

3. **چارچوب و کتابخانه:** از چارچوب Mocha² و کتابخانه‌ی Chai³ که ابزارهایی محبوب در زبان‌های JavaScript و TypeScript برای توسعه‌ی آزمون هستند استفاده شده است. این ابزارها روشی ساختاریافته و گویا را برای نوشتن موارد آزمایشی در قالب کد به صورت واضح و مختصر ارائه می‌کنند.

¹ در مقابل خطاهای ناشی از ناسازگاری نوع داده ایمن است.

² mochajs.org

³ chaijs.com

محدوده‌ی آزمون:

تمامی عملکردهای سه قرارداد هوشمند اصلی تعریف شده در فصل سوم به همراه سه قرارداد هوشمند ارث برده شده توسط آن‌ها مورد آزمون و راستی‌آزمایی قرار می‌گیرند. همچنین تعاملات بین هر قرارداد هوشمند با سایر قراردادها نیز به طور کامل آزمایش می‌شود. هدف از مجموعه‌ی آزمون نوشته‌شده این است که تمامی جنبه‌های مختلف این شش قرارداد را از لحاظ دستورات تعریف شده، شاخه‌های ایجاد شده، توابع و خط‌های نوشته شده به طور کامل پوشش دهد.

اجرای آزمون:

مجموعه‌ی آزمون از طریق دستور `hardhat test` در ترمینال اجرا می‌شود. این دستور از چارچوب توسعه‌ی `hardhat`، تمامی آزمون‌های نوشته شده در پوشه‌ی `test` پروژه را اجرا می‌کند. شکل 4-1 خروجی این دستور که در ترمینال چاپ می‌گردد را نشان می‌دهد و بیانگر این است که تمامی آزمون‌های نوشته شده با موفقیت قبول شده‌اند.

```
→ FractoReal git:(main) hh test

FractoRealFractions
Deployment
  ✓Should deploy and mints (1662ms)
Behavioral Full Test for Election and Management after EM is chosen
  ✓Should elect a new owner successfully (703ms)
noContract test
  ✓registerCandidate should revert if request is from contract

FractoRealFractions
Deployment
  ✓Should deploy and mint (656ms)
mint not allowed
  ✓Should revert if not from erc721 contract
setURI
  ✓Should revert if not owner
  ✓Should set URI
rebuildNFT
  ✓should revert if not set
  ✓should revert if not owned
  ✓should rebuild successfully if owned (56ms)
splitRent
  ✓should revert if no rent available
  ✓behavioural: DAO and split and withdraw rent (testing DAO for proposals, splitRent and RentSplited) (474ms)

FractoRealNFT
Deployment
  ✓Should be deployed
  ✓Should set the right owner
  ✓Should not have any tokens
mint
  phaseOneMint
    ✓should revert if sale is not started
    ✓should revert if sale is ended
    with invalid signature
      ✓should revert with invalid price
      ✓should revert with invalid tokenId
      ✓should revert with invalid signer
      ✓should revert with invalid contract address
      ✓should revert with invalid signature
      ✓should revert with invalid minter
    with valid signature
      ✓should mint 1 token
      ✓should mint multiple tokens (92ms)
      ✓should revert if already minted
      ✓should revert with invalid sent eth
      ✓should revert with invalid price
      ✓should revert with invalid tokenId
      ✓should revert with another minter
  phaseTwoMint
    ✓should revert if sale is not started
    ✓should revert if erc1155 address not set
    ✓should mint (493ms)
fractionize
  ✓should revert if not minted
  ✓should revert if erc155 not set
  ✓should revert if not approved or owner (51ms)

  ✓should fractionize (80ms)
onlyOwner
setTimes
  phaseOneStartTime
    ✓should revert if not owner
    ✓should set phaseOneStartTime
  phaseTwoStartTime
    ✓should revert if not owner
    ✓should set phaseTwoStartTime
mint
  ✓should revert if not owner
  ✓should mint 1 token
withdraw
  ✓should revert if not owner
  ✓should withdraw
setBaseURI
  ✓should revert if not owner
  ✓should set baseURI
setMeterages
  ✓should revert if not owner
  ✓should set meterages
  ✓should revert on invalid length
batchMint
  ✓should revert if not owner
  ✓should batch mint 5 tokens (54ms)
setErc1155Address
  ✓should revert if not owner
  ✓should set erc1155 address
RentManagement
  Reverts test, other test are done with DAO in frf
  ✓should revert on setResident if not authorized
  ✓should revert on setRentFee if not authorized
  ✓should revert on withdrawRent if not authorized
inheritance function tests
  supportsInterface
    ✓should return true for ERC721 interface
  _increaseBalance
    ✓should increase Balance for selected token
noContract test
  ✓phaseOneMint should revert if request is from contract

FractoRealNFT
setMeterages
  ✓should set meterages for 100 units (86ms)
mint
  ✓should mint 100 units (956ms)
batchMint
  ✓should mint 100 units (479ms)

63 passing (7s)
→ FractoReal git:(main)
```

شکل 4-1 خروجی دستور اجرای آزمون

اندازه‌گیری سطح پوشش کد:

برای متوجه‌شدن از عملکرد آزمون‌های نوشته شده و اطمینان از اینکه تمامی بخش‌های کد مورد

راستی‌آزمایی قرار گرفته‌اند، از افزونه‌ی ¹solidity-coverage استفاده شده است. این افزونه به طور یکپارچه با Hardhat ادغام شده است و امکان اندازه‌گیری میزان و سطح پوشش آزمون‌های نوشته شده در قراردادهای هوشمند را فراهم می‌کند. شکل 2-4 خروجی این ابزار را به ما نشان می‌دهد. همان‌طور که از جدول خروجی پیداست، مجموعه آزمون نوشته شده تمامی جنبه‌های قراردادهای هوشمند از لحاظ دستورات، شاخه‌ها، توابع و خط‌های تعریف شده را به صورت ۱۰۰ درصد پوشش می‌دهد.

all files contracts/

100% Statements 162/162 100% Branches 148/148 100% Functions 54/54 100% Lines 252/252

File	Statements	Branches	Functions	Lines
BuildingManagerElection.sol	100%	43/43	100%	32/32
ChargeManagement.sol	100%	7/7	100%	10/10
FractionsDAO.sol	100%	27/27	100%	28/28
FractoRealFractions.sol	100%	41/41	100%	28/28
FractoRealNFT.sol	100%	33/33	100%	36/36
RentManagement.sol	100%	11/11	100%	14/14

63 passing (7s)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	100	100	100	100	
BuildingManagerElection.sol	100	100	100	100	
ChargeManagement.sol	100	100	100	100	
FractionsDAO.sol	100	100	100	100	
FractoRealFractions.sol	100	100	100	100	
FractoRealNFT.sol	100	100	100	100	
RentManagement.sol	100	100	100	100	
All files	100	100	100	100	

شکل 2-4 خروجی دستور اندازه‌گیری سطح پوشش کد

3-4 تجزیه و تحلیل آسیب‌پذیری

پس از آزمون تمامی بخش‌های قراردادهای هوشمند و اطمینان از صحت عملکرد آن‌ها، یک تجزیه و تحلیل آسیب‌پذیری استاتیک با استفاده از ابزار Slither² بر روی قراردادهای هوشمند انجام شد. Slither یک ابزار مبتنی بر پایتون است که برای تجزیه و تحلیل قراردادهای هوشمند نوشته شده با زبان‌های Solidity و Vyper و باهدف پیدا کردن آسیب‌پذیری‌های احتمالی در آن‌ها طراحی شده است. این ابزار قابلیت‌های مختلفی را ارائه می‌دهد، از جمله این قابلیت‌ها می‌توان به موارد زیر اشاره کرد:

- اجرای مجموعه‌ای از کشف‌کننده‌های آسیب‌پذیری برای شناسایی نقاط ضعف در کد با مثبت کاذب کم. تعداد این کشف‌کننده‌ها بر اساس مستندات این ابزار ۹۴ عدد گزارش شده است.

¹ github.com/sc-forks/solidity-coverage

² github.com/crytic/slither

- پشتیبانی و ادغام راحت با چارچوب Hardhat استفاده شده در فرایند توسعه‌ی این سامانه
- خروجی‌های مختلف با ارائه‌ی جزئیات کامل برای کمک به درک بهتر کد

نتایج تجزیه و تحلیل Slither:

مسائل گزارش شده توسط Slither بر اساس نوع گزارش یا سطح آسیب‌پذیری به پنج نوع مختلف تقسیم می‌شوند. این مسائل به ترتیب سطح اهمیت از دید آسیب‌پذیری عبارت‌اند از:

1. **مسائل بهینه‌سازی:** مناطق بالقوه‌ای که می‌توان کد را برای بهبود بازدهی یا کاهش هزینه گاز اصلاح کرد.
2. **مسائل آگاه‌کننده:** یافته‌هایی که لزوماً نشان‌دهنده آسیب‌پذیری نیستند، اما به مناطقی اشاره می‌کنند که می‌توان کد را برای خوانایی بهتر و افزایش قابلیت نگهداری بهبود داد.
3. **آسیب‌پذیری‌های سطح پایین:** این مسائل نشان‌دهنده‌ی ضعف‌های بالقوه با حداقل تأثیر امنیتی هستند و ممکن است نیاز به توجه داشته باشند، اما در مقایسه با آسیب‌پذیری‌های سطوح بالاتر کمتر مورد سوءاستفاده قرار می‌گیرند.
4. **آسیب‌پذیری‌های سطح متوسط:** این نوع از یافته‌ها حاکی از وجود آسیب‌پذیری‌هایی با پتانسیل متوسط جهت سوءاستفاده برای اهداف مخرب است. نقاط گزارش شده در این بخش و بخش بالاتر نیاز به رسیدگی دارند و باید به سرعت مورد توجه قرار گیرند.
5. **آسیب‌پذیری‌های سطح بالا:** این آسیب‌پذیری‌ها ماهیت شدیدی دارند و می‌توانند منجر به ازدست‌دادن دارایی‌ها و یا کنترل قرارداد هوشمند شوند.

شکل 3-4 تصویری از گزارش Slither را به صورت خلاصه شده ارائه می‌دهد که شامل تعداد کل خطوط کد بررسی شده و تعداد آسیب‌پذیری‌های شناسایی شده می‌شود. جدول 1-4، موارد پیدا شده در کد را بر اساس گزارش‌های Slither که در شکل 4-4 و شکل 5-4 به صورت کامل قابل مشاهده است را به طور خلاصه بیان کرده است.

جدول 1-4 مسائل پیدا شده توسط Slither در کد بر اساس نوع

نوع مسئله	یافته‌ها	تعداد گزارش شده
بهینه‌سازی	-	۰
آگاه‌کننده	عملیات پرهزینه در حلقه فراخوانی سطح پایین خارجی ^۱	۳

^۱ Low-level external call

۹	ورود مجدد احتمالی سطح پایین مقایسه بر اساس مهر زمانی	سطح پایین
۰	-	سطح متوسط
۰	-	سطح بالا

```
(venv) → FractoReal git:(main) X slither . --print human-summary
'npx hardhat clean' running (wd: /home/yedoost/w/github.com/ahbanavi/FractoReal)
'npx hardhat clean --global' running (wd: /home/yedoost/w/github.com/ahbanavi/FractoReal)
'npx hardhat compile --force' running (wd: /home/yedoost/w/github.com/ahbanavi/FractoReal)
INFO:Printers:
Compiled with Builder
Total number of contracts in source files: 6
Number of contracts in dependencies: 30
Source lines of code (SLOC) in source files: 676
Source lines of code (SLOC) in dependencies: 1504
Number of assembly lines: 0
Number of optimization issues: 0
Number of informational issues: 3
Number of low issues: 9
Number of medium issues: 0
Number of high issues: 0
```

Use: Openzeppelin-Ownable, Openzeppelin-ERC721
ERCs: ERC721, ERC165

Name	# functions	ERCs	ERC20 info	Complex code	Features
ChargeManagement	15			No	Receive ETH Send ETH Tokens interaction
FractoRealFractions	69	ERC165		Yes	Receive ETH Send ETH Tokens interaction
FractoRealNFT	90	ERC165, ERC721		No	Assembly Receive ETH Send ETH Ecrecover Tokens interaction Assembly

شکل 3-4 گزارش خلاصه شده Slither

بر اساس این خلاصه گزارش، کد از نظر بهینگی به صورت کاملاً بهینه نوشته شده است و همچنین از آسیب پذیری های سطح متوسط و بالا مصون می باشد. هرچند که شامل ۳ گزارش آگاه کننده و همچنین ۹ گزارش سطح پایین است که در جلوتر با جزئیات کامل به آن ها خواهیم پرداخت. تعداد کل خطوط بررسی شده در شش قرارداد هوشمند اصلی برابر ۶۷۶ خط، و در ۳۰ قراردادهای هوشمند ارث بری شده در کتابخانه ها برابر ۱۵۰۴ خط می باشد. در مجموع سامانه پیاده سازی شده شامل ۲۱۸۰ خط کد در ۳۶ قرارداد هوشمند است.

1-3-4 یافته های آگاه کننده

در شکل 4-4، Slither سه یافته آگاه کننده را شناسایی کرد که در دو دسته ی «عملیات پرهزینه در حلقه» در تابع نهایی سازی انتخابات مدیر ساختمان و «فراخوانی سطح پایین خارجی» در تابع اجرای پیشنهاد FD تقسیم می شوند.

```
(venv) → FractoReal git:(main) X slither . --exclude-low
'npx hardhat clean' running (wd: /home/yedoost/w/github.com/ahbanavi/FractoReal)
'npx hardhat clean --global' running (wd: /home/yedoost/w/github.com/ahbanavi/FractoReal)
'npx hardhat compile --force' running (wd: /home/yedoost/w/github.com/ahbanavi/FractoReal)
INFO:Detectors:
BuildingManagerElection.finalizeElection() (contracts/BuildingManagerElection.sol#182-240) has costly operations i
nside a loop:
- delete candidates[_candidateList[i_scope_0]] (contracts/BuildingManagerElection.sol#218)
BuildingManagerElection.finalizeElection() (contracts/BuildingManagerElection.sol#182-240) has costly operations i
nside a loop:
- delete voters[i_scope_1] (contracts/BuildingManagerElection.sol#229)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#costly-operations-inside-a-loop
INFO:Detectors:
Low level call in FractionsDAO.executeProposal(uint256) (contracts/FractionsDAO.sol#187-204):
- (success,data) = proposal.targetAddress.call(proposal.data) (contracts/FractionsDAO.sol#198-200)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Slither:. analyzed (36 contracts with 76 detectors), 3 result(s) found
```

شکل 4-4 گزارش یافته‌های آگاه‌کننده Slither

این یافته‌ها اگرچه توسط Slither علامت زده شده‌اند، آسیب‌پذیری امنیتی را نشان نمی‌دهند و صرفاً ماهیت اطلاعاتی دارند. شرح این یافته‌ها به صورت زیر است:

- **عملیات پرهزینه در حلقه:** دو مورد در تابع `finalizeElection` از قرارداد هوشمند BME شناسایی شد که شامل حذف عناصر از آرایه‌های ذخیره‌سازی مرتبط با نامزدها و رأی‌دهندگان می‌شوند. درحالی‌که این عملیات به‌عنوان پرهزینه گزارش شده است، اما برای عملکرد صحیح منطق انتخابات ضروری هستند و قبلاً به بهترین حد ممکن بهینه‌سازی شده‌اند.
- **فراخوانی سطح پایین خارجی:** آخرین یافته‌ی آگاه‌کننده در تابع `executeProposal` از قرارداد هوشمند FD گزارش شده است. طبق اطلاعات موجود در مستندات Slither، این مورد زمانی گزارش می‌شود که کد مربوطه بررسی نکند که فراخوانی بر روی یک قرارداد هوشمند اجرا می‌شود یا خیر؛ بااین‌حال، این گزارش در این نقطه یک گزارش مثبت کاذب است، چرا که پیاده‌سازی به‌گونه‌ای صورت گرفته است که پیش‌ازاین در تابع ثبت پیشنهاد که در 3-2-5-4 توضیح داده شده است، شرط موردنیاز برای اجرای صحیح فراخوانی سطح پایین بررسی شده است.

2-3-4 یافته‌های سطح پایین

همان‌طور که در شکل 4-5 قابل مشاهده است، Slither ۹ یافته سطح پایین و کم‌تأثیر را شناسایی کرده است. در ادامه، بررسی دقیق‌تر نشان می‌دهد که این یافته‌ها به آسیب‌پذیری‌های عملیاتی در روش پیاده‌سازی این سامانه ختم نخواهند شد.

```
(venv) → FractoReal git:(main) X slither . --exclude-informational
'npx hardhat clean' running (wd: /home/yedooost/w/github.com/ahbanavi/FractoReal)
'npx hardhat clean --global' running (wd: /home/yedooost/w/github.com/ahbanavi/FractoReal)
'npx hardhat compile --force' running (wd: /home/yedooost/w/github.com/ahbanavi/FractoReal)
INFO:Detectors:
Reentrancy in FractoRealFractions.splitRent(uint256) (contracts/FractoRealFractions.sol#130-160):
  External calls:
    - rentAmount = ERC721.withdrawRent(tokenId) (contracts/FractoRealFractions.sol#133)
  State variables written after the call(s):
    - nonSharesRents += notSplitedRents (contracts/FractoRealFractions.sol#156)
    - tokenIdShareHolders[tokenId][i].rents += rent (contracts/FractoRealFractions.sol#145)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2
INFO:Detectors:
Reentrancy in FractionsDAO.executeProposal(uint256) (contracts/FractionsDAO.sol#187-204):
  External calls:
    - (success,data) = proposal.targetAddress.call(proposal.data) (contracts/FractionsDAO.sol#198-200)
  Event emitted after the call(s):
    - ProposalExecuted(proposalId,data) (contracts/FractionsDAO.sol#203)
Reentrancy in FractoRealFractions.splitRent(uint256) (contracts/FractoRealFractions.sol#130-160):
  External calls:
    - rentAmount = ERC721.withdrawRent(tokenId) (contracts/FractoRealFractions.sol#133)
  Event emitted after the call(s):
    - RentSplited(tokenId,rentAmount) (contracts/FractoRealFractions.sol#159)
Reentrancy in FractoRealFractions.withdrawRent(uint256) (contracts/FractoRealFractions.sol#166-181):
  External calls:
    - address(msg.sender).sendValue(rentMoney) (contracts/FractoRealFractions.sol#178)
  Event emitted after the call(s):
    - RentWithdrawn(tokenId,msg.sender,rentMoney) (contracts/FractoRealFractions.sol#180)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3
INFO:Detectors:
BuildingManagerElection.castVote(uint256,address) (contracts/BuildingManagerElection.sol#145-173) uses timestamp for comparisons
  Dangerous comparisons:
    - block.timestamp > votingEnd (contracts/BuildingManagerElection.sol#155)
BuildingManagerElection.finalizeElection() (contracts/BuildingManagerElection.sol#182-240) uses timestamp for comparisons
  Dangerous comparisons:
    - block.timestamp < votingEnd (contracts/BuildingManagerElection.sol#189)
FractionsDAO.castVote(uint256,bool) (contracts/FractionsDAO.sol#137-176) uses timestamp for comparisons
  Dangerous comparisons:
    - block.timestamp > proposal.voteEndTime (contracts/FractionsDAO.sol#145)
FractoRealNFT.phaseOneMint(bytes,uint256,uint256) (contracts/FractoRealNFT.sol#115-136) uses timestamp for comparisons
  Dangerous comparisons:
    - block.timestamp < phaseOneStartTime (contracts/FractoRealNFT.sol#120)
    - block.timestamp > phaseTwoStartTime (contracts/FractoRealNFT.sol#121)
FractoRealNFT.startPhaseTwoMint() (contracts/FractoRealNFT.sol#152-191) uses timestamp for comparisons
  Dangerous comparisons:
    - block.timestamp < phaseTwoStartTime (contracts/FractoRealNFT.sol#154)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Slither: analyzed (36 contracts with 73 detectors), 9 result(s) found
```

شکل 4-5 گزارش آسیب‌پذیری‌های سطح پایین Slither

۹ مورد آسیب‌پذیری سطح پایین، به دو دسته‌ی زیر تقسیم می‌شوند:

- **ورود مجدد احتمالی سطح پایین:** گزارش، آسیب‌پذیری‌های بالقوه ورود مجدد را در دو تابع برداشت اجاره و اجرای پیشنهاده شناسایی کرده است. با این حال، پس از بررسی مشخص شد که این یافته‌ها به دلیل ماهیت پیاده‌سازی فراخوانی توابع خارجی در سامانه پیشنهادی، منجر به آسیب‌های ناشی از ورود مجدد نخواهند شد. در تابع برداشت اجاره، از آنجایی که فراخوانی خارجی، قرارداد دیگر طراحی شده در همین سامانه است، حمله ورود مجدد امکان‌پذیر نیست. به طور مشابه در هنگام اجرای پیشنهاد، پیشنهادها باید قبل از اجرا توسط فرایند رأی‌گیری DAO تأیید شوند، بنابراین خطر ورود مجدد را کاهش می‌دهد چرا که هر عملکرد بالقوه خطرناک در یک پیشنهاد در فرایند رأی‌گیری باطل می‌گردد.
- **مقایسه بر اساس مهر زمانی:** در سامانه پیشنهادی، مقایسه بر اساس مهر زمانی در توابع مربوط به رأی‌گیری در BME و FD و همچنین زمان‌بندی گام‌های اول و دوم فروش رخ می‌دهد. این مقایسات به دلیل اینکه ماینرها می‌توانند زمان یک بلوک را دست‌کاری کنند خطرناک تلقی می‌شوند. اما باتوجه به نحوه‌ی پیاده‌سازی پرتکل‌های اتریوم مانند Geth، اگر اختلاف زمانی که ماینرها یک بلوک را دست‌کاری می‌کنند بیشتر از ۱۵ ثانیه باشد، آن بلوک رد می‌شود؛ بنابراین اگر اختلاف زمانی بین مهرهای زمانی مقایسه شده بیشتر از ۱۵ ثانیه باشد ایمن در نظر گرفته می‌شود [31]. از آنجایی که ماهیت فرایندهای رأی‌گیری انتخابات و پیشنهادها به گونه‌ای است که از ۱۵ ثانیه به مراتب بیشتر طول می‌کشد، فاصله زمانی بین مقایسات ما بیشتر از ۱۵ ثانیه خواهد بود و

در نتیجه، این یافته‌ها در اجرای قراردادهای هوشمند ما غیرحیاتی تلقی می‌شوند.

به طور خلاصه، در فرایند تجزیه و تحلیل آسیب‌پذیری از Slither، یک ابزار تجزیه و تحلیل استاتیک زبان Solidity، برای شناسایی نقاط ضعف احتمالی در قراردادهای هوشمند پیاده‌سازی شده در این پایان‌نامه استفاده شد. بر اساس گزارش‌های این ابزار، هیچ آسیب‌پذیری سطح متوسط و بالایی شناسایی نشد که اطمینان خوبی را از وضعیت امنیتی کد ارائه می‌کند. همچنین درحالی‌که این گزارش‌ها از یافته‌های آگاه‌کننده و کم‌تأثیری نیز یاد می‌کنند، یک بررسی به‌صورت دستی و کامل تأیید کرد که تمامی این موارد به آسیب‌پذیری‌های بالقوه که منجر به ایجاد اختلال در فرایندهای سامانه می‌شود ختم نخواهند شد.

4-4 استقرار

این بخش به جزئیات فرایند استقرار قراردادهای هوشمند توسعه‌یافته در فصل قبل، در شبکه‌ی آزمایشی Sepolia که در 3-3-1 توضیح داده شد، می‌پردازد. فرایند استقرار از چارچوب توسعه Hardhat، در کنار یک اسکریپت سفارشی شده جهت تسهیل و خودکارسازی استفاده می‌کند.

محیط استقرار و ابزار استفاده شده:

4. شبکه بلاکچین: شبکه‌ی آزمایشی Sepolia¹
5. ابزار استقرار: چارچوب توسعه Hardhat و زبان TypeScript
6. رابط بلاکچین: واسط برنامه کاربردی (API²) Infura³
7. تأیید، ارتباط و مشاهده‌ی قراردادهای هوشمند در شبکه: کاوشگر Etherscan⁴
8. کیف پول اتریوم: Metamask⁵

فرایند استقرار:

1. ادغام Infura: رابط و دروازه‌ی بلاکچین Infura به‌عنوان مجرای ارسال تراکنش‌ها به شبکه‌ی Sepolia و سایر شبکه‌های بلاکچینی عمل می‌کند. یک کلید خصوصی API که از طریق ایجاد حساب کاربری در وبگاه Infura به دست می‌آید، به طور ایمن در پیکربندی Hardhat ذخیره می‌شود تا به اسکریپت استقرار مجوزهای لازم را برای تعامل با شبکه بدهد.
2. ادغام Etherscan: پس از استقرار، تأیید قراردادهای هوشمند در وبگاه Etherscan که یک

¹ sepolia.dev

² Application Programming Interface

³ infura.io

⁴ etherscan.io

⁵ metamask.io

کاوشگر بلوک‌های مرتبط با شبکه‌های اتریوم است، انجام می‌گردد. کلید اختصاصی Etherscan API که پس از ایجاد حساب در این وبگاه ایجاد می‌شود نیز در پیکربندی‌های Hardhat ذخیره شده تا توسط اسکریپت مورد استفاده قرار گیرد.

3. **کیف پول MetaMask:** برای استقرار قراردادهای هوشمند و ارسال تراکنش‌های مرتبط با این فرایند نیاز به یک کیف پول (حساب EOA) است. در این پیاده‌سازی، از یک کیف پول اختصاصی MetaMask که صرفاً برای اهداف استقرار و آزمایش ایجاد شده است استفاده می‌شود. کلید خصوصی مرتبط با این کیف پول در پیکربندی Hardhat گنجانده شده است و به اسکریپت استقرار اجازه می‌دهد تا در طول فرایند استقرار، تراکنش‌ها را از طریق این حساب اتریوم به شبکه ارسال کند. آدرس کیف پول استفاده شده در فرایندهای درون‌شبکه‌ای این پایان‌نامه به صورت زیر می‌باشد. تمامی تراکنش‌های ارسال شده از طریق این آدرس در کاوشگر مربوط به شبکه‌ی Sepolia در دسترس است:

0x722b489F1d5afa914C71c003Bc350Ae6B5702278

4. **اسکریپت استقرار:** اسکریپت استقرار که با زبان TypeScript نوشته شده است، به صورت سفارشی شده برای این سامانه توسعه یافته است. این اسکریپت هماهنگی‌های لازم را برای تعامل بخش‌های مختلف این فرایند، یعنی ارتباط Hardhat با Infura و کیف پول MetaMask را انجام می‌دهد تا قراردادهای هوشمند سامانه را به ترتیب در شبکه‌ی Sepolia مستقر کند. در نهایت این اسکریپت از طریق ارتباط با Etherscan، کد اصلی قراردادهای هوشمند را به این کاوشگر ارسال کرده و آن‌ها را تأیید می‌کند.

شکل 4-6، ادغام Etherscan، Infura و کیف پول MetaMask را در چارچوب توسعه‌ی Hardhat نشان می‌دهد. این پیکربندی در فایل با نام hardhat.config.ts ذخیره شده است. پس از انجام پیکربندی‌های موردنیاز، با اجرای دستور مربوطه در Hardhat، اسکریپت استقرار اجرا می‌گردد. شکل 4-7 دستور مربوطه و روند اجرایی آن را نشان می‌دهد. در ابتدا، اسکریپت استقرار آدرس عمومی صاحب ساختمان (BO) را از شخص مستقر کننده‌ی قرارداد سؤال می‌کند. سپس تعداد واحدهای ساختمان مشخص شده و پس از تأیید ورودی‌ها، اسکریپت قراردادهای هوشمند را برای استقرار به Infura ارسال می‌کند تا آن‌ها را در شبکه‌ی Sepolia مستقر کند. پس از استقرار این قراردادهای هوشمند، آدرس آن‌ها از طریق Infura به اسکریپت ارسال شده و در خروجی چاپ می‌گردد. در نهایت همان‌طور که در خروجی قابل مشاهده است، کد این قراردادهای هوشمند به کاوشگر Etherscan ارسال شده و عملیات صحت‌سنجی با موفقیت صورت می‌گیرد.

```
const config: HardhatUserConfig = {
  networks: {
    sepolia: {
      url: `https://sepolia.infura.io/v3/${INFURA_API_KEY}`, // Infura Integration
      accounts: [SEPOLIA_PRIVATE_KEY], // Metamask Wallet Integration
    },
  },
  etherscan: {
    apiKey: {
      sepolia: ETHERSCAN_API_KEY, // Etherscan Integration
    },
  },
};
```

شکل 4-6 پیکربندی‌های مورد نیاز Hardhat جهت فرایند استقرار

```
→ FractoReal git:(main) hh run scripts/deploy.ts --network sepolia
Deploying to sepolia network...
Enter the owner address: 0x722b489f1d5afa914c71c003bc350Ae6B5702278
Enter the maximum supply: 100

Confirm the following details:
Owner Address: 0x722b489f1d5afa914c71c003bc350Ae6B5702278
Max Supply: 100
Do you want to continue? (y/n): y

Deploying FractoRealNFT contract...
FractoRealNFT deployed to: 0x1F3706f4e43C061Fb880f832C96eEabC9978FD84.

Deploying FractoRealFractions contract...
FractoRealFractions deployed to: 0xaE2A3F9Ced096ff2236fA31b487978df267f8eED.

Deploying ChargeManagement contract...
ChargeManagement deployed to: 0x880E2EbF333bdB47855cB4f16E9E6DBa4928d91B.

Verifying contracts...
The contract 0x1F3706f4e43C061Fb880f832C96eEabC9978FD84 has already been verified on Etherscan.
https://sepolia.etherscan.io/address/0x1F3706f4e43C061Fb880f832C96eEabC9978FD84#code
The contract 0xaE2A3F9Ced096ff2236fA31b487978df267f8eED has already been verified on Etherscan.
https://sepolia.etherscan.io/address/0xaE2A3F9Ced096ff2236fA31b487978df267f8eED#code
The contract 0x880E2EbF333bdB47855cB4f16E9E6DBa4928d91B has already been verified on Etherscan.
https://sepolia.etherscan.io/address/0x880E2EbF333bdB47855cB4f16E9E6DBa4928d91B#code

All contracts deployed and verified successfully!
```

شکل 4-7 اجرای اسکریپت استقرار قراردادهای هوشمند در شبکه‌ی Sepolia

هر قرارداد هوشمند در قالب یک تراکنش از سمت کیف پول ما به شبکه ارسال می‌گردد. شکل 4-8 سه تراکنش مرتبط با عملیات استقرار را به طور خلاصه نشان می‌دهد. همچنین آدرس قراردادهای هوشمند مستقر شده در شبکه Sepolia در جدول 4-2 قابل مشاهده است. این قراردادهای هوشمند در کاوشگر Etherscan بخش مربوط به Sepolia به آدرس sepolia.etherscan.io قابل جستجو، مشاهده و تعامل هستند.

جدول 4-2 آدرس قراردادهای هوشمند مستقر شده در شبکه‌ی Sepolia

آدرس در شبکه Sepolia	قرارداد هوشمند
0x1F3706f4e43C061Fb880f832C96eEabC9978FD84	FractoRealNFT
0xaE2A3F9Ced096ff2236fA31b487978df267f8eED	FractoRealFractions
0x880E2EbF333bdB47855cB4f16E9E6DBa4928d91B	ChargeManagement

Transaction Hash	Method	Block	Age	From	To	Value	Txn Fee
0xd11efc067c...	0x60a06040	5751615	6 mins ago	0x722b489F..6B5702278	OUT Create: ChargeManage...	0 ETH	0.00100378
0xeb19fbf27ad...	0x61018060	5751614	6 mins ago	0x722b489F..6B5702278	OUT Create: FractoRealFrac...	0 ETH	0.00293107
0xf1c34336eea...	0x60a06040	5751613	6 mins ago	0x722b489F..6B5702278	OUT Create: FractoRealNFT	0 ETH	0.0024921

شکل 4-8 تراکنش‌های مرتبط با ایجاد قراردادهای هوشمند در شبکه‌ی Sepolia

4-5 تجزیه و تحلیل هزینه‌های شبکه

این بخش گزارشی از هزینه‌های استقرار و فراخوانی توابع مرتبط با سامانه پیاده‌سازی شده را ارائه می‌دهد. هزینه‌های گاز گزارش شده در این بخش از ابزار گزارش‌دهی گاز Hardhat گرفته شده و بر اساس قیمت گاز ۱۰ gwei و قیمت هر اتریوم ۱۸۰۰ دلار و ۹۲ سنت گزارش شده است. قیمت گاز و اتریوم بر اساس تاریخ اجرای ابزار یعنی ۱۲ آبان سال ۱۴۰۲ توسط خود ابزار محاسبه شده است. باتوجه به نوسان قیمت اتریوم و قیمت گاز بر اساس شلوغی شبکه، قیمت‌های دلاری گزارش شده در این بخش بر اساس زمان متغیر می‌باشد. این گزارش به صورت کامل در شکل 4-9 قابل مشاهده می‌باشد. پارامترهای تنظیم شده در هنگام نوشتن آزمون‌ها نیز سعی شده است طوری تنظیم شود که تقریباً شرایط پرهزینه‌ای را در هنگام اجرای ابزار شبیه‌سازی کند. برای مثال، حداکثر تعداد واحد هر ساختمان (تعداد توکن‌ها) ۵۰ عدد تنظیم شده است که این تنظیمات به طور مستقیم هزینه‌های گاز مرتبط با عملکردهای مبتنی بر حلقه را در توابع مورد نظر افزایش می‌دهد.

هزینه‌های استقرار

همان‌طور که در بخش قبل ذکر شد، استقرار هر قرارداد هوشمند در شبکه نیازمند تراکنش است و هر تراکنش نیز نیاز به پرداخت گاز برای پردازش دارد. هزینه‌ی گاز مرتبط با استقرار یک قرارداد هوشمند در شبکه بر اساس میزان بایت‌کد ارسال شده به شبکه برای ذخیره‌سازی عملگرهای توابع قرارداد هوشمند و همچنین متغیرهای ذخیره شده در هنگام استقرار تعیین می‌گردد. جدول زیر هزینه‌های مرتبط با استقرار هر قرارداد هوشمند در شبکه را نشان می‌دهد:

جدول 4-3 برآورد هزینه‌ی استقرار هر قرارداد هوشمند در شبکه اتریوم

قرارداد هوشمند	میانگین واحد گاز بر حسب gwei	گاز پرداختی بر حسب دلار
FractoRealNFT	۲,۴۹۰,۶۷۲	۴۴/۸۶
FractoRealFractions	۲,۹۲۹,۳۴۸	۵۲/۷۶
ChargeManagement	۱,۰۰۳,۲۰۵	۱۸/۰۷

Solidity and Network Configuration					
Solidity: 0.8.20	Optim: true	Runs: 500	viaIR: false	Block: 50,000,000 gas	
Network: ETHEREUM	L1: 10 gwei			1800.92 usd/eth	
Contracts / Methods	Min	Max	Avg	# calls	usd (avg)
ChargeManagement					
castVote	66,531	83,631	69,131	24	1.24
finalizeElection	-	-	222,167	3	4.00
payFee	53,195	56,189	54,692	4	0.98
registerCandidate	101,896	118,996	107,596	6	1.94
setFeeAmount	-	-	46,812	2	0.84
spendFee	-	-	34,935	2	0.63
startElection	-	-	44,674	2	0.80
startVoting	-	-	49,980	2	0.90
FractoRealFractions					
castVote	79,397	103,400	89,910	8	1.62
executeProposal	69,069	69,214	69,127	5	1.24
rebuildNFT	-	-	114,963	2	2.07
safeTransferFrom	50,246	153,490	128,464	5	2.31
setURI	-	-	46,980	1	0.85
splitRent	94,224	116,509	105,367	4	1.90
submitProposal	291,540	311,680	305,891	4	5.51
withdrawNonSharesRents	-	-	30,676	2	0.55
withdrawRent	-	-	35,697	4	0.64
FractoRealNFT					
batchMint	677,571	11,467,033	3,370,005	6	60.69
fractionize	-	-	260,444	1	4.69
mint	102,789	148,201	147,370	108	2.65
payRent	55,062	72,162	63,612	4	1.15
phaseOneMint	152,574	158,174	155,370	8	2.80
setBaseURI	-	-	46,688	1	0.84
setErc1155Address	-	-	46,150	5	0.83
setMeterages	47,712	2,295,015	993,548	6	17.89
setPhaseOneStartTime	-	-	29,784	17	0.54
setPhaseTwoStartTime	-	-	29,795	6	0.54
setResident	48,103	48,115	48,114	11	0.87
startPhaseTwoMint	5,089,513	8,319,165	6,704,339	2	120.74
withdraw	-	-	30,461	1	0.55
Deployments				% of limit	
ChargeManagement	-	-	1,003,205	2 %	18.07
FractoRealFractions	2,929,346	2,929,358	2,929,348	5.9 %	52.76
FractoRealNFT	-	-	2,490,672	5 %	44.86

شکل 4-9 گزارش گاز مصرفی هنگام استقرار قراردادهای هوشمند و فراخوانی توابع آنها

همان‌طور که در جدول مشخص است، هزینه‌ی استقرار هر قرارداد هوشمند با دیگری متفاوت می‌باشد. قرارداد FRF دارای بالاترین میانگین هزینه‌ی استقرار با هزینه‌ی ۵۲ دلار و ۷۶ سنت می‌باشد و پس از آن قرارداد هوشمند FRN با ۴۴ دلار و ۸۶ سنت قرار دارد. در نهایت قرارداد هوشمند CM با ۱۸ دلار و ۷ سنت در شبکه مستقر خواهد شد. کل هزینه استقرار برای هر سه قرارداد هوشمند را می‌توان با جمع کردن میانگین هزینه‌های USD فهرست شده در جدول محاسبه کرد (۱۱۵/۶۹ دلار). بااین‌حال، مهم است که ماهیت نوسان قیمت اتر را در نظر داشته باشیم، چرا که می‌تواند بر هزینه استقرار نهایی بر حسب دلار تأثیر بگذارد.

هزینه‌های توابع مهم

هزینه‌های اجرای توابع مهم در قراردادهای هوشمند سامانه به صورت زیر می‌باشد:

- **تابع ضرب توکن در گام اول¹:** این تابع مهم‌ترین تابعی است که توسط خریداران بالقوه هر واحد فراخوانی می‌شود. هزینه‌ی گاز مرتبط با این تابع به طور میانگین ۲ دلار و ۸۰ سنت می‌باشد که با وجود عملیات رمزگذاری و تأیید امضا در این تابع، هزینه‌ی کاملاً معقولی را نشان می‌دهد.
- **تابع آغاز عملیات گام دوم فروش²:** این تابع که صرفاً توسط مالک ساختمان قابل فراخوانی است، بیشترین هزینه‌ی گاز را در میان تمامی توابع سامانه متحمل می‌شود. هر چند قابل توجه است که این تابع در شرایطی آزمایش شده است که هر ساختمان ۵۰ واحد دارد و حلقه‌ی پیاده‌سازی شده در این تابع ۵۰ بار اجرا می‌گردد تا واحدهای موردنیاز در گام دوم فروش را پیدا کند. این تعداد در آزمون شبیه‌سازی شده ۳۰ واحد می‌باشد. سپس تمامی این ۳۰ واحد ابتدا در قرارداد هوشمند FRN و سپس در قرارداد FRF از طریق یک فراخوانی خارجی ضرب می‌گردد؛ از این رو هزینه‌ی تخمین زده شده در گزارش بالا می‌باشد. در یک سناریوی واقعی با تعداد واحد کل کمتر و تعداد بیشتر فروش در گام اول، پیش‌بینی می‌شود که هزینه‌ی گاز موردنیاز برای این تابع به میزان قابل توجهی کاهش یابد.
- **تابع تعیین متر از هر واحد³:** دومین تابع پرهزینه در قرارداد هوشمند می‌باشد. این تابع نیز همانند تابع قبل صرفاً توسط مالک ساختمان و صرفاً یک‌بار فراخوانی می‌گردد. در این تابع یک نداشت که در این برآورد ۵۰ موجودیت دارد درون شبکه‌ی بلاکچین ذخیره می‌شود و از این رو هزینه‌ی این تابع بالا می‌باشد. با تعداد واحد کمتر در هر ساختمان این هزینه نیز به میزان قابل توجهی کاهش می‌یابد.

هزینه‌های مرتبط با فراخوانی سایر توابع این سامانه در شکل 4-9 قابل مشاهده است و همان‌طور که از گزارش پیداست، سایر توابع با هزینه‌هایی بسیار کمتر از توابع ذکر شده در بالا پردازش می‌شوند.

هزینه در سایر شبکه‌های مبتنی بر EVM

تمامی تجزیه و تحلیل‌های هزینه‌ی ارائه شده در این بخش مختص شبکه‌ی اصلی بلاکچین اتریوم می‌باشد. سایر بلاکچین‌های مبتنی بر ماشین مجازی اتریوم (EVM)، مانند شبکه‌ی آربیتروم⁴ و یا بایننس⁵، در مقایسه

¹ phaseOneMint

² startPhaseTwoMint

³ setMeterages

⁴ arbitrum.io

⁵ Binance Smart Chain (BSC)

با شبکه‌ی اتریوم، با کارمزد تراکنش به‌مراتب کمتری تراکنش‌ها را پردازش می‌کند. قراردادهای هوشمند پیاده‌سازی شده در این پایان‌نامه به‌صورت کامل با تمامی شبکه‌های مبتنی بر EVM که از زبان Solidity پشتیبانی می‌کنند سازگار می‌باشند. علاوه‌برآن، با پیاده‌سازی بلاکچین خصوصی مبتنی بر EVM می‌توان هزینه‌های تراکنش را به حداقل رساند. هر چند انتخاب پلتفرم بلاکچین صرفاً بر اساس ارزان بودن کارمزد کار درستی نیست. این انتخاب باید با دقت زیادی انجام شود و عواملی مانند امنیت شبکه و سطح تمرکززدایی را نیز در کنار هزینه‌ی تراکنش در نظر گرفت.

6-4 تأخیر و توان عملیاتی

میزان تأخیر یک تراکنش بر اساس زمان ارسال یک تراکنش به شبکه و ذخیره‌شدن آن در یک بلوک محاسبه می‌گردد. در حال حاضر، در شبکه‌ی اتریوم این زمان به طور میانگین ۱۲/۰۹ ثانیه گزارش شده است [32]. با این وجود، این میزان تأخیر بیشتر تحت تأثیر گاز ارسال شده توسط کاربر می‌باشد. تراکنش‌های با میزان گاز پایین ممکن است توسط ماینرهای شبکه به مدت زمان طولانی نادیده گرفته شوند و تعاملات کاربر را با تأخیر بیشتری مواجه کنند.

توان عملیاتی در این سامانه به تعداد تراکنش‌هایی که شبکه می‌تواند در ثانیه پردازش کند (TPS^1) اشاره دارد. در زمان‌نگارش این پایان‌نامه، میزان TPS شبکه‌ی اتریوم حدود ۱۴ تراکنش بر ثانیه گزارش شده است، هر چند که از لحاظ تئوری شبکه‌ی اتریوم گنجایش ۱۱۹ تراکنش بر ثانیه را نیز دارد [33].

با این وجود، با هدف ارائه گزارشی دقیق‌تر، در این بخش میزان توان عملیاتی استقرار قراردادهای هوشمند و همچنین توابع مرتبط با توکنیزه کردن اندازه‌گیری شده است. برای این ارزیابی، یک اسکریپت سفارشی شده با زبان TypeScript توسعه داده شد و در چارچوب Hardhat اجرا گردید. این اسکریپت به طور مکرر (۱۰۰ مرتبه) توابع مربوط به استقرار و توکنیزه کردن را در شبکه‌ی محلی Ganache اجرا کرده و میانگین، حداقل و حداکثر زمان اجرای هر بخش را اندازه‌گیری می‌کند. اجرا و خروجی این اسکریپت در شکل 4-10 قابل مشاهده است. سخت‌افزار مرتبط با رایانه‌ای که بلاکچین محلی بر روی آن اجرا شده تأثیر بسیاری بر روی اندازه‌گیری‌های گزارش شده دارد. اسکریپت موردنظر در محیطی با مشخصات زیر اجرا گردیده است:

• پردازنده: Intel i7-6700K (8) @ 4.007GHz

• حافظه‌ی رم: 16 GB

• سیستم عامل: Arch Linux

¹ Transactions Per Second

• شبکه‌ی محلی: Ganache CLI v6.12.2 (ganache-core: 2.13.2)

بااین حال، باتوجه به این که در شبکه‌ی اتریوم توابع بر روی تأییدکنندگان (مایرها) متفاوتی اجرا می‌شود، اندازه‌گیری زمان اجرا از اهمیت کمتری در مقابل اندازه‌گیری پیچیدگی توابع که در بخش تجزیه و تحلیل هزینه‌ها در قالب گاز بیان شد، برخوردار است.

```
→ FractoReal git:(main) X hh run scripts/throughput.ts --network ganache
Testing with ganache network...
Number of transactions for each function: 100
Max supply: 301

Deployments:
  FractoRealNFT: Average: 97.43ms, Minimum: 88ms, Maximum: 126ms, TPS: 10.26
  FractoRealFractions: Average: 92.7ms, Minimum: 86ms, Maximum: 128ms, TPS: 10.79
  ChargeManagement: Average: 57.16ms, Minimum: 52ms, Maximum: 68ms, TPS: 17.49

Transactions:
  FractoRealNFT:
    phaseOneMint: Average: 83.88ms, Minimum: 71ms, Maximum: 261ms, TPS: 11.92
    transferFrom: Average: 255.17ms, Minimum: 75ms, Maximum: 367ms, TPS: 3.92
    fractionize: Average: 152.52ms, Minimum: 130ms, Maximum: 264ms, TPS: 6.56
  FractoRealFractions:
    safeTransferFrom: Average: 74.93ms, Minimum: 66ms, Maximum: 95ms, TPS: 13.35
    rebuildNFT: Average: 149.06ms, Minimum: 117ms, Maximum: 273ms, TPS: 6.71

Done!
```

شکل 4-10 مدت زمان استقرار و اجرای تراکنش‌ها

گزارش ارائه شده از توان عملیاتی، نشان داد که در تمامی حالات، تعداد تراکنش بر حسب ثانیه بیشتر از مقدار ۱ می‌باشد. این نشان می‌دهد که کاربران می‌توانند با ارائه‌ی گاز متناسب عملیات خود را در هر زمانی که می‌خواهند انجام دهند. به طور مثال، کمترین TPS گزارش شده، مرتبط با عملیات انتقال یک توکن FRN می‌باشد. این مقدار یعنی $\frac{3}{92}$ تراکنش بر ثانیه را اگر در میزان تأخیر شبکه‌ی اتریوم (میزان زمان استخراج یک بلوک) ضرب کنیم، به طور تقریبی عدد ۴۷ به دست می‌آید. این بدان معناست که در یک بلوک ۴۷ نفر (۴۷ تراکنش انتقال) می‌توانند توکن‌های خود را توسط تابع پیاده‌سازی شده در قرارداد هوشمند FRN انتقال دهند و اگر از این مقدار بیشتر شد، میزان تأخیر نیز به طور متناسب افزایش می‌یابد و تراکنش موردنظر در بلوک بعدی پردازش خواهند شد.

4-7 سایر ملاحظات امنیتی

علاوه بر تجزیه و تحلیل آسیب‌پذیری انجام شده با استفاده از Slither در بخش 4-3، در نظر گرفتن طیف گسترده‌تری از اصول امنیتی به منظور اطمینان از استحکام کلی قراردادهای هوشمند مستقر شده در شبکه‌ی بلاکچین ضروری است. این بخش چندین نگرانی کلیدی امنیتی از جمله یکپارچگی داده‌ها، دسترسی‌پذیری، مسئولیت‌پذیری و عدم انکار را بررسی می‌کند. هر یک از این اصول نقشی حیاتی در حفاظت از سامانه و داده‌های موجود در آن ایفا می‌کنند.

4-7-1 یکپارچگی داده‌ها

همان‌طور که در بخش 2-2-4 توضیح داده شد، بلاکچین به دلیل ماهیت تغییرناپذیر آن و الگوریتم‌های

اجماع، یکپارچگی تمام داده‌هایی را که در آن پردازش می‌شود یا به‌عنوان ورودی به آن ارسال می‌شود را به طور ذاتی تضمین می‌کند. داده‌های موجود در سامانه پیشنهادی به دو دسته‌ی داده‌های کوچک و داده‌های بزرگ تقسیم می‌گردد. داده‌های کوچک، مانند تراژ، مالک و ساکن هر ملک به طور کامل در شبکه‌ی بلاکچین ذخیره می‌گردند و از این‌رو از یکپارچگی بالایی برخوردار هستند. در مقابل، داده‌های بزرگ مانند عکس، فیلم یا اسناد حقوقی هر واحد، به دلیل اینکه ذخیره‌ی آن‌ها در شبکه بلاکچین پرهزینه است، توسط هش آن‌ها که در IPFS ذخیره شده به بلاکچین مرتبط می‌گردند. این عملیات نیز به دلیل تغییرناپذیر بودن هش اطلاعات، یکپارچگی داده‌ها را تضمین می‌کند. علاوه‌برآن، تمامی عملیات پردازش بر روی داده‌های ذخیره شده به طور مستقیم در محیط بلاکچین انجام می‌شود و خطر دست‌کاری غیرمجاز توسط عوامل مخرب را برطرف می‌کند.

با وجود تمام موارد ذکر شده، به‌رسمیت شناختن آسیب‌پذیری‌های بالقوه ذاتی در بخش الگوریتم اجماع مرتبط با شبکه‌ی اتریوم، حائز اهمیت است. درحالی‌که اتریوم با الگوریتم اثبات سهم (PoS) امنیت داده‌ها را به طور قابل‌توجهی افزایش داده است، خطر حمله‌ی ۵۱ درصد از لحاظ تئوری وجود دارد. هرچند باتوجه‌به ماهیت توزیع‌شده‌ی گره‌های تأییدکننده اتریوم و نیاز به تصاحب ۵۱ درصد اتریوم‌های موجود در بین آن‌ها که از لحاظ اقتصادی قابل‌توجه نیست، احتمال وقوع چنین حمله‌ای بسیار کم است.

4-7-2 دسترس‌پذیری

یکی از مهم‌ترین مزایای فناوری بلاکچین، انعطاف‌پذیری ذاتی آن در برابر حملاتی که باهدف ایجاد اختلال در دسترسی به خدمات انجام می‌شود می‌باشد. ماهیت توزیع‌شده و غیرمتمرکز شبکه‌ی بلاکچین، داده‌ها و عملکردها را در گره‌های متعدد پراکنده می‌کند. این افزونگی، شبکه را در برابر حملات انکار سرویس (DoS) بسیار مقاوم می‌کند. برخلاف یک سرور متمرکز که می‌تواند توسط ترافیک بیش از حد تحت فشار قرار گیرد و منجر به در دسترس نبودن آن برای کاربران اصلی شود، در محیط بلاکچین، سایر گره‌ها در شبکه به کار خود ادامه می‌دهند و تداوم در دسترس بودن خدمات را تضمین می‌کنند. در زمینه سامانه پیشنهادی ما، این عملکرد بدون وقفه، باعث می‌شود که حتی در هنگام حملات احتمالی، توابع مهم مانند ضرب توکن، خرید و فروش، و عملیات مرتبط با DAO که سطح دسترسی عمومی ندارند همچنان کار خود را به‌درستی و به‌صورت کامل انجام دهند. این در دسترس بودن مداوم، اعتماد و اطمینان کاربر را نسبت به قابلیت اطمینان سامانه تقویت می‌کند.

4-7-3 مسئولیت‌پذیری و عدم انکار

دفترکل غیرقابل‌تغییر و عمومی که توسط فناوری بلاکچین نگهداری و مدیریت می‌شود، دو ویژگی مهم زیر را برای ما به ارمغان می‌آورد:

- **مسئولیت پذیری یا پاسخگویی:** تمام تراکنش‌ها و تعاملات موجود در سامانه به طور دائم در شبکه اتریوم ثبت می‌شود و یک مسیر حسابرسی غیرقابل انکار را ایجاد می‌کند. این تضمین می‌کند که تمامی اتفاقات شبکه قابل ردیابی و رهگیری هستند و تمامی شرکت‌کنندگان در شبکه را در مقابل اقداماتشان مسئول می‌کند.
- **عدم انکار:** به دلیل ماهیت رمزگذاری نامتقارن استفاده شده در کیف پول‌های شبکه‌ای اتریوم، یک شرکت‌کننده نمی‌تواند به دروغ ادعا کند که فراهوانی یک تابع را با پارامترهای خاص آغاز کرده یا نکرده است، زیرا هر اقدامی از نظر رمزگذاری در دفترکل عمومی قابل تأیید است.

4-7-4 محرمانگی داده‌ها

بلاکچین‌های عمومی مانند اتریوم و بیت‌کوین علی‌رغم اینکه شفافیت و تغییرناپذیری اطلاعات را ارائه می‌کنند، به طور پیش‌فرض محرمانگی اطلاعات را فراهم نمی‌کنند. با این حال، این امر لزوماً به دو دلیل کلیدی زیر مانع پذیرش سامانه پیشنهادی توکنیزه کردن املاک و مستغلات ارائه شده نمی‌گردد:

1. **هویت‌های مستعار:** موجودیت‌ها و تراکنش‌های روی اتریوم با وجود اینکه عمومی هستند، اما از آدرس‌های کیف پول (EOA) که حکم هویت مستعار را دارد استفاده می‌کند و اطلاعاتی که مستقیماً هویت کاربر را آشکار کند را در خود ذخیره نمی‌کند. این نشان می‌دهد که اگرچه خود داده‌ها ممکن است برای عموم قابل مشاهده باشد، ردیابی آن‌ها به یک فرد خاص می‌تواند چالش برانگیز باشد.
2. **رمزگذاری فراداده:** سامانه پیشنهادی از سامانه فایل IPFS برای ذخیره‌سازی داده‌ها، خارج از شبکه‌ی بلاکچین استفاده می‌کند. هر چند این شبکه نیز به‌طور کلی عمومی است و با در دسترس داشتن آدرس (هش) یک فایل می‌توان به آن دسترسی پیدا کرد، می‌توان فایل‌های حساس را قبل از آپلود کردن در IPFS رمزگذاری کرد. این رویکرد نشان می‌دهد که سامانه پیشنهادی کاملاً سازگاری با روش‌های رمزگذاری داده‌ها را دارد و با این کار می‌توان از محرمانگی اطلاعات خاص و حساس، بر حسب نیاز اطمینان حاصل کرد.

4-8 مقایسه با کارهای مشابه

چند مطالعه پتانسیل فناوری بلاکچین را در املاک و مستغلات بررسی کرده‌اند که در بخش 2-7 به چهار مورد از آن‌ها اشاره شد. در جدول 4-4، این کارها و تفاوت‌هایشان به‌صورت خلاصه شرح داده شده است. این بخش سامانه پیشنهادی ما را با موارد مشابه مقایسه کرده و تفاوت‌ها و مشارکت‌هایش را مشخص می‌کند.

جدول 4-4 مقایسه راه‌حل پیشنهادی با مقالات مرتبط

راه‌حل پیشنهادی	[23]	[22]	[21]	[20]		
✓	X	X	✓	✓	توکنیزه کردن	دسته‌بندی
✓	X	X	✓	✓	معاملات	
✓	X	✓	X	X	مدیریت ملک	
✓	✓	X	X	X	مدیریت اراضی	
ERC-721 ERC-1155	ERC-721	ERC-4519	ERC-721 ¹	ERC-777	استاندارد توکن	
Sepolia	Ropsten	Goerli	خصوصی	اتریوم	بلاکچین	
✓	✓	X	X	X	مالکیت اشتراکی	
✓	X	X	X	X	رویکرد ترکیبی برای مالکیت اشتراکی	
✓	X	X	X	X	DAO	
✓	X	✓	X	X	مدیریت اجاره	
✓	X	X	X	X	مدیریت ساختمان	
✓	✓	✓	X	✓	پیاده‌سازی	
✓	X	X	X	X	آزمون و تجزیه و تحلیل آسیب‌پذیری	
✓	✓	✓	X	X	استقرار	

توکنیزه کردن املاک و مستغلات با استفاده از فناوری بلاکچین از Gupta و همکاران [20]

درحالی‌که این مطالعه به بررسی توکنیزه کردن املاک در بلاکچین اتریوم می‌پردازد، از استاندارد ERC-777 که به دلایل امنیتی منسوخ شده است استفاده می‌کند. سامانه پیشنهادی این پایان‌نامه از استانداردهای ایمن‌تر ERC-721 و ERC-1155 استفاده کرده و راه‌حل مقاوم‌تری را ارائه می‌دهد. علاوه بر آن، سامانه‌ی پیشنهادی ما با ترکیب عملکردهایی برای مالکیت اشتراکی و مدیریت آن از طریق DAO، جمع‌آوری اجاره و مدیریت ساختمان، فراتر از توکنیزه کردن اولیه ارائه شده در این مقاله می‌باشد.

توکنیزه کردن املاک و مستغلات از طریق توکن‌های غیرقابل معاوضه از W. Serrano [21]

¹ استاندارد مورد استفاده در این مقاله صرفاً بر اساس ERC-721 بوده و در اصل شخصی‌سازی شده است.

مانند مطالعه قبلی، این مطالعه نیز صرفاً بر روی توکنیزه کردن و معاملات مربوط به آن تمرکز دارد. تفاوت کلیدی این مقاله با کار قبلی این است که از یک بلاکچین خصوصی و یک نسخه اصلاح شده از استاندارد ERC-721 استفاده می کند. استفاده از بلاکچین خصوصی و استاندارد ERC-721 اصلاح شده، مقیاس پذیری، قابلیت همکاری و تعامل با سایر برنامه ها را در مقایسه با راه حل ما که از بلاکچین و استانداردهای مرسوم بدون اعمال محدودیت خاصی استفاده می کند، کاهش می دهد. علاوه بر آن، سامانه ما قابلیت هایی نظیر مالکیت اشتراکی و مدیریت واحدها و اجاره را ارائه می کند که در این مطالعه وجود ندارد.

اجاره‌ی خانه‌ی هوشمند مبتنی بر ERC-4519 از Arcenogui و همکاران [22]

این تحقیق صرفاً به یک جنبه‌ی خاص بلاکچین در املاک و مستغلات، یعنی مدیریت اجاره برای واحدهای تک مالکیتی می پردازد و عملکرد اصلی سامانه ما که توکنیزه کردن املاک و مستغلات به صورت اشتراکی است را در بر نمی گیرد. علاوه بر آن، این کار صرفاً بر روی واحدهای تک مالکیتی NFT از طریق استاندارد ERC-4519 تمرکز دارد؛ بر خلاف سامانه ما که با ارائه مالکیت اشتراکی و مدیریت اجاره‌ی واحدهای اشتراکی و مکانیزم های تقسیم اجاره بین مالکین مشترک، راه حلی جامع تر را ارائه می کند. همچنین این سامانه از شبکه‌ی آزمایشی Goerli که عمدتاً برای آزمون های پرتکلی شبکه و ماینرها استفاده می شود استفاده می کند. شبکه‌ی Sepolia مورد استفاده در کار ما، بر خلاف Goerli گزینه‌ی بهتری برای آزمایش های مربوط به برنامه های کاربردی و قراردادهای هوشمند می باشد [34].

مدیریت اراضی از طریق قراردادهای هوشمند از Stefanovic و همکاران [23]

این تحقیق به دلیل این که مفهوم مالکیت اشتراکی در بستر بلاکچین را بررسی می کند نزدیک ترین همسویی با کار ما را دارد. با این حال، رویکرد اتخاذ شده‌ی آن ها تنها از یک استاندارد توکن جهت توکنیزه کردن دارایی ها استفاده می کند و با رویکرد ترکیبی (چند استاندارد توکن) پیاده سازی شده در سامانه ما تفاوت دارد. این تحقیق مالکیت اشتراکی را با تعریف پارامترهای مرتبط با سهام هر کاربر در قرارداد هوشمند ERC-721 و محدود کردن توابع انتقال آن پیاده سازی کرده است. این رویکرد تعاملات با بازارهای ثانویه را محدود می کند، چرا که مالکان اشتراکی برای انتقال سهام خود، بر خلاف روش ما که از توابع استاندارد ERC-1155 استفاده می کند، باید از توابع شخصی سازی شده‌ی جدید تعریف شده توسط آن ها استفاده کنند. همچنین این پیاده سازی با محدود کردن توابع استاندارد انتقال توکن ERC-721، سطح تعامل کمتری را با بقیه برنامه های کاربردی شبکه‌ی بلاکچین رقم می زند. برخلاف این، سامانه ما یک رویکرد ترکیبی را با استفاده از استانداردهای ERC-721 و ERC-115 برای ارائه مالکیت تکی و اشتراکی پیاده سازی کرده است که این امر تعاملات را با بازارهای ثانویه تسهیل می کند. علاوه بر آن، با وجود اینکه پیاده سازی آن ها مالکیت اشتراکی را صرفاً جهت توکنیزه کردن ارائه می کند، قابلیت های مدیریت واحدهای اشتراکی از طریق

DAO و مدیریت اجاره و تقسیم آن که از اجزای اصلی سامانه ما هستند را شامل نمی‌شود.

در نتیجه، درحالی‌که تحقیقات موجود بینش‌های ارزشمندی را در مورد نقش بلاکچین در املاک و مستغلات ارائه می‌دهند، کار ما یک سامانه کامل‌تر و کاربرمحور را ارائه می‌کند. سامانه پیشنهادی ما با پرداختن به نگرانی‌های امنیتی، ترکیب قابلیت‌های پیشرفته مانند DAO و استفاده از توکن‌های استاندارد مختلف مطابق نیاز، خود را به‌عنوان یک راه‌حل جامع برای آینده توکنیزه‌کردن و مدیریت املاک و مستغلات معرفی می‌کند.

9-4 جمع‌بندی

این فصل ابتدا به طور جامع فرایندهای آزمون و استقرار قراردادهای هوشمند مرتبط با سامانه پیشنهادی املاک و مستغلات را تشریح کرد و سپس یک بررسی جامع از نگرانی‌های امنیتی و وضعیت عملکردی این سامانه از نظر هزینه و زمان را ارائه داد که تمامی این موارد را به‌صورت زیر می‌توان خلاصه کرد:

- **آزمون:** یک مجموعه آزمون جامع با استفاده از ابزارهای استاندارد توسعه آزمون در زبان TypeScript نوشته شد. این آزمون‌ها با استفاده از چارچوب Hardhat اجرا شده و گزارشی از پوشش ۱۰۰ درصدی را ارائه می‌کنند. این موضوع نشان می‌دهد که تمامی کدهای قراردادهای هوشمند به طور کامل آزمایش شده‌اند و می‌توان به عملکرد و صحت اجرای آن‌ها اعتماد کرد.
- **آسیب‌پذیری:** گزارشی جامع از آسیب‌پذیری توسط ابزار Slither ارائه شد. این گزارش نبود آسیب‌پذیری‌هایی با شدت متوسط یا بالاتر را تأیید کرد.
- **استقرار:** قراردادهای هوشمند با استفاده از چارچوب Hardhat در شبکه آزمایشی Sepolia مستقر شدند. برای این کار، از دروازه Infura و کیف پول Metamask و یک اسکریپت سفارشی شده استفاده شد. همچنین آدرس‌های قراردادهای هوشمند مستقر شده در Sepolia نیز ارائه داده شد.
- **هزینه‌ی گاز:** گزارشی کامل از هزینه‌ی اجرای تمامی توابع موجود در قراردادهای هوشمند سامانه و همچنین استقرار آن‌ها در شبکه، توسط ابزار مربوط در چارچوب Hardhat ارائه شد و سپس بر اساس این گزارش توابع مهم مرتبط با کاربران عمومی سامانه و همچنین توابع پرهزینه مرتبط با صاحب ساختمان بررسی شد. این گزارش، امکان‌پذیری عملکردهای مرتبط با توکنیزه‌کردن توسط مشتریان بالقوه‌ی سامانه را از نظر هزینه نشان داد.
- **تأخیر و توان عملیاتی:** در این بخش به میزان تأخیر و توان عملیاتی شبکه‌ی اتریوم اشاره شد و همچنین با توسعه‌ی یک اسکریپت، توان عملیاتی توابع مربوط به توکنیزه کردن و همچنین استقرار قراردادهای هوشمند بر حسب تراکنش بر ثانیه (TPS) اندازه‌گیری و ارائه شد.
- **سایر ملاحظات:** این بخش به بررسی سایر ملاحظات امنیتی مختلف فراتر از تجزیه و تحلیل

آسیب‌پذیری اصلی، از جمله یکپارچگی داده‌ها، محرمانه بودن، دسترس‌پذیری، پاسخگویی و عدم انکار پرداخت و نحوه‌ی رسیدگی به هر مورد را تشریح کرد.

این فصل ارزیابی جامعی از جنبه‌های امنیتی و عملکردی سامانه توکنیزه‌کردن املاک و مستغلات را ارائه و در نهایت پلتفرم پیشنهادی را با سایر کارهای مرتبط گذشته مقایسه کرد. یافته‌ها نشان می‌دهد که این سامانه به‌صورت ایمن، بهینه و کارآمد طراحی شده است و بنابراین مناسب استقرار در دنیای واقعی می‌باشد.

فصل پنجم

بحث و نتیجه گیری

1-5 نتیجه گیری

این بخش به خلاصه‌ای از رویکردهای استفاده‌شده در این تحقیق و یافته‌های کلیدی آن می‌پردازد و نشان می‌دهد که چگونه این نتایج به پرسش تحقیق پاسخ می‌دهند.

بازبینی پرسش و هدف تحقیق

- **پرسش تحقیق:** چگونه می‌توانیم به طور مؤثر یک سامانه مبتنی بر بلاکچین را برای توکنیزه کردن و مدیریت املاک و مستغلات با تضمین امنیت و کارایی و درعین حال دستیابی به اهدافی مانند افزایش نقدشوندگی و دسترسی پیاده‌سازی کنیم؟
- **هدف تحقیق:** هدف اصلی این تحقیق، پیاده‌سازی یک سامانه مدیریت سرمایه‌گذاری املاک و مستغلات بر بستر بلاکچین با استفاده از قرارداد هوشمند و NFT می‌باشد.

رویکردها

به‌منظور پرداختن به اهداف تحقیق و پاسخ به پرسش اصلی آن، این پایان‌نامه از یک رویکرد چندمرحله‌ای شامل مرور ادبیات، توسعه و استقرار سامانه، آزمون دقیق و سخت‌گیرانه و همچنین ارزیابی جامع استفاده کرد. این رویکردها به شرح زیر می‌باشند:

1. **مرور ادبیات:** در ابتدا، برای درک مفاهیم اساسی فناوری بلاکچین که زیربنای اصلی تحقیق بود، یک مرور ادبیات گسترده انجام شد. این بررسی بر روی فناوری بلاکچین، با تأکید ویژه بر اتریوم، قراردادهای هوشمند، توکنیزه کردن و DAO تمرکز داشت. این مرور ادبیات، چارچوب نظری لازم برای مفهوم‌سازی راه‌حل پیشنهادی را به‌گونه‌ای که مؤثر باشد ارائه کرد.
2. **توسعه سامانه:** در ادامه، مرحله طراحی و توسعه‌ی سامانه توکنیزه‌کردن املاک و مستغلات آغاز گردید. در این مرحله، قراردادهای هوشمند مختلفی به زبان برنامه‌نویسی Solidity توسعه داده شد تا به نیازهای مختلف بازیگران سامانه در بستر بلاکچین پاسخ دهد. سپس روش پیاده‌سازی هر قرارداد هوشمند با جزئیات کامل شرح داده شد و در نهایت گردش کار و تعاملات بازیگران مختلف سامانه با قراردادهای هوشمند موجود در آن بررسی گردید.
3. **آزمون‌های دقیق:** پس از پیاده‌سازی سامانه، آزمون‌های دقیقی برای صحت‌سنجی و اطمینان از عملکرد بخش‌های مختلف سامانه انجام داده شد. این مرحله شامل توسعه‌ی آزمون‌های جامع در چارچوب توسعه و همچنین اجرای تجزیه و تحلیل آسیب‌پذیری با استفاده از ابزار مربوطه بود. این آزمایش‌ها در شناسایی و رفع هرگونه مشکل یا آسیب‌پذیری احتمالی در حین فرایندهای پیاده‌سازی برنامه‌های کاربردی مبتنی بر بلاکچین از اهمیت بالایی برخوردار هستند.
4. **استقرار:** پس از صحت‌سنجی و تجزیه و تحلیل آسیب‌پذیری قراردادهای هوشمند و اطمینان از

عملکرد و امنیت آن‌ها، سامانه توسعه داده شده در شبکه‌ی آزمایشی اتریوم برای ارزیابی‌های بیشتر و اطمینان بیشتر از صحت قراردادهای هوشمند مستقر شد.

5. **ارزیابی جامع:** مرحله‌ی نهایی شامل ارزیابی سامانه مستقر شده در معیارهای مختلف عملکردی بود. این ارزیابی بر جنبه‌هایی مانند یکپارچگی و محرمانگی داده‌ها، دسترسی‌پذیری، مسئولیت‌پذیری، مقرون‌به‌صرفه بودن و همچنین تأخیر و توان عملیاتی متمرکز بود. تجزیه و تحلیل هزینه‌های گاز مرتبط با استقرار و تعاملات قراردادهای هوشمند، بینش‌هایی را در مورد کارایی سامانه از لحاظ مقرون‌به‌صرفه بودن ارائه کرد. برآورد تأخیر تراکنش و اندازه‌گیری توان عملیاتی سامانه برای درک عملکرد سامانه در شبکه‌ی بلاکچین اتریوم انجام شد. با به‌کارگیری روش‌های مختلف ارزیابی جامع، اثربخشی و امکان‌سنجی کلی سامانه توکنیزه کردن املاک و مستغلات مبتنی بر بلاکچین و قراردادهای هوشمند نشان داده شد.

یافته‌های کلیدی

این رویکرد چندمرحله‌ای، یک تحقیق جامع را تضمین می‌کند که شامل زمینه‌های نظری، توسعه‌ی کامل، آزمون‌های دقیق و ارزیابی جامع می‌شود که در نهایت، منجر به توسعه‌ی یک سامانه توکنیزه کردن و مدیریت املاک و مستغلات کارآمد و ایمن در بستر بلاکچین به همراه یافته‌های کلیدی زیر گردید:

- **توسعه و استقرار سامانه:** سامانه پیشنهادی به‌صورت کامل در قالب قراردادهای هوشمند به زبان برنامه‌نویسی Solidity توسعه داده شد. قراردادهای هوشمند نوشته شده با این زبان به‌صورت کامل با شبکه‌ی اتریوم و سایر شبکه‌های مبتنی بر EVM سازگار هستند. در هنگام توسعه‌ی این سامانه، از استانداردهای توکنیزه کردن ERC-721 NFT و ERC-1155 استفاده شد. همچنین مدیریت ساختمان و واحدهای اشتراکی از طریق قراردادهای هوشمند دیگری که DAO، مدیریت اجاره و هزینه‌های سرانه‌ی خدمات را شامل می‌شود میسر گردید. در نهایت برای اطمینان از عملکرد سامانه در محیط عملیاتی بلاکچین، قراردادهای هوشمند در شبکه‌ی آزمایشی اتریوم به‌درستی مستقر شدند.

- **پوشش کامل آزمون:** آزمون‌های نوشته‌شده برای بخش‌های سامانه، سطح پوشش کامل را به دست آورد و ما را از صحت عملکرد تمامی بخش‌های سامانه آگاه کرد.

- **بدون آسیب‌پذیری بالقوه:** تجزیه و تحلیل آسیب‌پذیری با استفاده از ابزارهای مربوطه، هیچ آسیب‌پذیری مهم و سطح متوسط به بالایی را شناسایی نکرد.

- **حفظ یکپارچگی و محرمانگی داده‌ها:** این تحقیق به بررسی ملاحظات مربوط به حفظ یکپارچگی و محرمانگی داده‌ها در چارچوب توکنیزه کردن دارایی‌ها با استفاده از قراردادهای هوشمند پرداخت

و در مورد هر کدام به اطمینان مورد نظر رسید.

- **کارایی هزینه:** تجزیه و تحلیل هزینه‌های مرتبط با استقرار و فراخوانی توابع قرارداد هوشمند نشان داد که این سامانه راه‌حلی عملی و امکان‌پذیر را برای بازیگران مختلف از لحاظ هزینه ارائه می‌کند.
- **کارایی توان عملیاتی:** ارزیابی معیارهای تأخیر و توان عملیاتی سامانه پیشنهادی در شبکه‌ی اتریوم، سطوح عملکرد رضایت‌بخشی را بیان کرد که نشان‌دهنده مقیاس‌پذیری و کارایی این سامانه می‌باشد. با حفظ تأخیر تراکنش در محدوده قابل قبول و توان عملیاتی که پاسخگوی سناریوهای مورد استفاده در دنیای واقعی است، سامانه می‌تواند حجم بالایی از تراکنش‌ها را بدون به خطر انداختن سرعت یا قابلیت اطمینان در شبکه‌ی اتریوم جای دهد.

اثر بخشی سامانه پیشنهادی

یافته‌ها نشان می‌دهند که سامانه توکنیزه کردن و مدیریت املاک و مستغلات پیشنهادی به طور مؤثر با دستیابی به موارد زیر به سؤال تحقیق می‌پردازد:

- **امنیت:** یک رویکرد چندجانبه برای اطمینان از امنیت سامانه اجرا شد. آزمون‌های نوشته شده برای بخش‌های سامانه، سطح پوشش کامل را به دست آورد. همچنین تجزیه و تحلیل آسیب‌پذیری با استفاده از ابزارهای مربوطه، هیچ آسیب‌پذیری مهم و سطح متوسط به بالایی را شناسایی نکرد. همچنین این سامانه سایر ملاحظات را برای حفظ یک‌پارچگی و محرمانگی داده‌ها را بررسی کرد.
- **کارایی:** تجزیه و تحلیل هزینه‌های مرتبط با استقرار و فراخوانی توابع قرارداد هوشمند به همراه بررسی توان عملیاتی شبکه‌ی اتریوم نشان داد که این سامانه راه‌حلی عملی و امکان‌پذیر را برای بازیگران مختلف سامانه ارائه می‌کند.
- **نقدشوندگی و دسترسی:** با ارائه مالکیت اشتراکی از طریق توکنیزه کردن بر اساس استانداردهای موجود، این سامانه می‌تواند به طور بالقوه نقدشوندگی را در بازار املاک و مستغلات افزایش دهد و با کاهش موانع سرمایه‌گذاری برای سرمایه‌گذاران خرد، دسترسی آسان‌تر به فرصت‌های سرمایه‌گذاری را تسهیل کند.

باتوجه به یافته‌های این پایان‌نامه، می‌توان نتیجه گرفت که رویکرد ترکیبی چند توکنه ارائه شده در این پایان‌نامه، پتانسیل قابل توجهی برای تحول بازار سستی املاک و مستغلات دارد. این رویکرد می‌تواند به سرمایه‌گذاران فرصت‌های جدیدی برای سرمایه‌گذاری در املاک و مستغلات ارائه دهد و به مالکان و مدیران املاک کمک کند تا فرایندهای خود را به طور کارآمدتر و شفاف‌تر مدیریت کنند. این یافته‌ها و نتایج، در راستای علاقه روزافزون به فناوری بلاکچین برای کاربردهای املاک و مستغلات می‌باشد و مزایای بالقوه توکنیزه کردن را هم برای سرمایه‌گذاران و هم برای مالکان نشان می‌دهد.

2-5 محدودیت‌ها

این پایان‌نامه، محدودیت‌های زیر را در ارتباط با تحقیق حاضر به رسمیت می‌شناسد:

چالش‌های حقوقی و عدم قطعیت قانونی

یکی از محدودیت‌های مهم، فقدان مقررات قانونی پیرامون توکنیزه کردن و استفاده از فناوری بلاکچین در حوزه‌های مختلف می‌باشد. نبود مقررات و چارچوب‌های نظارتی روشن ممکن است موانعی را برای پذیرش گسترده‌ی راه‌حل‌های مبتنی بر بلاکچین در صنعت املاک و مستغلات ایجاد کند. بدون شفافیت قانونی کافی، ذی‌نفعان ممکن است از مشارکت در معاملات تسهیل شده توسط سامانه ما تردید کنند و این امر مانع پذیرش سامانه در جامعه کنونی شود.

تغییرناپذیری بلاکچین

یکی از محدودیت‌های ذاتی فناوری بلاکچین، تغییرناپذیری کدهای مستقر شده بر روی آن می‌باشد. درحالی‌که این ویژگی یکپارچگی و امنیت تراکنش‌ها را تضمین می‌کند و به‌عنوان ویژگی مثبت بلاکچین تلقی می‌شود، توانایی به‌روزرسانی یا اصلاح قراردادهای هوشمند را پس از استقرار محدود می‌کند. در نتیجه، باوجوداینکه اقدامات مرتبط و آزمون‌های دقیق برای به‌حداقل رساندن مشکلات احتمالی انجام گردیده است، هرگونه اشکال یا نقصی که پس از استقرار کشف شود، بدون استقرار یک قرارداد هوشمند جدید قابل اصلاح نمی‌باشد که این امر که به طور بالقوه می‌تواند باعث اختلال در عملیات جاری در قرارداد هوشمند گردد. اگرچه قراردادهای هوشمند قابل ارتقا¹ راه‌حلی را برای این محدودیت ارائه می‌دهند، اما به دلیل اولویت قراردادن سادگی در طراحی سامانه، در این تحقیق از این ویژگی صرف‌نظر شده است.

نوسانات نرخ گاز

همان‌طور که در بخش 4-5 ذکر شد، هزینه‌ی کارمزدهای مرتبط با تراکنش‌های شبکه‌ی اتریوم که به آن گاز می‌گویند، بر اساس ترافیک شبکه ممکن است نوسانات قابل توجهی داشته باشد. درحالی‌که بررسی ارائه شده در این تحقیق نشان می‌دهد که هزینه‌های مرتبط با استقرار و فراخوانی توابع قراردادهای هوشمند این سامانه در محدوده‌ی قابل قبولی است، نوسانات آن در آینده می‌تواند در مقرون به صرفه بودن کلی سامانه در سناریوهای دنیای واقعی تأثیر گذارد.

علی‌رغم این محدودیت‌ها، تحقیق ارائه شده بینش ارزشمندی را در مورد پتانسیل فناوری بلاکچین برای متحول کردن صنعت املاک و مستغلات ارائه می‌دهد. با تصدیق و پرداختن به این چالش‌ها، ذی‌نفعان می‌توانند اقدامات لازم را برای کاهش خطرات و به حداکثر رساندن مزایای اجرای راه‌حل‌های مبتنی بر

¹ Upgradeable Smart Contracts

بلاکچین در صنعت املاک و مستغلات انجام دهند. همکاری مستمر بین ذی‌نفعان صنعت املاک و مستغلات، قانون‌گذاران و کارشناسان بلاکچین برای غلبه بر محدودیت‌های ذکرشده و اعمال پتانسیل حداکثری آن در این صنعت ضروری است.

3-5-3 پیشنهاداتی برای مطالعات آتی

یافته‌های این پایان‌نامه، بیانگر پتانسیل توسعه‌های دیگر در حوزه‌ی توکنیزه‌کردن املاک و مستغلات می‌باشد. در زیر، برخی از این مناطق برای کاوش و تحقیق بیشتر ارائه شده است:

- **ادغام با بازارهای غیرمتمرکز ثانویه:** تمرکز فعلی سامانه ارائه شده بر روی جنبه‌های توکنیزه‌کردن و مدیریت واحدها بود. باتوجه‌به این که این سامانه از توکن‌های استاندارد استفاده کرده است، کار آینده می‌تواند امکان ادغام یکپارچه با بازارهای غیرمتمرکز ثانویه را باهدف تسهیل انتقال واحدهای املاک و مستغلات بررسی کند. چنین ادغامی منجر به ایجاد یک اکوسیستم جامع سرمایه‌گذاری در املاک و مستغلات در چشم‌انداز بلاکچین می‌شود.
 - **تطابق با قوانین و مقررات:** از آنجایی که فناوری بلاکچین همواره به تکامل خود ادامه می‌دهد، نظارت بر تغییرات قوانین و مقررات حاکم بر نقش بلاکچین و توکنیزه‌کردن دارایی‌ها ضروری می‌باشد. تحقیقات آینده می‌تواند بر کشف راه‌حلی برای اطمینان از انطباق با مقررات در حال ظهور در این حوزه و سازگاری سامانه پیشنهادی بر اساس آن‌ها تمرکز کند.
 - **ارتقا DAO:** پیاده‌سازی DAO کنونی می‌تواند بیشتر توسعه یابد تا فرایندهای پیچیده‌تر رأی‌گیری، ساختارهای مدیریتی و مکانیزم‌های حل اختلاف را در بر گیرد. اجرای این پیشرفت‌ها می‌تواند منجر به ایجاد یک چارچوب DAO کامل‌تر و کاربرمحورتر در سامانه شود که به نوبه خود باعث تقویت مشارکت و تصمیم‌گیری‌های جمعی می‌گردد.
 - **تعمیم معماری سامانه:** تحقیقات آینده می‌تواند پتانسیل تعمیم معماری توکنیزه‌کردن این سامانه و مالکیت اشتراکی را برای تطبیق با طیف وسیع‌تری از دارایی‌های مختلف، فراتر از املاک و مستغلات، بررسی کند. این دارایی‌ها می‌توانند شامل دارایی‌های باارزش مانند طلا و جواهرات و یا اسناد مرتبط با خودرو و یا آثار هنری یا هر موردی که بتوان آن را توکنیزه‌کرد شوند.
- با پرداختن به این حوزه‌ها در مطالعات آینده، سامانه توکنیزه‌کردن املاک و مستغلات پیشنهادی می‌تواند به یک راه‌حل مقاوم و مطمئن تبدیل شود که نوآوری و کارایی بلاکچین در صنعت املاک و مستغلات را نشان می‌دهد و پتانسیل آن برای کاربردهای گسترده‌تر در توکنیزه‌کردن سایر دارایی‌ها را تقویت می‌کند.

4-5 جمع‌بندی

در پایان، این پایان‌نامه یک پیاده‌سازی و بررسی جامع از توکنیزه‌کردن املاک و مستغلات با استفاده از

توکن‌های غیرقابل معاوضه (NFT) و نیمه‌قابل معاوضه و فناوری‌های غیرمتمرکز از جمله قراردادهای هوشمند، IPFS و بلاکچین اتریوم ارائه کرد و با توسعه یک سامانه جدید، نشان داد که بلاکچین چگونه می‌تواند صنعت املاک و مستغلات را متحول کند. سامانه پیاده‌سازی شده با به‌کارگیری قراردادهای هوشمند، توکنیزه کردن از طریق رویکرد ترکیبی توکن‌های غیرقابل معاوضه و نیمه قابل معاوضه، مدیریت جمعی غیرمتمرکز با سازمان‌های غیرمتمرکز خودگردان (DAO) و انجام آزمون‌ها و بررسی‌های امنیتی دقیق قبل از استقرار، راه‌حلی ایمن، شفاف، غیرمتمرکز و کارآمد را برای مالکیت جزئی و مدیریت اجاره و هزینه‌های ساختمان در بستر بلاکچین ارائه می‌دهد. درحالی‌که عدم قطعیت‌های قانونی و نوسانات قیمت گاز همچنان چالش برانگیز است، این تحقیق بینش‌های ارزشمندی را در مورد کاربرد بلاکچین برای املاک و مستغلات ارائه می‌کند. پیشرفت‌های آینده می‌توانند این محدودیت‌ها را برطرف کنند و راه را برای بازار املاک و مستغلات فراگیرتر و دردسترس‌تر از طریق پذیرش گسترده‌تر فناوری بلاکچین هموار کنند.

منابع و مأخذ

- [1] M. Ellington, X. Fu, and Yunyi, “Real Estate Illiquidity and Returns: A Time-varying Regional Perspective.” Rochester, NY, Apr. 22, 2020. doi: 10.2139/ssrn.3396571.
- [2] E. Ionascu, M. Marilena, and I. Anghel, “Transparency of Real Estate Markets: Conceptual and Empirical Evidence,” *Audit Financ.*, vol. 17, May 2019, doi: 10.20869/AUDITF/2019/154/013.
- [3] “7 Problems That Trip Up Real Estate Investors & How To Avoid Them.” Accessed: Mar. 06, 2023. [Online]. Available: <https://investinganswers.com/articles/7-problems-trip-real-estate-investors-and-how-avoid-them>
- [4] N. Kalyuzhnova, “Transformation of the real estate market on the basis of use of the blockchain technologies: opportunities and problems,” *MATEC Web Conf.*, vol. 212, p. 06004, 2018, doi: 10.1051/mateconf/201821206004.
- [5] G. Sazandrishvili, “Asset tokenization in plain English,” *J. Corp. Account. Finance*, vol. 31, no. 2, pp. 68–73, 2020, doi: 10.1002/jcaf.22432.
- [6] A. Saari, J. Vimpari, and S. Junnila, “Blockchain in real estate: Recent developments and empirical applications,” *Land Use Policy*, vol. 121, p. 106334, Oct. 2022, doi: 10.1016/j.landusepol.2022.106334.
- [7] “Bitcoin: A Peer-to-Peer Electronic Cash System.” Accessed: Mar. 05, 2023. [Online]. Available: <https://bitcoin.org/en/bitcoin-paper>
- [8] J. Abou Jaoude and R. George Saade, “Blockchain Applications – Usage in Different Domains,” *IEEE Access*, vol. 7, pp. 45360–45381, 2019, doi: 10.1109/ACCESS.2019.2902501.
- [9] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger - Edition: BERLIN VERSION 2bcd2d – 2023-08-25,” *Ethereum Proj. Yellow Pap.*, Aug. 2023, Accessed: Nov. 12, 2023. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [10] “Ethereum Glossary,” [ethereum.org](https://ethereum.org/en/glossary). Accessed: Nov. 12, 2023. [Online]. Available: <https://ethereum.org/en/glossary>
- [11] “Ethereum development documentation,” ethereum.org. Accessed: Nov. 15, 2023. [Online]. Available: <https://ethereum.org>
- [12] A. Altarawneh, T. Herschberg, S. Medury, F. Kandah, and A. Skjellum, “Buterin’s Scalability Trilemma viewed through a State-change-based Classification for Common Consensus Algorithms,” in *2020 10th Annual Computing and Communication Workshop and Conference (CCWC)*, Jan. 2020, pp. 0727–0736. doi: 10.1109/CCWC47524.2020.9031204.
- [13] “Ethereum,” ethereum.org. Accessed: Nov. 18, 2023. [Online]. Available: <https://ethereum.org>
- [14] C. Pinzón and C. Rocha, “Double-spend Attack Models with Time Advantage for Bitcoin,” *Electron. Notes Theor. Comput. Sci.*, vol. 329, pp. 79–103, Dec. 2016, doi: 10.1016/j.entcs.2016.12.006.
- [15] V. Buterin, “Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform,” *Ethereum Whitepaper*. Accessed: Mar. 06, 2023. [Online]. Available: <https://ethereum.org/whitepaper/>
- [16] “Solidity 0.8.23 documentation.” Accessed: Nov. 18, 2023. [Online]. Available:

<https://docs.soliditylang.org/en/v0.8.23/>

- [17] W. Entriken, D. Shirley, J. Evans, and N. Sachs, “‘ERC-721: Non-Fungible Token Standard,’ Ethereum Improvement Proposals, no. 721, January 2018,” Ethereum Improvement Proposals. Accessed: Nov. 26, 2023. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-721>
- [18] S. B. Far, S. M. H. Bamakan, Q. Qu, and Q. Jiang, “A Review of Non-fungible Tokens Applications in the Real-world and Metaverse,” *Procedia Comput. Sci.*, vol. 214, pp. 755–762, Jan. 2022, doi: 10.1016/j.procs.2022.11.238.
- [19] S. Wang, W. Ding, J. Li, Y. Yuan, L. Ouyang, and F.-Y. Wang, “Decentralized Autonomous Organizations: Concept, Model, and Applications,” *IEEE Trans. Comput. Soc. Syst.*, vol. 6, no. 5, pp. 870–878, Oct. 2019, doi: 10.1109/TCSS.2019.2938190.
- [20] A. Gupta, J. Rathod, D. Patel, J. Bothra, S. Shanbhag, and T. Bhalerao, “Tokenization of Real Estate Using Blockchain Technology,” in *Applied Cryptography and Network Security Workshops*, J. Zhou, M. Conti, C. M. Ahmed, M. H. Au, L. Batina, Z. Li, J. Lin, E. Losiouk, B. Luo, S. Majumdar, W. Meng, M. Ochoa, S. Picek, G. Portokalidis, C. Wang, and K. Zhang, Eds., in Lecture Notes in Computer Science. Cham: Springer International Publishing, 2020, pp. 77–90. doi: 10.1007/978-3-030-61638-0_5.
- [21] W. Serrano, “Real Estate Tokenisation via Non Fungible Tokens,” in *The 2022 4th International Conference on Blockchain Technology*, in ICBCT’22. New York, NY, USA: Association for Computing Machinery, Jul. 2022, pp. 81–87. doi: 10.1145/3532640.3532651.
- [22] J. Arcenegui, R. Arjona, and I. Baturone, “Non-Fungible Tokens Based on ERC-4519 for the Rental of Smart Homes,” *Sensors*, vol. 23, no. 16, Art. no. 16, Jan. 2023, doi: 10.3390/s23167101.
- [23] M. Stefanović, Đ. Pržulj, S. Ristić, D. Stefanović, and D. Nikolić, “Smart Contract Application for Managing Land Administration System Transactions,” *IEEE Access*, vol. 10, pp. 39154–39176, 2022, doi: 10.1109/ACCESS.2022.3164444.
- [24] “Ropsten, Rinkeby & Kiln Deprecation Announcement,” Ethereum Foundation Blog. Accessed: Dec. 04, 2023. [Online]. Available: <https://blog.ethereum.org/2022/06/21/testnet-deprecation>
- [25] A. Saari, S. Junnila, and J. Vimpri, “Blockchain’s Grand Promise for the Real Estate Sector: A Systematic Review,” *Appl. Sci.*, vol. 12, no. 23, Art. no. 23, Jan. 2022, doi: 10.3390/app122311940.
- [26] “Deprecate ERC777 implementation · Issue #2620 · OpenZeppelin/openzeppelin-contracts,” GitHub. Accessed: Dec. 04, 2023. [Online]. Available: <https://github.com/OpenZeppelin/openzeppelin-contracts/issues/2620>
- [27] “Hardhat | Ethereum development environment for professionals by Nomic Foundation.” Accessed: Dec. 12, 2023. [Online]. Available: <https://hardhat.org>
- [28] “ERC 721 - OpenZeppelin Docs.” Accessed: Feb. 03, 2024. [Online]. Available: <https://docs.openzeppelin.com/contracts/5.x/api/token/erc721#IERC721>
- [29] “ERC 1155 - OpenZeppelin Docs.” Accessed: Feb. 17, 2024. [Online]. Available: <https://docs.openzeppelin.com/contracts/5.x/api/token/erc1155#IERC1155>
- [30] “ERC-1155: Multi Token Standard,” Ethereum Improvement Proposals. Accessed: Feb. 17, 2024. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-1155>

- [31] “Timestamp Dependence - Ethereum Smart Contract Best Practices.” Accessed: Apr. 21, 2024. [Online]. Available: <https://consensys.github.io/smart-contract-best-practices/development-recommendations/solidity-specific/timestamp-dependence/>
- [32] “Ethereum Average Block Time Daily Trends: Ethereum Statistics | YCharts.” Accessed: Apr. 25, 2024. [Online]. Available: https://ycharts.com/indicators/ethereum_average_block_time
- [33] Chainspect, “Chainspect: Ethereum [TPS, Max TPS, Block Time & TTF],” Chainspect. Accessed: Apr. 25, 2024. [Online]. Available: <https://chainspect.app/chain/ethereum>
- [34] “Ethereum Networks,” ethereum.org. Accessed: May 09, 2024. [Online]. Available: <https://ethereum.org/en/developers/docs/networks/>

پیوست‌ها

پیوست الف: کدهای قرارداد هوشمند FractoRealFractions.sol یا FRF

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721Enumerable.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
import "@openzeppelin/contracts/utils/cryptography/MessageHashUtils.sol";
import "@openzeppelin/contracts/utils/Arrays.sol";
import "@openzeppelin/contracts/utils/Address.sol";

import "../FractoRealFractions.sol";
import "../RentManagement.sol";

/// Phase one or two has not started yet.
error PhaseSaleNotStarted();
/// Phase one or two has ended.
error PhaseSaleEnded();
/// Invalid signer, you should use our website for minting.
error InvalidSigner();
/// Invalid ETH has been sended.
error InvalidETH();
/// Withdraw failed.
error WithdrawFailed();
/// Contracts are not allowed to mint.
error ContractMintNotAllowed();
/// Lenght of ids and metrages are not equal.
error LenghtMismatch();
/// ERC1155 address is not set.
error ERC1155AddressNotSet();

/// @custom:security-contact ahbanavi@gmail.com
contract FractoRealNFT is ERC721, ERC721Enumerable, Ownable, RentManagement {
    using ECDSA for bytes32;
    using MessageHashUtils for bytes32;
    using Arrays for uint256[];
    using Address for address payable;

    FractoRealFractions public erc1155;

    string private _baseTokenURI;
    uint256 public immutable MAX_SUPPLY;

    uint256 public phaseOneStartTime = type(uint256).max;
    uint256 public phaseTwoStartTime = type(uint256).max;

    mapping(uint256 tokenId => uint256) public meterages;

    event phaseOneStartTimeSet(uint256 startTime);
    event phaseTwoStartTimeSet(uint256 startTime);
    event phaseTwoStarted();

    constructor(
        address initialOwner,
        uint256 maxSupply_
    ) ERC721("FractoRealNFT", "FNT") Ownable(initialOwner) {
        MAX_SUPPLY = maxSupply_;
    }

    /**
     * @dev Sets the meterages for multiple NFTs.
     * @param ids The array of NFT IDs.
     * @param metrages_ The array of meterages corresponding to the NFT IDs.
     * Requirements:
     * - The `ids` and `metrages` arrays must have the same length.
     * - Only the contract owner can call this function.
     */
    function setMeterages(
        uint256[] memory ids,
        uint256[] memory metrages_
    ) public onlyOwner {
        uint256 length = ids.length;

        if (length != metrages_.length) revert LenghtMismatch();
    }
}
```

```

        unchecked {
            for (uint256 i; i < length; ++i) {
                meterages[ids.unsafeMemoryAccess(i)] = meterages_
                    .unsafeMemoryAccess(i);
            }
        }
    }

    modifier noContract() {
        if (tx.origin != msg.sender) revert ContractMintNotAllowed();
        _;
    }

    function mint(address to, uint256 tokenId) public onlyOwner {
        _mint(to, tokenId);
    }

    function batchMint(address to, uint256[] memory ids) public onlyOwner {
        uint256 length = ids.length;

        for (uint256 i; i < length; ) {
            _mint(to, ids.unsafeMemoryAccess(i));

            unchecked {
                ++i;
            }
        }
    }

    /**
     * @dev Function to mint a token during phase one of the sale.
     * This function checks if the phase one sale has started and if the caller is not a
    contract.
     * It then checks if the signature is valid and if the price paid is correct.
     * If all checks pass, the token is minted and assigned to the caller.
     *
     * @param signature The signature of the message.
     * @param tokenId The ID of the token to be minted.
     * @param priceToPay The price in wei to be paid for the token.
     */
    function phaseOneMint(
        bytes calldata signature,
        uint256 tokenId,
        uint256 priceToPay
    ) external payable noContract {
        if (block.timestamp < phaseOneStartTime) revert PhaseSaleNotStarted();
        if (block.timestamp >= phaseTwoStartTime) revert PhaseSaleEnded();

        // we check for price before signature check to save gas
        if (msg.value != priceToPay) revert InvalidETH();

        // hash is based of msg.sender, contract address, tokenId and priceToPay
        // recover signer from signature
        if (
            keccak256(
                abi.encodePacked(msg.sender, address(this), tokenId, priceToPay)
            ).toEthSignedMessageHash().recover(signature) != owner()
        ) revert InvalidSigner();

        // everything is ok, mint the token
        _mint(msg.sender, tokenId);
    }

    function setErc1155Address(
        FractoRealFractions erc1155Address
    ) public onlyOwner {
        erc1155 = erc1155Address;
    }

    /**
     * @dev Starts phase two of the minting process.
     * This function checks if the phase two sale has started and if the ERC1155 address is
    set.
     * It then mints the remaining tokens that have not been minted yet and assigns them to
    the ERC1155 contract.
     * The token IDs and their corresponding meterages are stored in separate arrays.

```

```

    * Finally, it calls the `mintBatch` function of the ERC1155 contract to mint the
    tokens in a batch.
    * Emits a `phaseTwoStarted` event after the minting process is complete.
    */
    function startPhaseTwoMint() public {
        // Check if phase two sale has started
        if (block.timestamp < phaseTwoStartTime) revert PhaseSaleNotStarted();
        // Check if ERC1155 address is set
        if (address(erc1155) == address(0)) revert ERC1155AddressNotSet();

        // Calculate the number of remaining tokens to be minted
        uint256 arraySize = MAX_SUPPLY - totalSupply();

        // Create arrays to store the unminted tokens and their meterages
        uint256[] memory unmintedTokens = new uint256[](arraySize);
        uint256[] memory unmintedTokensMeteres = new uint256[](arraySize);

        uint256 counter = 0;
        // Iterate through the token IDs to find the unminted tokens
        for (uint256 tokenId_ ; tokenId_ != MAX_SUPPLY; ) {
            // Check if the token is not owned by anyone
            if (_ownerOf(tokenId_) == address(0)) {
                // Mint the token and assign it to the ERC1155 contract
                _safeMint(address(erc1155), tokenId_);
                // Store the token ID and its meterage in the arrays
                unmintedTokens[counter] = tokenId_;
                unmintedTokensMeteres[counter] = meterages[tokenId_];

                unchecked {
                    ++counter;
                }
            }

            unchecked {
                ++tokenId_;
            }
        }

        // Mint the unminted tokens in a batch using the ERC1155 contract
        erc1155.mintBatch(owner(), unmintedTokens, unmintedTokensMeteres, "");

        // Emit an event to indicate that phase two has started
        emit phaseTwoStarted();
    }

    /**
     * @dev Fractionize a token by transferring it to the ERC1155 contract and minting a
     new token with fractional ownership.
     * @param from The address of the token owner.
     * @param tokenId The ID of the token to be fractionized.
     */
    function fractionize(address from, uint256 tokenId_) public {
        // First transfer to erc1155
        // If the TX sender is not the owner or approved by the owner, then TX revert here.
        // Also, if the erc1155 address is not set, this function reverts with an invalid
        receiver error.
        safeTransferFrom(from, address(erc1155), tokenId_);

        // If the previous owner of tokenId wasn't `from`, tx reverted in the last function
        erc1155.mint(from, tokenId_, meterages[tokenId_], "");
    }

    // Contract time setting
    function setPhaseOneStartTime(uint256 startTime_) external onlyOwner {
        phaseOneStartTime = startTime_;
        emit phaseOneStartTimeSet(startTime_);
    }

    function setPhaseTwoStartTime(uint256 startTime_) external onlyOwner {
        phaseTwoStartTime = startTime_;
        emit phaseTwoStartTimeSet(startTime_);
    }

    /// Withdraw funds
    function withdraw() external onlyOwner {
        payable(msg.sender).sendValue(address(this).balance);
    }

```

```

    /// Set base token URI
    function setBaseURI(string calldata baseURI) external onlyOwner {
        _baseTokenURI = baseURI;
    }

    function _baseURI() internal view override returns (string memory) {
        return _baseTokenURI;
    }

    // The following functions are overrides required by Solidity.

    function _update(
        address to,
        uint256 tokenId,
        address auth
    ) internal override(ERC721, ERC721Enumerable) returns (address) {
        return super._update(to, tokenId, auth);
    }

    function increaseBalance(
        address account,
        uint128 value
    ) internal override(ERC721, ERC721Enumerable) {
        super._increaseBalance(account, value);
    }

    function supportsInterface(
        bytes4 interfaceId
    ) public view override(ERC721, ERC721Enumerable) returns (bool) {
        return super.supportsInterface(interfaceId);
    }
}

```

پوست ب: کدهای قرارداد هوشمند RM یا RentManagement.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/utils/Address.sol";

abstract contract RentManagement is ERC721 {
    using Address for address payable;
    /// Only residents
    error OnlyResidents();
    /// Invalid rent amount
    error InvalidRentAmount();
    /// Rent amount to withdraw must be greater than zero
    error InvalidRentAmountToWithdraw();
    mapping(uint256 tokenId => uint256 rent fee) public rentsFee;
    mapping(uint256 tokenId => address resident) public residents;
    mapping(uint256 tokenId => uint256 amount) public paidRents;
    // mapping for keep track of balances to withdraw
    mapping(uint256 tokenId => uint256 amount) public rents;

    event RentPaid(uint256 indexed tokenId, address payer, uint256 amount);
    event RentWithdrawn(uint256 indexed tokenId, address to, uint256 amount);
    event RentFeeSet(uint256 indexed tokenId, uint256 amount);
    event ResidentSet(uint256 indexed tokenId, address resident);

    modifier onlyAuthorized(uint256 tokenId) {
        _checkAuthorized(_ownerOf(tokenId), msg.sender, tokenId);
        _;
    }

    modifier onlyResidents(uint256 tokenId) {
        if (residents[tokenId] != msg.sender) revert OnlyResidents();
        _;
    }

    function payRent(uint256 tokenId) external payable onlyResidents(tokenId) {
        if (msg.value != rentsFee[tokenId]) revert InvalidRentAmount();

        paidRents[tokenId] += msg.value;
        rents[tokenId] += msg.value;
        emit RentPaid(tokenId, msg.sender, msg.value);
    }

    function withdrawRent(
        uint256 tokenId
    ) external onlyAuthorized(tokenId) returns (uint256) {
        uint256 rentAmount = rents[tokenId];
        if (rentAmount == 0) revert InvalidRentAmountToWithdraw();

        // re-entrancy protection
        rents[tokenId] = 0;
        emit RentWithdrawn(tokenId, msg.sender, rentAmount);
        // because rents is set to zero before, we are safe from reentrancy attack
        payable(msg.sender).sendValue(rentAmount);
        return rentAmount;
    }

    function setRentFee(
        uint256 tokenId,
        uint256 rentAmount
    ) external onlyAuthorized(tokenId) {
        rentsFee[tokenId] = rentAmount;
        emit RentFeeSet(tokenId, rentAmount);
    }

    function setResident(
        uint256 tokenId_,
        address resident
    ) public onlyAuthorized(tokenId_) {
        residents[tokenId_] = resident;
        emit ResidentSet(tokenId_, resident);
    }
}
```

پیوست پ: کدهای قرارداد هوشمند FractoRealFractions.sol یا FRF

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/token/ERC1155/extensions/ERC1155Supply.sol";
import "@openzeppelin/contracts/token/ERC721/utils/ERC721Holder.sol";
import "@openzeppelin/contracts/utils/cryptography/EIP712.sol";
import "@openzeppelin/contracts/utils/Address.sol";

import "../FractoRealNFT.sol";
import "../FractionsDAO.sol";

/// Only ERC721 contract is allowed to mint.
error OnlyERC721Allowed();
/// Only owner or ERC721 contract is allowed to mint.
error OnlyOwnerOrERC721Allowed();
/// Owner does not own all tokens of this id
error OwnerDoesNotOwnAllTokens();
/// Token id is not set.
error TokenIdNotSet();

/// @custom:security-contact ahbanavi@gmail.com
contract FractoRealFractions is
    ERC1155,
    ERC1155Supply,
    Ownable,
    ERC721Holder,
    EIP712,
    FractionsDAO
{
    using Address for address payable;

    event Received(address from, uint256 amount);
    event RentSplited(uint256 indexed tokenId, uint256 rentAmount);
    event RentWithdrawn(
        uint256 indexed tokenId,
        address indexed to,
        uint256 rentAmount
    );

    FractoRealNFT public immutable ERC721;
    uint256 public nonSharesRents;

    struct ShareHolders {
        address holder;
        uint256 share;
        uint256 rents;
    }

    mapping(uint256 tokenId => ShareHolders[]) private tokenIdShareHolders;
    mapping(uint256 tokenId => mapping(address holder => uint256 index))
        private tokenIdShareHoldersIndex;

    constructor(
        address initialOwner,
        FractoRealNFT ERC721_
    ) ERC1155("") Ownable(initialOwner) EIP712("FractoRealFractions", "1") {
        ERC721 = ERC721_;
    }

    modifier onlyERC721() {
        if (msg.sender != address(ERC721)) revert OnlyERC721Allowed();
        _;
    }

    function setURI(string memory newuri) public onlyOwner {
        _setURI(newuri);
    }

    function mint(
        address to,
        uint256 id,
        uint256 amount,

```

```

        bytes memory data
    ) public onlyERC721 {
        _mint(to, id, amount, data);
    }

    function mintBatch(
        address to,
        uint256[] memory ids,
        uint256[] memory amounts,
        bytes memory data
    ) public onlyERC721 {
        _mintBatch(to, ids, amounts, data);
    }

    /**
     * @dev Rebuilds an NFT by burning all tokens of a given ID owned by the caller and
    transferring the ownership of the ERC721 token to the caller.
     * @param tokenId The ID of the NFT to be rebuilt.
     * @notice This function can only be called by the owner of all tokens of the given ID.
     * @notice The total supply of the given ID must be set.
     * @notice The caller must have ownership of all tokens of the given ID.
    */
    function rebuildNFT(uint256 tokenId) public {
        uint256 tokenIdTotalSupply = totalSupply(tokenId);

        // Check if the token ID is set
        if (tokenIdTotalSupply == 0) revert TokenIdNotSet();

        // Check if the caller owns all tokens of the given ID
        if (balanceOf(msg.sender, tokenId) != tokenIdTotalSupply)
            revert OwnerDoesNotOwnAllTokens();

        // Burn all tokens of the given ID owned by the caller, that means all tokens of
    the given ID will be burned
        _burn(msg.sender, tokenId, tokenIdTotalSupply);

        // Transfer the ownership of the ERC721 token to the caller
        erc721.safeTransferFrom(address(this), msg.sender, tokenId);
    }

    function getShareHolderInfo(
        uint256 tokenId,
        address holder
    ) external view returns (ShareHolders memory) {
        uint256 index = tokenIdShareHoldersIndex[tokenId][holder];
        if (index == 0) {
            // return zero
            return ShareHolders(address(0), 0, 0);
        }

        return tokenIdShareHolders[tokenId][index - 1];
    }

    /**
     * Splits the rent of a token among its share holders.
     * @param tokenId The ID of the token.
    */
    function splitRent(uint256 tokenId) external {
        // call withdrawRent function of erc721 contract and get the rent amount
        // if the rent amount is zero, it will revert
        uint256 rentAmount = erc721.withdrawRent(tokenId);

        uint256 totalShares = totalSupply(tokenId);

        uint256 sharedRents = 0;
        // for each token owner, calculate the rent amount based on their share
        for (uint256 i = 0; i < tokenIdShareHolders[tokenId].length; i++) {
            uint256 share = tokenIdShareHolders[tokenId][i].share;
            uint256 rent = (share * rentAmount) / totalShares;
            sharedRents += rent;

            // increase the rents of the token owner
            tokenIdShareHolders[tokenId][i].rents += rent;
        }

        // If the total rents shares of token owners is not equal to the total shares of
    the token,

```

```

        // the remaining rent should be given to the owner of this contract.
        // This is because we don't set shares for the owner of this contract while minting
to save gas.
        // also there might be a case of uneven shares after dividing, so the remaining
rent (1 to 9 wei)
        // is given to the owner for simplicity. if we try to log the remaining rent, it
will be lost in gas fees
        // and not worth it.
        if (sharedRents != rentAmount) {
            uint256 notSplitedRents = rentAmount - sharedRents;
            nonSharesRents += notSplitedRents;
        }

        emit RentSplited(tokenId, rentAmount);
    }

/**
 * Allows a token holder to withdraw their accumulated rent for a specific token.
 * @param tokenId The ID of the token.
 */
function withdrawRent(uint256 tokenId) external {
    // Retrieve the rent amount for the token holder
    uint256 rentMoney = tokenIdShareHolders[tokenId][
        tokenIdShareHoldersIndex[tokenId][msg.sender] - 1
    ].rents;

    // Set the rent amount to zero to prevent double withdrawal (reentrancy attack)
    tokenIdShareHolders[tokenId][
        tokenIdShareHoldersIndex[tokenId][msg.sender] - 1
    ].rents = 0;

    // Transfer the rent amount to the token holder
    payable(msg.sender).sendValue(rentMoney);

    emit RentWithdrawn(tokenId, msg.sender, rentMoney);
}

function withdrawNonSharesRents() external onlyOwner {
    uint256 rentMoney = nonSharesRents;
    nonSharesRents = 0;
    payable(msg.sender).sendValue(rentMoney);
}

// function to recieve ether
receive() external payable {
    emit Received(msg.sender, msg.value);
}

// The following functions are overrides required by Solidity.

function _update(
    address from,
    address to,
    uint256[] memory ids,
    uint256[] memory values
) internal virtual override(ERC1155, ERC1155Supply) {
    super._update(from, to, ids, values);

    // update the share holders
    uint256 len = ids.length;
    for (uint256 i; i != len; ) {
        uint256 tokenId = ids[i];

        if (from == address(0) && to == owner()) {
            // skip for owner minting to save gas
            unchecked {
                ++i;
            }
            continue;
        }

        // check if token is locked due to a proposal
        if (isTokenLocked(tokenId)) {
            revert TokenLocked(tokenId);
        }

        uint256 value = values[i];

```

```

uint256 fromShareIndex = tokenIdShareHoldersIndex[tokenId][from];
if (fromShareIndex != 0) {
    uint256 index = tokenIdShareHoldersIndex[tokenId][from] - 1;
    tokenIdShareHolders[tokenId][index].share -= value;
}

if (to == address(0)) {
    // burn
    unchecked {
        ++i;
    }
    continue;
}

if (tokenIdShareHoldersIndex[tokenId][to] > 0) {
    uint256 index = tokenIdShareHoldersIndex[tokenId][to] - 1;
    tokenIdShareHolders[tokenId][index].share += value;
} else {
    tokenIdShareHolders[tokenId].push(
        ShareHolders({holder: to, share: value, rents: 0})
    );
    tokenIdShareHoldersIndex[tokenId][to] = tokenIdShareHolders[
        tokenId
    ].length;
}

unchecked {
    ++i;
}
}
}
}

```

پیوست ت: کدهای قرارداد هوشمند FractionsDAO.sol یا FD

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";

abstract contract FractionsDAO is ERC1155 {
    uint256 public proposalsId;

    // Mapping of proposalIds to proposals
    mapping(uint256 => Proposal) public proposals;
    mapping(uint256 => mapping(address => bool)) public hasVoted;

    // write a mapping for token id to a number of active proposals
    // so we can revert transfer if there are active proposals
    mapping(uint256 tokenId => uint256 lock) public activeProposals;

    /// Token ownership is required for the operation
    error TokenOwnershipRequired();

    /// Token is locked due to an active proposal
    error TokenLocked(uint256 tokenId);

    /// Proposal has already been executed
    error ProposalAlreadyExecuted(uint256 proposalId);

    /// Proposal has already been rejected
    error ProposalAlreadyRejected(uint256 proposalId);

    /// Proposal has not passed
    error ProposalNotPassed(uint256 proposalId);

    /// User has already voted for this proposal
    error AlreadyVoted(uint256 proposalId, address voter);

    /// Voting period has ended
    error VotingPeriodEnded(uint256 proposalId, uint256 currentTimestamp);

    /// Proposal execution failed
    error ProposalExecutionFailed(uint256 proposalId, bytes data);

    /// Address has no code
    error AddressEmptyCode(address target);

    event ProposalSubmitted(
        uint256 indexed proposalId,
        uint256 indexed tokenId,
        address indexed proposer,
        string description
    );
    event Voted(uint256 indexed proposalId, address voter, bool vote);
    event ProposalExecuted(uint256 indexed proposalId, bytes data);
    event ProposalPassed(uint256 indexed proposalId);
    event ProposalRejected(uint256 indexed proposalId);

    struct Proposal {
        uint256 id; // Unique proposal ID
        uint256 tokenId; // Related ERC1155 token ID
        address proposer; // Proposal creator
        uint256 voteThreshold; // Minimum voting power required to pass or reject the
proposal
        uint256 voteEndTimestamp; // Timestamp when voting ends
        address targetAddress; // Address of the contract to call
        bytes data; // Contract call data for the proposal
        bool passed; // Whether the proposal has passed
        bool rejected; // Whether the proposal has been rejected
        bool executed; // Whether the proposal has been executed
        uint256 votesFor; // Total votes for the proposal
        uint256 votesAgainst; // Total votes against the proposal
        string description; // Description of the proposal
    }

    /**
     * Submits a new proposal to the DAO.
     * @param tokenId The ID of the token owned by the proposer.

```

```

    * @param voteThreshold The minimum number of votes required for the proposal to pass
    or reject.
    * @param targetAddress The address of the contract or account that the proposal is
    targeting.
    * @param data The data to be executed if the proposal is approved.
    * @param description A description of the proposal.
    * @param voteEndTimestamp The timestamp indicating when the voting period for the
    proposal ends.
    * @return The ID of the new proposal.
    */
    function submitProposal(
        uint256 tokenId,
        uint256 voteThreshold,
        address targetAddress,
        bytes calldata data,
        string calldata description,
        uint256 voteEndTimestamp
    ) public returns (uint256) {
        // Check if the proposer owns the specified tokenId
        if (balanceOf(msg.sender, tokenId) == 0)
            revert TokenOwnershipRequired();

        uint256 proposalId = proposalsId;

        // if targetAddress is not a contract, revert
        if (targetAddress.code.length == 0) revert AddressEmptyCode(targetAddress);

        proposals[proposalId] = Proposal({
            id: proposalId,
            tokenId: tokenId,
            proposer: msg.sender,
            voteThreshold: voteThreshold,
            voteEndTimestamp: voteEndTimestamp,
            passed: false,
            rejected: false,
            data: data,
            votesFor: 0,
            votesAgainst: 0,
            targetAddress: targetAddress,
            executed: false,
            description: description
        });

        ++proposalsId;

        activeProposals[tokenId]++;

        emit ProposalSubmitted(proposalId, tokenId, msg.sender, description);

        return proposalId;
    }

    /**
    * Allows a user to cast their vote on a proposal.
    * @param proposalId The ID of the proposal.
    * @param vote_ The vote (true for 'yes', false for 'no').
    * Requirements:
    * - The proposal must not have been executed.
    * - The user must not have already voted for this proposal.
    * - The voting period must not have ended.
    * - The user must have a sufficient token balance for voting.
    * Emits a {Voted} event.
    * If the proposal receives enough votes, it will either pass or be rejected.
    * If the proposal passes, it emits a {ProposalPassed} event and decreases the count of
    active proposals for the token.
    * If the proposal is rejected, it emits a {ProposalRejected} event and decreases the
    count of active proposals for the token.
    */
    function castVote(uint256 proposalId, bool vote_) public {
        Proposal storage proposal = proposals[proposalId];

        if (proposal.executed) revert ProposalAlreadyExecuted(proposalId);

        if (hasVoted[proposalId][msg.sender])
            revert AlreadyVoted(proposalId, msg.sender);

        if (block.timestamp > proposal.voteEndTimestamp)

```

```

        revert VotingPeriodEnded(proposalId, block.timestamp);

    uint256 balance = balanceOf(msg.sender, proposal.tokenId);

    if (balance == 0) revert TokenOwnershipRequired();

    if (vote_) {
        proposal.votesFor += balance;
    } else {
        proposal.votesAgainst += balance;
    }

    emit Voted(proposalId, msg.sender, vote_);

    hasVoted[proposalId][msg.sender] = true;

    uint256 voteThreshold = proposal.voteThreshold;

    // Check if the proposal has passed
    if (!proposal.rejected && !proposal.passed) {
        if (proposal.votesFor >= voteThreshold) {
            proposal.passed = true;
            emit ProposalPassed(proposalId);
            activeProposals[proposal.tokenId]--;
        } else if (proposal.votesAgainst >= voteThreshold) {
            proposal.rejected = true;
            emit ProposalRejected(proposalId);
            activeProposals[proposal.tokenId]--;
        }
    }
}

/**
 * Executes a passed proposal.
 * @param proposalId The ID of the proposal to be executed.
 * Requirements:
 * - The proposal must have passed.
 * - The proposal must not have been executed.
 * - The proposal must not have been rejected.
 * Emits a {ProposalExecuted} event.
 */
function executeProposal(uint256 proposalId) public {
    Proposal storage proposal = proposals[proposalId];

    if (proposal.executed) revert ProposalAlreadyExecuted(proposalId);
    if (proposal.rejected) revert ProposalAlreadyRejected(proposalId);
    if (!proposal.passed) revert ProposalNotPassed(proposalId);

    // to prevent re-entrancy
    proposal.executed = true;

    /// calls should be on behalf of the fractions contract
    (bool success, bytes memory data) = proposal.targetAddress.call(
        proposal.data
    );
    if (!success) revert ProposalExecutionFailed(proposalId, data);

    emit ProposalExecuted(proposalId, data);
}

// function to check if a token is locked
function isTokenLocked(uint256 tokenId) public view returns (bool) {
    return activeProposals[tokenId] != 0;
}
}

```

پیوست ث: کدهای قرارداد هوشمند CM یا ChargeManagement.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/utils/Address.sol";

import "../BuildingManagerElection.sol";
import "../FractoRealNFT.sol";

contract ChargeManagement is BuildingManagerElection {
    using Address for address payable;

    /// Only the building manager can call this function
    error OnlyBuildingManager();
    /// Insufficient fee amount
    error InvalidFeeAmount();
    /// Spend fee failed
    error SpendFailed();
    /// event for fee payments
    event FeePaid(uint256 indexed tokenId, address payer, uint256 amount);
    /// event for fee spending
    event FeeSpent(address to, uint256 amount);
    /// event for fee amount change
    event FeeAmountChanged(uint256 newFeeAmount);

    FractoRealNFT public immutable erc721;

    address public buildingManager;
    uint256 public feeAmount;

    mapping(uint256 tokenId => uint256 amount) public paidFees;

    modifier onlyBuildingManager() {
        if (msg.sender != buildingManager) revert OnlyBuildingManager();
        _;
    }

    constructor(FractoRealNFT erc721_) {
        erc721 = erc721_;
    }

    /**
     * Allows a resident or unit owner to pay the fee for a specific token ID.
     * @param tokenId The ID of the token for which the fee is being paid.
     */
    function payFee(
        uint256 tokenId
    ) external payable onlyResidentOrUnitOwner(tokenId) {
        if (msg.value != feeAmount) revert InvalidFeeAmount();

        paidFees[tokenId] += msg.value;

        emit FeePaid(tokenId, msg.sender, msg.value);
    }

    /**
     * @dev Transfers a specified amount of Ether to the given address.
     * Only the building manager is allowed to call this function.
     * Emits a `FeeSpent` event after the transfer is completed.
     *
     * @param amount The amount of Ether to be transferred.
     * @param to The address to which the Ether will be transferred.
     */
    function spendFee(uint256 amount, address to) external onlyBuildingManager {
        emit FeeSpent(to, amount);

        payable(to).sendValue(amount);
    }

    /**
     * Sets the fee amount for the building.
     * Can only be called by the building manager.
     * Emits a `FeeAmountChanged` event with the new fee amount.
     * @param newFeeAmount The new fee amount to be set.
     */
}
```

```

function setFeeAmount(uint256 newFeeAmount) external onlyBuildingManager {
    feeAmount = newFeeAmount;

    emit FeeAmountChanged(newFeeAmount);
}

function setBuildingManager(address newBuildingManager) internal override {
    buildingManager = newBuildingManager;
}

function getErc721() public view virtual override returns (FractoRealNFT) {
    return erc721;
}
}

```

پیوست جیم: کدهای قرارداد هوشمند BME یا BuildingManagerElection.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.20;

import "./FractoRealNFT.sol";

abstract contract BuildingManagerElection {
    /// Invalid Status
    error InvalidStatus(ElectionState current, ElectionState expected);
    /// Voting has ended
    error VotingHasEnded();
    /// Voting is not Ended
    error VotingNotEnded();
    /// Already voted
    error AlreadyVoted(address voter);
    /// Does not own ERC721
    error DoesNotOwnERC721();
    /// Candidate is not registered
    error CandidateNotRegistered();
    /// Candidate already registered
    error CandidateAlreadyRegistered();
    /// Contract call not allowed
    error ContractCall();
    /// Only resident or unit owner can call this function
    error OnlyResidentOrUnitOwner();

    event StateChanged(ElectionState newState);
    event CandidateRegistered(address candidate);
    event Voted(uint256 indexed tokenId, address indexed voter, address indexed candidate);
    event BuildingManagerElected(address newBuildingManager);

    enum ElectionState {
        NotStarted,
        acceptCandidates,
        Voting
    }

    struct Candidate {
        address candidate;
        uint256 voteCount;
        bool isRegistered;
    }

    modifier noContract() {
        if (tx.origin != msg.sender) revert ContractCall();
        _;
    }

    ElectionState private _electionState;

    mapping(uint256 tokenId => address voter) public voters;

    address[] private _candidateList;
    mapping(address => Candidate) public candidates;

    // timestamp when voting ends
    uint256 public votingEnd;

    modifier onlyResidentOrUnitOwner(uint256 tokenId) {
        FractoRealNFT erc721 = getErc721();
        if (
            erc721.ownerOf(tokenId) != msg.sender &&
            erc721.residents(tokenId) != msg.sender
        ) revert OnlyResidentOrUnitOwner();
        _;
    }

    function _ownsERC721(address owner) private view returns (bool) {
        return _erc721TokenCount(owner) > 0;
    }

    /**
     * Registers a candidate for the building manager election.
     * Only callable when the election state is set to acceptCandidates.
     * The caller must own an FRN (ERC-721) token.
     */
}
```

```

    * Emits a {CandidateRegistered} event upon successful registration.
    */
function registerCandidate() public noContract {
    ElectionState electionState = _electionState;

    if (electionState != ElectionState.acceptCandidates)
        revert InvalidStatus(electionState, ElectionState.acceptCandidates);

    address candidate_ = msg.sender;

    if (!_ownsERC721(candidate_)) revert DoesNotOwnERC721();

    Candidate storage candidate = candidates[candidate_];
    if (candidate.isRegistered) revert CandidateAlreadyRegistered();

    candidate.candidate = candidate_;
    candidate.isRegistered = true;
    _candidateList.push(candidate_);

    emit CandidateRegistered(candidate_);
}

function startElection() public {
    ElectionState electionState = _electionState;
    if (electionState != ElectionState.NotStarted)
        revert InvalidStatus(electionState, ElectionState.NotStarted);

    _electionState = ElectionState.acceptCandidates;

    emit StateChanged(ElectionState.acceptCandidates);
}

function startVoting(uint256 votingEnd_) public {
    ElectionState electionState = _electionState;

    if (electionState != ElectionState.acceptCandidates)
        revert InvalidStatus(electionState, ElectionState.acceptCandidates);

    _electionState = ElectionState.Voting;
    votingEnd = votingEnd_;

    emit StateChanged(ElectionState.Voting);
}

/**
 * Allows a resident or unit owner to cast a vote for a candidate.
 * @param tokenId The token ID representing the resident or unit owner.
 * @param candidate_ The address of the candidate being voted for.
 * @notice Only residents or unit owners can cast votes.
 * @notice The voting must be in the "Voting" state.
 * @notice The voting must not have ended.
 * @notice The voter must not have already voted.
 * @notice The candidate must be registered.
 * @notice Emits a {Voted} event with the token ID, voter's address, and candidate's
address.
 */
function castVote(
    uint256 tokenId,
    address candidate_
) public onlyResidentOrUnitOwner(tokenId) {
    // Get the current election state and check if the election state is not in the
    "Voting" state
    ElectionState electionState = _electionState;
    if (electionState != ElectionState.Voting)
        revert InvalidStatus(electionState, ElectionState.Voting);

    // Check if the voting has ended
    if (block.timestamp > votingEnd) revert VotingHasEnded();

    // Get the address of the voter and check if the voter has already voted
    address voter = voters[tokenId];
    if (voter != address(0)) revert AlreadyVoted(voter);

    // Check if the candidate is registered
    Candidate storage candidate = candidates[candidate_];
    if (!candidate.isRegistered) revert CandidateNotRegistered();
}

```

```

        // Increment the vote count for the candidate
        ++candidate.voteCount;

        // Update the voter's address in the voters mapping for preventing double voting
        voters[tokenId] = msg.sender;

        // Emit a Voted event with the token ID, voter's address, and candidate's address
        emit Voted(tokenId, msg.sender, candidate_);
    }

    /**
     * Finalizes the election and determines the winning candidate as the building manager.
     * This function can only be called when the election is in the "Voting" state and the
     * voting period has ended.
     * It finds the candidate with the most votes and sets them as the building manager.
     * After setting the building manager, it resets the voting state, clears the candidate
     * list, and resets the voter vote count.
     * Finally, it emits the {StateChanged} and {BuildingManagerElected} events.
     */
    function finalizeElection() public {
        // Get the current election state and check if the election state is not in the
        "Voting" state
        ElectionState electionState = _electionState;
        if (electionState != ElectionState.Voting)
            revert InvalidStatus(electionState, ElectionState.Voting);

        // Check if the voting has ended
        if (block.timestamp < votingEnd) revert VotingNotEnded();

        // Find the candidate with the most votes
        uint256 maxVotes = 0;
        Candidate storage winningCandidate = candidates[_candidateList[0]];
        uint256 length = _candidateList.length;
        for (uint256 i; i != length; ) {
            Candidate storage candidate = candidates[_candidateList[i]];
            uint candidateVotes = candidate.voteCount;

            if (candidateVotes > maxVotes) {
                maxVotes = candidateVotes;
                winningCandidate = candidate;
            }

            unchecked {
                ++i;
            }
        }
        address winner = winningCandidate.candidate;

        // Set the winning candidate as the building manager
        setBuildingManager(winner);

        // Reset voting state
        _electionState = ElectionState.NotStarted;

        // Clear candidate mapping and list
        for (uint256 i; i != length; ) {
            delete candidates[_candidateList[i]];
            unchecked {
                ++i;
            }
        }

        delete _candidateList;

        // reset voters mapping for each token ID
        length = getErc721().totalSupply();
        for (uint256 i; i != length; ) {
            delete voters[i];
            unchecked {
                ++i;
            }
        }

        // reset voting end timestamp
        votingEnd = 0;

        emit StateChanged(ElectionState.NotStarted);
    }

```

```

        emit BuildingManagerElected(winner);
    }

    function _erc721TokenCount(address owner) private view returns (uint256) {
        return getErc721().balanceOf(owner);
    }

    function setBuildingManager(address newBuildingManager) internal virtual;

    function getErc721() public view virtual returns (FractoRealNFT);
}

```

Abstract

Real estate investment has long been considered a valuable asset due to its potential to generate a steady stream of income. The emergence of blockchain technology and smart contracts has led to the tokenization of assets, which can increase liquidity, facilitate access for small investors, and enhance transparency. However, previous research, despite its limited number, has primarily focused on a single aspect of blockchain applications in real estate, often neglecting challenges such as co-ownership asset management and the collection and management of rental and service fees for multi-ownership units. This research addresses these limitations by presenting a comprehensive approach to developing and implementing a real estate investment management system. The main innovation of the research is the combined use of different token standards to represent and manage assets. The system includes features such as full and fractional tokenization of assets and provision of individual and shared ownership, management of shared units through a decentralized autonomous organization (DAO), comprehensive rental and service fees management, and transparent building manager elections on the blockchain. The findings indicate that this system can effectively increase liquidity and access to the real estate market and reduce transaction costs by reducing the need for intermediaries. Comprehensive testing and associated evaluations ensure the correctness of the proposed system's performance and cost-effectiveness. Additionally, the use of blockchain enhances transparency and trust in the market and provides efficient solutions to existing problems. This research demonstrates that real estate tokenization with blockchain can transform the traditional real estate market.

Keywords: Blockchain, Tokenization, Smart Contract, NFT, Ethereum, Real Estate



ISLAMIC AZAD UNIVERSITY

MASHHAD BRANCH

Faculty of Engineering - Department of Computer Engineering

Thesis for receiving «M.Sc» degree on Software Engineering

Subject:

Management of Real Estate Investments on the Blockchain

Platform based on Smart Contract and NFT

Thesis Advisor:

D. Bahrepour Ph.D.

Consulting Advisor:

SR. Kamel Tabbakh Ph.D.

By:

Amir Hossein Banavi

Summer 2024